Haver Python Reference

Release 3.5.0

Haver Analytics

June 25, 2025

CONTENTS

CHAPTER

ONE

OUTLINE OF VERSION IMPROVEMENTS

- v3.5.0, Jun 2025
  - Advanced functions: All functions from the advanced functions dialog box in VG3 plus seasonal adjustment are now also available in Python.
  - The contents of the deprecation warning listed below under v3.4.0 regarding PeriodDType='B' is now in effect. dataframes in Haver-daily frequency now possess a DatetimeIndex (with pandas frequency 'B').
- v3.4.0, Nov 2024
  - **Deprecation warning**: Following the deprecation in pandas of PeriodDType='B' for PeriodIndexes, future versions of the Haver package for Python will return dataframes in Haver-daily frequency (which correspond to pandas' 'B') with a DatetimeIndex instead of a PeriodIndex. Dataframes with other frequencies, including calendar daily, will continue to use PeriodIndexes.

    Please adjust your code accordingly, if necessary.

    This change is likely to happen with Haver package version 3.5.0.
  - Basic functions: 9 functions consisting of percent changes, differences, and log differences can now be specified. In other DLX software, these functions are referred to as 'basic functions'. Functions can be specified in code lists for individual codes or as a default function via a separate argument. In the context of this enhancement, the following changes to Haver package functions have been made:
    - `Haver.metadata()`, `Haver.data()`, and `Haver.fmtcodes()` receive a new argument `function=None` for specifying a default function applied to the input code list.
    - `Haver.fmtcodes()` and `Haver.codelists()` receive a new argugment `stripfunc=False` for stripping function specifications from output code lists.
    - `Haver.fmtcodes()` additionally receives a new argument `separate=False` for returning lists of individual code components instead of a single list of formatted codes.
  - `Haver.data()` receives a new argument `underscore=True`, which allows to use *db:ser* and *ser@db* formats in the column labeling of Haver DataFrames instead of using the default underscore as separator.
  - `Haver.fmtcodes()` from now on only returns code lists in lower case. Previously, it did not change the casing of the input list. This change has been made to be in line with the general package principle of always returning strings in lower case. **Warning**: This is a potentially code breaking change.
  - The Haver package code has been prepared for the release of pandas 3.0.0. Notably, the string dtype of Haver Meta-DataFrames will continue to be 'object' and not 'string[pyarrow_numpy]'.
- v3.3.0, Aug 2024
  - New series code formats functionality:
    - In addition to the *database:series* code format, functions now understand the *series@database* code format, as used in Excel. Mixtures of the two formats are also allowed.

    - To control the series code format of output lists, a new argument `codeformat` has been added to functions `Haver.data()`, `Haver.metadata()`, `Haver.codelists()`, and `Haver.dbcodes()`. Existing argument `format` of `Haver.dbcodes()` continues to work, but newly written code should use the newly introduced `codeformat` argument.
    - New function `Haver.fmtcodes()` takes a list of Haver codes and returns a lists of Haver codes, formatted according to its `codeformat` argument.
  - Haver.metadata() received, in analogy to Haver.data(), a new arg `rtype='havermetadataframe'`. The alternative to the default is '2tuple', which returns a two-element tuple containing, first, a plain pandas dataframe and, second, a haverinfo dict.
  - internal changes
- v3.2.3, Feb 2024
  - The number of rows that a queried dataframe can have is now unrestricted. There used to be a limit of 18,900 rows.
  - internal changes
- v3.2.2, Oct 2023
  - internal changes
  - The minimum required Python version has increased from 3.5 to 3.6.
- v3.2.1, Feb 2023
  - internal changes only
- v3.2.0, Nov 2022
  - New warning messages are issued by `Haver.data()` when querying daily or calendar daily series. Local queries will warn you whenever a calendar daily (7-day) series is treated as a regular daily (5-day) series; and they will indicate the exact number of series that are concerned. DLX Direct queries will issue a generic warning message whenever daily or calendar daily series are queried under `caldaily=False`.
  - Bug fix: Under v3.1.0, `Haver.metadata()` and `Haver.data()` accidentally wrote a logging text file called 'c.log' to the current Python working directory. This as been fixed.
  - other internal changes
- v3.1.0, May 2022
  - new calendar daily frequency
    - Calendar daily series contain seven data points per week (regular Haver daily series contain five data points per week). In order to not break your existing scripts/code, calendar daily series are treated as regular daily series by default.
    - `Haver.metadata()` has new parameters `caldaily=False` and `wecodes='keep'`
    - `Haver.data()` has new parameter `caldaily=False`
- v3.0.1, Nov 2021
  - internal changes only
- v3.0.0, Sep 2021
  - DLX Direct support has been added. Clients with an active DLX Direct subscription can access Haver databases over the internet.

    New function `Haver.direct()` turns DLX Direct mode on and off.
  - New function `Haver.dbcodes()` returns a list of all codes in a database.

  - Speed improvements: Haver.metadata() runs in about 80% of the original time, and Haver.data() in about 40% of the original time.
  - New function `Haver.usermode()` provides an alternative database access mode. It should only be used if a technical support person asks you to do so.
- v2.1.1, Feb 2021
  - internal changes only
- v2.1.0, Dec 2020
  - As an alternative to specifying a fixed database path for all databases, individual database paths can be set in a variety of ways. This includes usage of the information in the main database location ('INI') file of the DLX intallation.
- v2.0.1, Oct 2020
  - Fixed problems related to DLL loading.
- v2.0.0, Apr 2020
  - Haver Data Manager (HDM) functionality has been added. Clients with an HDM subscriptions can maintain their own data in DLX databases.
  - `Haver.path()` has a new keyword 'hdm'. This is related to the new HDM functionality.
  - `Haver.data()` has new return types '2tuple' and '3tuple', specified via parameter `rtype`.
  - `Haver.data()` has a new parameter `codeformat` which can be used to either return simple Haver series codes only, or return full Haver series codes only.
  - Error messages have changed very slightly. **Warning**: This is a potentially code breaking change. See section *Error Trapping*. Error message identifiers have remained stable.
  - The minimum required Python version has increased from 3.4 to 3.5. Minimum dependency versions have increased to pandas 0.17.0 and numpy 1.10.1.
- v1.1.0, Sep 2018
  - internal changes only
  - memory leak bug fix
- v1.0.1, Aug 2018
  - internal changes only
- v1.0.0, May 2017
  - first released package version

CHAPTER

TWO

THE HAVER PACKAGE

Query data and metadata from Haver Analytics databases

Author: Haver Analytics

License: Provided under DLX Subscription Agreement according to the Agreement Definition of MATTER

If you are new to the Haver package, we recommend consulting Section *Introduction to the Haver Package for Python* of the Haver Python Reference available in the client section of http://www.haver.com. If you want to jump straight to function documentation, see the help entries for *Haver.data* and *Haver.metadata*.

2.1 See Also

All functions and objects related to Haver queries:

- *Haver.data()* : query time series data from Haver Analytics databases
- *Haver.metadata()* : query time series metadata from Haver Analytics databases
- *Haver DataFrame* : return type of `Haver.data()`
- *Haver Meta-DataFrame* : return type of `Haver.metadata()`
- *Haver ErrorReport dictionary* : return type for incorrectly specified queries
- *Haver.path()* : set and query the path to Haver databases
- *Haver.databases()* : print or return list of Haver Analytics databases
- *Haver.dbcodes()* : obtain a list of codes contained in a Haver Analytics database
- *Haver.limits()* : set and get limits for Haver Analytics data queries
- *Haver.codelists()* : extract lists of Haver series codes
- *Haver.fmtcodes()* : format lists of Haver series codes
- *Haver.verbose()* : set verbose message mode of Haver functions to `True` or `False`
- *Haver.direct()* : choose between local and internet access of Haver Analytics databases
- *Haver.usermode()* : set and query the user mode

If your institution subscribes to the *Haver Data Manager (HDM) tool* for creating and modifying your own DLX databases, you may also be interested in:

- *Haver.hdm.create()* : create new groups or series, or overwrite existing groups or series in an HDM database
- *Haver.hdm.delete()* : delete groups or series from an HDM database
- *Haver.hdm.update()* : update time series data in an HDM database

- *Haver.hdm.inputframes()* : create Input-Frame(s) for groups or metadata or data
- *Haver.hdm.path()* : set and query the path to HDM databases
- *Haver.hdm.groups()* : query group information from a database
- *Haver.hdm.exampledb()* : prepare the HDM example database for a particular example section
- *Haver.hdm.exampledata()* : create input data that can immediately be used for database content modification
- *Haver.hdm.apipath()* : apply a custom setting of the HDM API DLL path, if necessary

CHAPTER

THREE

INTRODUCTION TO THE HAVER PACKAGE FOR PYTHON

3.1 Description

This help entry provides an introductory overview of functions that allow you to access data stored in Haver Analytics databases from within Python. It also defines terminology and conventions that are relevant for the entire documentation of the package.

3.2 Getting to Know the Haver Package Quickly

The basic operations of Haver data queries are easy and can be learned quickly. In a nutshell, the functions that access Haver Analytics databases are called `Haver.data()` and `Haver.metadata()` . Both take as arguments specifications for databases and time series codes to be retrieved. Data and metadata are returned in pandas DataFrames. These DataFrames will be referred to as Haver DataFrames and Haver Meta-DataFrames. Data and metadata are linked closely: A time series retrieval results in a DataFrame object that has a Meta-DataFrame attached to it, which can be accessed at any time. Conversely, a Haver Meta-DataFrame resulting from a metadata query can be used as an input argument to `Haver.data()` in order to query the corresponding time series data.

This help entry and some other help entries for the Haver package are somewhat lengthy, but this is not because usage is complicated. Rather, they provide much detail that can be skipped by beginning users but that may become useful at a later stage.

If you want to get a very quick introduction, read (sub-)sections *Where to Go from Here*, and *Examples* of this help entry, and do take note of the other section and subsection headings so you know where to look up information at a later point.

The other sections than the ones just mentioned define terminology and/or provide information for efficiently gaining a deeper understanding of the package. If you decide to only skim through them for now, we recommend that you do not postpone a more thorough reading for too long.

3.3 Preliminaries

3.3.1 DLX, Access to Databases, Operating System

'Haver' is short for the company name Haver Analytics, whereas 'DLX' refers to a software bundle that is distributed by Haver Analytics ('DLX' is short for 'Data Link Express'). DLX comprises all software components that update and provide access to Haver Analytics databases. For the practical purposes of this help file this distinction does not matter and you can read 'Haver' and 'DLX' interchangeably. This package uses the term 'Haver' mostly when referring to databases, series codes, etc., related to Haver Analytics or the DLX software.

Accessing Haver databases requires that you have access to Haver databases on a local or on a network drive. You do not need to have any Haver desktop client software installed.

3.3.2 Requirements

The Haver package is only available on Windows operating systems.

Python dependencies are pandas and numpy.

3.3.3 Conventions of Haver Functions and Objects and of Haver Help Files

Usage of Terms 'code', 'symbol', 'variable', etc.

Codes of Haver time series are referred to as series codes, Haver codes, or Haver series codes. The time series themselves are referred to as Haver series or Haver time series.

To unambiguously identify a Haver time series, you must indicate the series code and the database name. The term series code may or may not imply this full specification. If it should be stressed that a full specification is necessary, the word full is prepended to 'Haver code', 'series code', etc. If this is done using the form: database name - colon - series code, the specification is said to be in *database:seriescode* format, or *db:ser* for short. Conversely, if it should be stressed that a series code specification does NOT include the database name, the word 'simple' is prepended to 'Haver code', 'series code', etc.

The following table summarizes this information, with examples of full series codes given in *database:seriescode* format.

term	example
Haver code, series code, Haver series code	gdp, usecon:gdp
full Haver code, full series code, etc.	usecon:gdp
simple Haver code, etc.	gdp

Side note: If a simple series code exists in two databases, it usually refers to the same concept, potentially with differences in frequencies (e.g. 'daily:ffed' and 'weekly:ffed'). Cases where a simple series code exists in multiple databases and refers to different concepts are very rare.

The term 'variable' is never used for Haver series in order to avoid ambiguity. It is only used to refer to Python workspace variables and to the columns of a DataFrame object.

Haver codes can be expanded by function names, for example, to indicate the calculation of a growth rate. To avoid ambiguity of the term 'function', such functions will be referred to as 'math functions' (as opposed to functions of the Haver package, like `Haver.data()`).

Series Code Formats

In addition to the *database:seriescode* format introduced in the previous section, the Haver package for Python understands the *seriescode@database* format (or *ser@db*, for short) that is, e.g., used in the DLX add-in for Excel. Whenever you supply a list of series codes to a package function, codes can be either in *db:ser* format or *ser@db* format, or they can even be a mixture of both. The format used in output lists can be controled by the `codeformat` argument that many functions possess. The default format for output lists is typically *db:ser*. If you want them to be in *ser@db* format, you have to make that request via the `codeformat` argument.

This guide uses both the *ser@db* and the *db:ser* formats.

The application of a math function to a series, such as a growth rate, is indicated as in DLXVG3 and Excel: The series code appears in parentheses, preceded by the function name, and possibly supplemented by an argument, which is separated by a comma; examples: 'diff%(usecon:gdp)', 'index(usecon:gdp,2000=100)'. Such codes are called 'function codes'. Function codes typically appear in full format, but sometimes also in simple format, so all of the formats *db:ser*, *ser@db*, and *ser* are allowed.

A 'Haver code list' or simply 'code list' is a list of Haver series codes, in any format, including mixed formats, and with or without functions.

Casing

Function names of Haver functions are always lower case. As a convention in all of the documentation the Haver package is imported by the statement

```
>>> import Haver
```

Since the Python language is case-sensitive, the function `data()` of the Haver package is called by

```
>>> Haver.data([...])
```

If you do not like the capital 'H' in 'Haver' in the above statement, you may import the package as

```
>>> import Haver as haver
```

You could also import the package under an entirely different alias, e.g.

```
>>> import Haver as hv
```

We recommend that you do not do so, however. The import aliases 'Haver' and 'haver' are consistent with function names of DLX interfaces to other analytical software, so using the aliases 'Haver' and 'haver' produces statements that are more portable.

Arguments to Haver functions are never case-sensitive. In particular, you can specify Haver series codes in upper case or lower case or any mixture thereof.

Almost all character-based return values of Haver functions are lower case. In particular, Haver series codes or database names returned by Haver functions are always lower case. The field names of a Haver Meta-DataFrame are also lower case. The exception to the 'return character values in lower case' convention of Haver functions are the values of some fields of Haver Meta-DataFrames, as well as the database list returned by `Haver.databases()`.

Argument Abbreviations

Most values given as arguments to Haver functions can be abbreviated. For example, in order to specify the output frequency of a data set returned by `Haver.data()`, you can use `frequency='q'` instead of `frequency='quarterly'`.

Example Code Variable Names

The most important objects of the Haver package are *Haver DataFrame* and *Haver Meta-DataFrame*, to be described below and in other help entries. Whenever an example Python code statement in a help entry assigns such an object to a workspace variable, the names 'hd' and 'hmd' are used, possibly with suffixes that are easily understood within the context ('hd_1', 'hd_2', 'hmd_d', etc.). Dictionaries that help files refer to as *Haver ErrorReport dictionaries* are called 'her'. Any other Python variables that are defined in example code start with 'ex_'.

Date Format

Dates are always in the format yyyy-MM-dd. For example, 24 March 2014 is denoted 2014-03-24.

Missing Values

Missing values for time series data are frequently referred to as 'NaN' ('not a number'). In data sets missing values are recorded as `numpy.nan`.

3.4 DLX Direct: Accessing Databases Directly from a Haver Server

While this guide is mainly written in terms of DLX queries to local databases, the Haver package for Python also integrates with DLX Direct, which is a Haver subscription service that enables you to connect to Haver Analytics servers for database access. Depending on your institution's DLX subscriptions, you may be able to access only one of the two services, or both.

In Python you can switch between the two different modes of query with a single function call. If you would like to use DLX Direct in Python, make sure to give *Haver.direct* a thorough read.

3.5 Error Trapping

You can trap errors that pertain to the core workings of the Haver package as in

```
try:
    Haver.data('usecon:gdp', aggmode='WRONGARG')
except Haver._HaverPkgError:
    [...]
```

If you want to check for the specific cause of the error via the error message string, we recommend that you use only the short error identifiers that are placed within parentheses at the beginning of the message. For example, use

```
try:
    Haver.data('usecon:gdp', aggmode='WRONGARG')
except Haver._HaverPkgError as err:
    if '(inputargs:invArgValue)' in str(err):
        [...]
```

instead of

```
try:
    Haver.data('usecon:gdp', aggmode='WRONGARG')
except Haver._HaverPkgError as err:
    if '(inputargs:invArgValue) Argument \'aggmode\' incorrectly specified.' in str(err):
        [...]
```

This is because the short error identifiers are stable, while the rest of the error message may change, thus causing your code to break.

3.6 Concurrent Execution

Warning: The Haver package for Python is not thread-safe.

Do not use the Haver package in combination with e.g. the `threading` module in order to run multiple threads. If you try to do so, Python will respond with non-informative errors, or may even crash. If you want to run multiple queries in parallel, use multiple processes, e.g. via the `multiprocessing` module.

3.7 Preparatory Steps for Python Code Examples

3.7.1 Example Databases

The Haver package comes with three example databases that are used frequently in the 'Examples' section of Haver help entries: 'HAVERD', 'HAVERW', and 'HAVERMQA'. They contain daily, weekly, and monthly/quarterly/annual data, respectively. These databases are subsequently referred to as the 'example databases'.

Do not use data contained in example databases for analysis. They are shipped for demonstration purposes only. To ensure exact replicability of the examples, the data in the example databases is never changed and hence several years old.

The Haver databases that your institution subscribes to and that you can use in analysis are referred to as the 'actual databases' or 'actual Haver databases'. Keep in mind that, while the example databases contain only between 3 and 161 series, actual Haver databases contain thousands or even hundreds of thousands of series.

In rare cases, help files will refer to series in actual databases. Such references are only made outside of 'Examples' sections.

In addition to the three example databases mentioned above, a fourth example database, called 'HDMEX', is part of the Haver package for Python. This database is used only for examples related to the *Extension Functions for Modifying HDM Databases* and hence only of interest to you if your institution has an active subscription to the Haver Data Manager (HDM) service.

3.7.2 Setting the Haver Database Path to Actual and Example Databases

Before Haver database queries can be executed the correct database directory path has to be set. This is done with `Haver.path()`. In most cases you will not have to do this as `Haver.path()` tries to set the correct path to the actual Haver databases automatically.

The path to the example databases is different than the path to the actual databases. `Haver.path()` provides a convenient syntax shortcut for setting the path to example databases and for restoring the path to the actual databases. This is done by the statements

```
>>> Haver.path('examples')
>>> Haver.path('restore')
```

These statements enclose each 'Examples' section of Haver help entries. As an additional safeguard there is a statement

```
>>> Haver.path()
```

at the very top of each 'Examples' section which, if executed, displays the path setting that was in place before the 'Examples' section was entered.

Depending on the configuration of Python and of DLX on your machine, it may occur that calling `Haver.path('examples')` and `Haver.path('restore')` does not work. However, you can still set the path to the example databases manually in order to execute example statements.

To set the path to the example databases manually, first take note of the path setting to the actual databases, which is returned by

```
>>> Haver.path()
```

Then, execute

```
>>> Haver.path(EXAMPLE_PATH)
```

where EXAMPLE_PATH is the subdirectory 'dat' of the installation directory of the Haver package. For example, if you had installed the Haver package to 'c:\users\me\Python\lib', you would have to execute

```
>>> Haver.path('c:\users\me\Python\lib\Haver\dat')
```

If you do not know the installation path, you can display it by

```
import Haver, os
os.path.split(Haver.__file__)[0] + '\\dat'
```

Once you are done looking at the examples, manually set the path back to the actual Haver databases.

Note: The example databases reside in a subdirectory of the Haver package installation directory. Since, in general, you must have write access to the database directory in order to query data, querying example databases requires write access to the Haver package installation directory. If you do not have write access, you can copy the 'dat' directory to a location where you have write access, and set the Haver path to that directory manually.

3.7.3 DataFrame Display

Depending on your current pandas display settings, some output of the examples may not be printed to the screen fully. As an example, consider a Haver-Meta-DataFrame. It has 18 columns. If pandas' option `display.max_columns` is set to a lower value, you will not see the full output. You can query the current setting by

```
>>> import pandas as pd
>>> pd.get_option('display.max_columns')
```

It is easy to set the value to a more appropriate one if necessary:

```
>>> pd.set_option('display.max_columns', 20)
```

Other options that may help to optimize the display of Haver-DataFrames are: `display.max_colwidth`, `display.max_rows`, `display.width`, and `display.large_repr`. For more information, see the pandas documentation.

3.7.4 Turning DLX Direct Mode Off

Since the example databases reside on your local network, any example code must be executed with *DLX Direct* mode turned off:

```
>>> Haver.direct('off')
```

This statement is implicit in code examples. It is not shown explicitly. If you try to execute code examples with DLX Direct mode turned on, the example statements will fail.

3.8 Where to Go from Here

First, read and/or execute the Examples section below. It provides introductory examples to give you a first idea on how to use functions of the Haver package.

You can then go on to a more detailed exposition of the functions *Haver.metadata* and *Haver.data*. The help files for the two functions contain a lot of detail, too much detail probably for a beginning Haver package user. You absolutely do not have to read everything, but do take note of the (sub-) section headings in these help files so you know where to look up information at a later point. You should read the files thoroughly enough to be able to understand the 'Examples' sections of both files.

If at any point you have trouble retrieving data because of path settings, look at *Haver.path*.

The Haver package imposes artificial limits on the size of data queries. These limits may never become relevant for you. If they do, you can change the settings for these limits. Should a Haver routine notify you that a query is not possible because of query size limits, *Haver.limits* provides information on the customization of query limits.

3.9 Examples

This section provides introductory examples. It's purpose is to give you an overview of the Haver package. Detailed explanations are skipped at this point.

```
Haver.path()
Haver.path('examples')
```

Display current path setting:

```
print( Haver.path() )
```

Set the database path to the directory of the example databases and check contents:

```
Haver.path('examples')
print( Haver.databases() )
```

This directory contains three databases: HAVERD, HAVERW, and HAVERMQA. Function `Haver.metadata()` accesses metadata information of Haver databases. We use it here to get an overview of what is in the databases.

```
print( Haver.metadata(database='haverd') )
print( Haver.metadata(database='haverw') )
```

Actual Haver Analytics databases have thousands or even hundreds of thousands of series. In such and other cases, assigning the output to a variable is appropriate. As usual, when assigning to a variable no output is printed to the screen:

```
hmd_mqa = Haver.metadata(database='havermqa')
```

Variable `hmd_mqa` now holds a Haver Meta-DataFrame. Its data type is a pandas DataFrame. You can display the contents of the DataFrame and of a subset thereof in the Python console. pandas DataFrames provide many ways of indexing into the data. See the pandas documentation for details.

Let's pull in three exchange rate series from 'HAVERD'. `Haver.data()` is the function to retrieve time series data. We specify a starting date in order to limit the output.

```
hd_1 = Haver.data(['fxtwb', 'fxtwm', 'fxtwotp'],
       'haverd', startdate='2012-02-15')
```

`hd_1` is a pandas DataFrame:

```
print( type(hd_1) )
```

The metadata has automatically been saved within this object. It is accessible through the `haverinfo` member (dict) of the Haver DataFrame:

```
print( hd_1.haverinfo['metadata'] )
```

Alternatively, we can assign the Haver Meta-DataFrame to a variable and then customize the output:

```
hmd_1 = hd_1.haverinfo['metadata']
print( hmd_1.loc[:,['code', 'descriptor']] )
```

We can pull in data from more than one database using the *database:seriescode* format in the `codes` argument. Note that in the following query the daily series gets aggregated to weekly frequency. The frequency of the resulting DataFrame is weekly.

```
hd_2 = Haver.data(['haverD:fxtwb', 'haverW:farbn'],
           startdate='2012-01-01')
print( hd_2 )
```

Next, we pull in data from the entire 'HAVERMQA' database. To do this, we use the Haver Meta-DataFrame stored in `hmd_mqa` which we have created earlier and feed it into the data retrieval. Since the lowest frequency of the queried series is annual, series with higher frequencies get aggregated. `Haver.data()` also informs us that two series cannot be aggregated. They get dropped from the query.

```
hd_mqa  = Haver.data(hmd_mqa)
print( hd_mqa )
```

You can specify a higher frequency if you want, say, quarterly. Then `Haver.data()` drops annual series from the query. We look at the last 10 observations of the first 5 codes in the DataFrame:

```
hd_mqa2 = Haver.data(hmd_mqa, frequency='q')
print( hd_mqa2.iloc[-10:, 0:4] )
```

You can plot the data from the Haver DataFrame (requires Matplotlib):

```
import matplotlib.pyplot as plt
hd_mqa2.loc[:,['mm', 'ms']].plot()
plt.show()
```

```
>>> Haver.path('restore')
```

CHAPTER

FOUR

FUNCTIONS AND OBJECTS

4.1 Haver.data

`Haver.data`(*codes*, *database=None*, *startdate=None*, *enddate=None*, *frequency=None*, *rtype='haverdataframe'*, *aggmode='strict'*, *eop=True*, *dates=False*, *uselimits=True*, *codeformat=[':', 'auto']*, *caldaily=False*, *function=None*, *underscore=True*)

Description

`Haver.data()` queries time series data contained in Haver Analytics databases. The data are returned as a pandas DataFrame.

`Haver.hasfullcodes()` is a small helper function.

This help entry assumes that you are familiar with the Haver Python Reference Section *Introduction to the Haver Package for Python*.

Syntax

```
hd = Haver.data(codes)
hd = Haver.data(codes, database)
hd = Haver.data(codes, database, [...])
hd = Haver.data(codes, None    , [...])

Haver.hasfullcodes(x)
```

4.1.1 Arguments

codes

[str or tuple of str or list of str] codes is typically a list of str, or it is an object that fulfills the conditions of `Haver.hasfullcodes()`. If a list of str is supplied, the elements can be simple series codes or additionally supply the database where the code resides, in which case the elements must be specified either in *database:series* or *series@database* formats. Any element of `codes` can be a function code.

Examples of list of str elements are 'gdp', 'usecon:gdp', 'gdp@usecon', 'diff%(gdp)', 'diff%(usecon:gdp)', and 'diff%(gdp@usecon)'.

Haver series codes may also be supplied through an object that passes `Haver.hasfullcodes()`. The only difference of usage of the codes argument in comparison to `Haver.metadata()` is that it is NOT optional here (see section 'Notes' below). A single series code can also be supplied as str.

database
[str or list or tuple of str with one element] This argument is optional and specifies a default Haver database (excluding the path to the database, and excluding any file extension). If this argument is omitted, all elements of `codes` must be supplied as full series codes. Otherwise the function errors out.

startdate
[str or datetime.date] Specifies the starting date of the query. If is is specified as str, it is converted to a `datetime.date` object using the input format yyyy-MM-dd, e.g. ('2005-12-31). Even though `startdate` is supplied as a `datetime.date` (or date string), it is interpreted within the context of the frequency of the DataFrame. This is an important point. See section *Specifying Query Starting and Ending Periods* in the Haver Python Reference for details.

Different minimum starting years apply to different frequencies. Daily, weekly, and monthly DataFrames may not start before 1921, quaterly ones not before 1920, and annual ones not before 1901.

enddate
[str or datetime.date] Specifies the ending date of the query. Analogous comments as for `startdate` apply.

The maximum end year for all freuencies is 2099.

frequency
[str or list or tuple of str with one element] Specifies the target frequency of the query. One of 'caldaily', 'daily', 'weekly', 'monthly', 'quarterly', 'annual', or 'yearly', or any abbreviation thereof, down to a single character.

rtype
[str] Specifies the return type. One of 'haverdataframe' (default), '2tuple', or '3tuple', or any unambiguous abbreviation thereof. A Haver DataFrame is a pandas DataFrame that has a dictionary 'haverinfo' attached which contains additional query information, including the series metadata. Return type '2tuple' returns a tuple that has a pure time series DataFrame and a pure metadata DataFrame, in that order. Return type '3tuple' contains, as an additional third element, a dictionary with query information.

aggmode
[str] The aggregation mode of the query. One of 'strict', 'relaxed', or 'force', or any abbreviation therof. See section *Temporal Aggregation Modes* in the Haver Python Reference for details. Default: 'strict'.

dates
[bool] Determines whether time periods are shown as dates (e.g. '2000-03-31' instead of '2000Q1'). This may be especially useful for weekly data. See *Daily and Weekly Queries* in the Haver Python Reference for more details. Default: `False`.

eop
[bool] Determines whether dates are recorded beginning-of-period or end-of-period. Only relevant if `dates=True`. Default: `True`.

uselimits
[bool] If `False`, `Haver.data()` ignores settings regarding query limits. Default: `True`.

codeformat
[str or list of str] Specifies the desired output format of Haver series code lists as well as the labeling of the columns of the returned Haver DataFrame with series codes. If a two-element list is supplied, the first element, which is one of ':' or '@', specifies the default format of code lists of a *Haver ErrorReport dictionary* and of the *haverinfo Member Dictionary*.

The second list element is one of 'simple', 'auto', or 'full', or any unabmiguous abbreviation thereof. It specifies the series code format for the column labels of the returned DataFrame. 'full' generates codes in *db:ser* or *ser@db* format, whereas 'simple' creates column headings using simple Haver codes (i.e., *series* only). 'auto' will choose between the two as described in section *Duplicate Codes, Python Keywords*.

For a detailed description of the `codeformat` specification, and in particular how it relates to `Haver.data()`, see the help entry for function *Haver.fmtcodes*, section *The codeformat Specification* and examples.

For more information about series code formats, see subsection *Series Code Formats* of the package introduction section.

caldaily
[bool] If `True`, treats calendar daily series as such. That is, these series are fully treated as having their own separate (7-day) frequency. The default `caldaily=False` treats them as regular daily (5-day) series. The implications of this are discussed in *Queries Involving Calendar Daily Series*. Explicitly requesting a calendar daily or a daily frequency overrides the `caldaily` argument. `frequency='caldaily'` always sets `caldaily=True`. `frequency='daily'` always sets `caldaily=False`.

function
[str or int] Specifies a default math function to be applied to all codes that are not explicit function codes. For math function names and their numeric equivalents see *Math Functions* in the details section below.

underscore
[bool] If `True` (default), uses an underscore to separate simple codes from databases in the column labeling of Haver DataFrames. The usage of underscores may be useful in that series of a dataframe can be accessed using dot notation, as in `df.usecon_gdp`. If `underscore=False`, the (default or explicit) value of argument `codeformat` is used as a separator. Note that the `underscore` setting is only relevant for the column labels of a Haver DataFrame. In particular, all code lists always use either ':' or '@'.

4.1.2 Returns

pandas.core.frame.DataFrame
The precise form of the object returned depends on the value of the argument `rtype`, but in all cases involves one or two pandas DataFrame objects. See the documentation for `rtype` above.

Haver ErrorReport dictionary
In the case of invalid series code and/or database specifications, a Haver ErrorReport dictionary is returned.

For the default `rtype='haverdataframe'`, the returned value is just the dictionary. For `rtype='2tuple'` and `rtype='3tuple'`, the return values are the tuples `(her, None)` and `(her, None, None)`, respectively.

`Haver.hasfullcodes()` returns a bool.

4.1.3 Notes

(Syntax Detail)

Both `Haver.data()` and `Haver.metadata()` take arguments `codes` and `database`. They allow you to conveniently specify Haver Analytics series codes. Usage of these arguments in the two functions is very similar, with just one exception: If the argument `codes` is omitted in `Haver.metadata()`, the function interprets the request as referring to all series codes contained in the database specified. In `Haver.data()`, such an interpretation is not made, and consequently the argument `codes` is not optional here.

The order of series records within the returned object corresponds to the order of series in the input argument codes.

As is generally the case with Haver functions, (str) values of arguments supplied are not case-sensitive.

An object that passes the test of `Haver.hasfullcodes()` can be used as input argument `codes` instead of a list (tuple) of str. `Haver.hasfullcodes()` returns a logical `True` if an object satisfies the following conditions:

- it is a pandas DataFrame
- whose first two columns are named 'database' and 'code'
- and have admissible data types, as described above under 'Input Arguments'
- with no empty entries
- and have only alphanumeric entries
- and have at least one entry (the DataFrame has at least one row).

See the `Haver.data()` 'Examples' section in the Haver Python Reference for more information on this technique.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.1.4 Temporal Aggregation

Determination of the Data Set Frequency

If option `frequency` is not specified the data set frequency corresponds to the lowest frequency of the series in the query. For example, querying a daily and a monthly series will result in a monthly DataFrame, with daily values aggregated to monthly ones.

If you do specify option `frequency`, then this setting takes precedence over the rule laid out in the previous paragraph. For example, when querying a daily and a monthly series and specifying `frequency='weekly'`, then the daily series gets aggregated to weekly values, whereas the monthly series gets dropped from the query. This behavior occurs because the Haver package supports temporal aggregation, but not temporal disaggregation. pandas DataFrames provide such functionality, however.

Temporal Aggregation Modes

A Haver aggregation mode can either be 'strict', 'relaxed', or 'force'. You can invoke a particular aggregation mode by setting the argument `aggmode` to one of these three possible values. The default is 'strict'.

Note the difference between the argument `aggmode` and the metadata field 'aggtype'. Examples for 'aggtype's are 'AVG' and 'SUM' (see section *Metadata Fields* in *Haver Meta-DataFrame*). They define which calculations are carried out when a series is aggregated. aggmodes determine how these calculations behave in the presence of NaNs. `aggmode` is set for a particular query, whereas 'aggtype' is a fixed setting that an individual series has.

For many queries, the default aggregation mode 'strict' is the correct one to use. However, temporal aggregation of daily and weekly data to a lower frequency should be done under 'relaxed' or 'force' in most cases. Most economic daily time series, for example, have about 10 NaNs per year because of holidays. Whenever a daily series with 'aggtype'=AVG or 'aggtype'=SUM has a missing value, its aggregated monthly value will be NaN also, under the default aggregation mode 'strict'. As a result, aggregating such daily series to monthly series will produce NaNs mostly. You can remedy this by specifying `aggmode='relaxed'` or `aggmode='force'`.

The following table describes the behavior that obtains with each combination of `aggmode` and 'aggtype'. The term 'aggregated span' used in the table stands for a time span in the original frequency that aggregates to one observation of the target frequency. For example, 1973-Jan - 1973-Mar is an aggregated span for quarterly aggregation to 1973-Q1.

aggtype	aggmode	rule for returned value
EOP	strict	Returns the value of the last period in the aggregated span. If this value is missing, it returns missing.
	relaxed	Returns the last nonmissing value of the aggregated span. Returns missing only if all values in the aggregated span are missing.
	force	Same as under "relaxed".
AVG	strict	Does not return an aggregated value as soon as one value in the aggregated span is missing.
	relaxed	Calculates an aggregated value as soon as one value in the aggregated span is nonmissing.
	force	Same as under "relaxed".
SUM	strict	Does not return an aggregated value as soon as one value in the aggregated span is missing.
	relaxed	Same as under "strict".
	force	Calculates an aggregated value as soon as one value in the aggregated span is nonmissing.

4.1.5 Specifying Query Starting and Ending Periods

Start and end dates have to be supplied either as Python `datetime.date` object or as str. The latter are converted to `datetime.date` objects using the date mask 'yyyy-MM-dd' (e.g. 2005-12-31'). If the start date is after the last observation of the requested series an error occurs and similarly for the end date. To minimize the possibility of erroneous query specifications, start and end dates are restricted to the years 1900-2099.

If the argument `enddate` is after the last observation of the implied data set, or if the argument `startdate` is before the first observation of the implied data set, observations in the data set returned are padded with NaNs accordingly. In other words, time periods of the data set will always start with `startdate`, if specified, and will always end with `enddate`, if specified.

Start and ending dates are supplied to `Haver.data()` as calendar dates (in the form of Python `datetime.date` objects), but the interpretation of what these dates mean depends on the frequency of the data set returned. For example, for `startdate='2012-01-01'` and the data set is returned in annual frequency, it is taken to mean '2012'. This can be confusing if the frequency of the returned data set is not known in advance. Let's say that you think you have a list of monthly series codes but there is actually one or more annual codes among them, so the data get aggregated to annual frequency. If you specified `startdate` as above and in addition `enddate='2012-05-31'`, both dates get interpreted as '2012'. `Haver.data()` will return a value for 2012, and it will use all data points from 2012-Jan to 2012-Dec to calculated this value. If there were in addition a daily series in the query, an aggregated value for 2012 would be calculated for it too, based on all data points from 2012-01-01 to 2012-12-31. To summarize:

- Start and ending periods are supplied as calendar dates (in the form of as Python `datetime.date` objects), but are interpreted as periods within the context of the frequency of the data set returned.
- Any aggregation that takes place uses all underlying periods of the 'aggregated span', with no exceptions. The rules described in section *Temporal Aggregation Modes* apply, with no exceptions.

For daily and weekly data sets it may happen that the dates of the data retrieved are slightly (by a few days) outside of the range specified by arguments `startdate` and `enddate`.

4.1.6 When Data Cannot Be Retrieved

In some instances, `Haver.data()` cannot retrieve data for a particular series or for a particular query. Dependening on the reason why this happens, `Haver.data()` responds in two different ways: It may either drop some series codes from the query and retrieve data for the other series codes, or it may retrieve no data at all and instead return a Haver ErrorReport dictionary. The paragraphs below state in which cases each behavior occurs and what you can do to fix up the query.

Dropping of Series

There are four instances where a query is successful (i.e. a Haver DataFrame is returned) but some of the requested series get dropped from the query:

1. A series does not have any data points. It only occurs if the series has no data point in the database. This is very rarely the case. A series does not get dropped if you specify starting and ending dates in such a way that there are no valid data points in this time span. The series will solely consist of NaNs in this case.
2. A series cannot be aggregated to the data set frequency. This occurs, for example, when a series holds some kind of fraction (e.g. the trade balance as a percentage of GDP). Then the aggregated value cannot simply be calculated as a sum or average. Such series have aggtype=NDF or aggtype=NST (see section *Metadata Fields* in *Haver Meta-DataFrame*). In these cases, Haver Analytics frequently provides separate series with aggregated values that are based on correct calculations.
3. A series cannot be disaggregated to the data set frequency. Temporal disaggregation is currently not supported by the Haver Package.
4. A math function that requires series with positive values is applied to series that do not fulfill this criterion; or some other inconsistency of a math function specification with series data or metadata arises.

If at least one series whose data can be retrieved remains, `Haver.data()` returns a Haver DataFrame. If one or more series were dropped, this object contains information on the codes that failed, and on the reasons why this happened. The function `Haver.codelists()` enables you to get at lists of dropped series codes.

Query Errors

As soon as one series code or one database specified in a query cannot be found, or as soon as a series code is syntactically misspecified (including math function misspecifications), `Haver.data()` returns a Haver ErrorReport dictionary. This structure contains details on what went wrong with the query. The `Haver.codelists()` method can be applied to the Haver ErrorReport dictionary and returns lists of series codes that caused the query to fail.

Duplicate Codes, Python Keywords

Duplicates of full series codes are removed automatically. A warning is issued. For example, the query

```
>>> hd = Haver.data(['fxtwb', 'fxtwb'], 'haverd')
```

will return a data set that contains 'haverd:fxtwb' just once.

Duplicate simple series codes are allowed. In this case, variable names in the output DataFrames will be prepended with the database names. Full series codes are also used as variable names if at least one of the simple series codes is identical to a Python keyword (e.g. `for`). For example,

```
>>> hd = Haver.data(['daily:ffed', 'usecon:gdp'])
```

will use 'ffed' and 'gdp' as DataFrame column names, whereas

```
>>> hd = Haver.data(['daily:ffed', 'weekly:ffed'])
```

and

```
>>> hd = Haver.data(['daily:ffed', 'usecon:for'])
```

will use 'daily_ffed', 'weekly_ffed' and 'daily_ffed', 'usecon_for', respectively. In all cases, the database origin of the series is tractable via the `haverinfo` member dict of a Haver DataFrame.

Duplicate full series codes are allowed if they pertain to different math functions. For example, the query

```
>>> hd = Haver.data(['usecon:gdp', 'diff%(usecon:gdp)', 'diff%(usecon:gdp)', 'yryr
↪%(usecon:gdp)'])
```

will retrieve three series: 'usecon:gdp', 'diff%(usecon:gdp)', and 'yryr%(usecon:gdp)'.

4.1.7 Math Functions

Math functions can be specified by using function codes as inputs via argument `codes`, or by usage of the argument `function`, or by a combination of both.

For numeric formulas of the math functions, see the help pages that are available from Haver's DLXVG3 data browser or Haver's Excel add-in.

Basic Math Functions

The following ten math functions are referred to as 'basic functions' in DLXVG3:

Name	Alternative	Numeric code	Description
[level]		1	
difa%(X)	pcta(X)	2	% Change - Annual Rate
diff%(X)	pctp(X)	3	% Change - Period to Period
yryr%(X)	pcty(X)	4	% Change - Year to Year
difa(X)	difa(X)		Difference - Annual Rate
diff(X)	difp(X)	5	Difference - Period to Period
yryr(X)	dify(X)	6	Difference - Year to Year
difal(X)	loga(X)	7	Log Change - Annual Rate
diffl(X)	logp(X)	8	Log Change - Period to Period
yryrl(X)	logy(X)	9	Log Change - Year to Year

The symbol X in the above table stands for a series code. Basic functions have two naming schemes. The main one (column 1) is also used by DLXVG3 and Excel. If you find the alternative naming scheme (column 2) more intuitive, you can use it in your code lists. However, all output code lists generated by Haver package functions adhere to the main naming scheme. The numeric code for math functions (column 3) corresponds to the key mapping that DLXVG3 uses. It cannot be used for function codes, but argument `function` does accept them. As function codes, you can specify, for example, 'yryr%(usecon:gdp)' and 'loga(usecon:gdp)' but not '3(usecon:gdp)'. Argument function can be specified using any of the three ways, for example, `function='yryr%'`, `function='loga'`, and `function=7`.

The first math function (numeric code 1, top row of the table) isn't really a function, since it refers to the unchanged series in levels. This 'function' does not have a name. Series in levels are always referred to by the omission of any math function name, and *not*, for example, by 'level(usecon:gdp)'.

The math function description in column 4 of the table will be added to elements of the series description list contained in the haverinfo dictionary (key `vardescription`) that is part of a Haver DataFrame.

Series with non-positive values may not be used in growth rate calculations and log transforms. Such series are marked by the 'diftype' *metadata field*, where a value of 0 says that such calculations are allowed, whereas a value of 1 says that they are not. For series of the latter type, only functions with numeric codes 1, 5, or 6 are allowed, plus function 'difa' (which does not have a numeric code). Series that are requested using one of the other math functions and that have diftype=1 will be dropped from the query. Note also that in DLXVG3 the function keybinding 2 calculates 'difa%' / '% Change - Annual Rate' for series with diftype=0 and 'difa' / 'Difference - Annual Rate' for series with diftype=1. There is no such automatic switching in Python: `function=2` is allowed for series with diftype=0 and generates an error for series with diftype=1.

Any function codes that are dropped from the query will be stored in the codelist 'nopct' and, if your query results are stored in a dataframe called 'hd', are accessible via `Haver.codelists(hd)['nopct']`.

If any of the query frequency, the query start date, or the query ending date are not supplied as arguments, `Haver.data` determines their values automatically based on the series metadata. Basic function codes that are dropped from the query have no influence on this process.

Advanced Math Functions and Seasonal Adjustment

Advanced functions follow similar rules as basic functions. The differences in Python are:

- They do not have an alternative name.
- They do not have a numeric code.
- Some of them receive an argument, separated by a comma from the series code.

The best overview for advanced functions is probably their dialog box in DLXVG3 (shortcut Alt+F). In Python, functions from that dialog box are mapped into the following 30 advanced function names, plus seasonal adjustment:

Name	Description
–	*Functions with one argument*
diff%(X,n)	Point to Point % Change
difv%(X,n)	Average % Change
difa%(X,n)	Annualized % Change
diff%c(X,n)	Point to Point % Change Centered
difv%c(X,n)	Average % Change Centered
difa%c(X,n)	Annualized % Change Centered
diff(X,n)	Point to Point Difference Change
difv(X,n)	Average Difference Change
difa(X,n)	Annualized Difference Change
diffc(X,n)	Point to Point Difference Change Centered
difvc(X,n)	Average Difference Change Centered
difac(X,n)	Annualized Difference Change Centered
diffl(X,n)	Point to Point Log Change
difvl(X,n)	Average Log Change
difal(X,n)	Annualized Log Change
difflc(X,n)	Point to Point Difference Log Change Centered
difvlc(X,n)	Average Difference Log Change Centered
difalc(X,n)	Annualized Difference Log Change Centered
movv(X,n)	Moving Average
mova(X,n)	Annualized Moving Total
movt(X,n)	Moving Total

continues on next page

Table 1 – continued from previous page

Name	Description
movvc(X,n)	Moving Average Centered
movac(X,n)	Annualized Moving Total Centered
movtc(X,n)	Moving Total Centered
–	*Functions without argument(s)*
ytd(X)	Year to Date
dytd(X)	Difference Year to Date (first period of year in levels, differences thereafter)
na2z(X)	N/A to Zero
z2na(X)	Zero to N/A
zs(X)	Z-Score (applied to the data span in the query)
–	*Functions with special arguments*
index(X,*date*=*val*)	Index
–	*date* can be of lower frequency than the query, but not higher
–	*val* is the index base value
–	see the text below for date specifications
sa(X)	Seasonal Adjustment (based on full series span)
sa(X,*date1*)	adjusts based on and returns only data for *date1* and forward
sa(X,*date1*:*date2*)	adjusts based on and returns only data for the span from *date1* to *date2*
–	see the text below for date specifications

X stands again for a series code and parameter n in the above counts the number of lags to be applied in the formula. For example, for the quarterly series NCH@USECON, 'diff%(nch@usecon,4)' calculates a 4-quarter percentage change. Other examples are: 'difv%c(nch@usecon,4)', 'ytd(nch@usecon)', 'index(nch@usecon, 20201=100)', 'index(nch@usecon, 2020=100)', 'sa(nch)', 'sa(nch, 201010)', and 'sa(nch, 20101:20204)'.

In the `function` parameter of `Haver.data()`, you leave out the series code (X). Example: `function='difv%c(4)''`. For functions without parameter, the parentheses are optional. Examples: `function='ytd'`, `function='ytd()'`.

The argument n is always interpreted in terms of the query frequency, not in the original frequency of the series. For example, for the monthly series LANAGRA@USECON, 'diff%(lanagra@usecon,4)' will correspond to a 4-qtr % change if the query frequency is quarterly. It will be calculated based on the aggregated series values.

'sa()' without date arguments runs the seasonal adjustment algorithm on the entire series data that exists in the database. If you want to base the algorithm on a subset of data, for example on a query span that is shorter in length, you must specify *date1* and (optionally) *date2*. If *date2* is not given, the series end date in the database is used.

Dates for functions 'sa()' and 'index()' can have one of the frequency formats YYYY, YYYYQ, YYYYMM, YYYYMMDD. For 'index()', *date* can be of lower frequency than the original series frequency, but not higher. For 'sa()', if you specify dates in a higher frequency than the query, they will be converted to the corresponding query frequency period. If you specify dates in a lower frequency, they will be converted to the corresponding first subperiod of the higher frequency. Example: For a monthly query frequency, the quarterly date specification '20174' will be converted to '201710'. It is recommended that you specify dates for both 'sa()' and 'index()' in the frequency of the query.

If temporal aggregation is performed, it is done before the functions are applied.

In case of missing values, 'sa()' applies linear interpolation to the underlying series values before the seasonal adjustment algorithm is run. If temporal aggregation is performed, it happens before the interpolation step.

Misspecification of advanced function codes will result in the query being aborted. A `_HaverPkgError` will be thrown.

There are some additional restrictions that apply to some of the functions whose verification requires either series metadata, or series data, or information about other series codes in the query. Violation of these restrictions will result in the series being dropped from the query.

- Seasonal adjustment is only performed if the query frequency is monthly or quarterly.

- Seasonal adjustment requires a minimum data span of 36 months or 12 quarters.
- As is the case for basic functions, advanced functions that involve percentage or log changes as well as the 'index()' function cannot be applied to series that contain non-positive values.
- 'ytd()' and 'dytd()' cannot be applied to series which are of datatype '%' or 'INDEX'.

As for is the case for basic function codes, any advanced function codes that are dropped from the query will be accessible in the codelist 'nopct' of the query output. For example, if your query results are stored in a dataframe called 'hd', the list of dropped codes can be obtained via `Haver.codelists(hd)['nopct']`. Note that this list includes, besides codes for which percent and log changes are not permitted, codes that are dropped for any other reason; for example, 'sa()" codes whose data span is too short for seasonal adjustment.

It can happen, in rare cases, that a dropped advanced function code influences the query frequency. For example, if you query a monthly and a quarterly series and request seasonal adjustment for the latter, and if the latter series gets dropped due to insufficient data for seasonal adjustment, the query frequency is still quarterly, not monthly. These are edge cases though. It is nevertheless recommended to specify the frequency that one wants explicitly, which avoids such complications.

4.1.8 Daily and Weekly Queries

When querying a weekly series, the dates assigned to data points correspond to the reference day-of-week of the series (e.g. the day on which the data source publishes new data). When aggregating a daily series to weekly frequency, this release day is set to Friday. When querying multiple weekly series, the dates shown correspond to the release day of the first series in the data set.

By default, all Haver-DataFrames show time information as periods (option default `dates=False`), meaning that the DataFrame's index is of type `PeriodIndex`, as opposed to `DatetimeIndex`. For weekly data, this can potentially create confusion with respect to what the exact reference day-of-week is since, for each row, the start-day-of-week and the end-day-of-week are displayed. If you want to avoid ambiguity, you can use option `dates=True`. Together with the option default `eop=True` it shows the correct reference dates.

If you supply arguments `startdate` and `enddate` to daily or weekly queries, it may happen that the starting and ending dates of the data retrieved are slightly (by a few days) outside of the implied time span.

When aggregating daily or weekly series to a lower frequency, familiarity with Haver aggregation modes is necessary. See section *Temporal Aggregation Modes* above.

4.1.9 Queries Involving Calendar Daily Series

Before reading this section, be sure to be familiar with *Calendar Daily Series and Their Weekend Component Series*.

Calendar daily (7-daily) series are supported in Python, but they have to be explicitly switched on. The main relevant option is `caldaily`, which defaults to `False`, but setting option `frequency` to 'daily' or 'caldaily' also has an impact on the outcome.

If you supply the argument `caldaily=True`, calendar daily series are treated has having their own 7-day frequency (as opposed to the default (5-day) daily frequency). This has a number of implications for queries involving multiple frequencies. All of these implications follow standard package rules for handling temporal aggregation and disaggregation (the latter not being supported and resulting in series being dropped from the query).

Before enumerating the implications of `caldaily=True`, be aware of the default behavior (`caldaily=False`):

1. If you query calendar daily series along with daily series, the resulting frequency will be daily (corresponding to pandas DataFrame frequency 'Business daily'). All series are kept in the query. Only the weekdays of the calendar daily series are used.

2. Since the default is to treat calendar daily series as regular (5-day) series, by default only these values are used in temporal aggregation calculations. If you want to aggregate a calendar daily series to a frequency of weekly or lower, you almost surely want to base calculations on all seven values of each week. You *must* explicitly supply `caldaily=True` for this to happen.

Warning: This box reiterates the critical importance of the `caldaily` setting for temporal aggregation pointed out in point 2 above. If you aggregate a calendar daily series to weekly or lower, be sure to explicitly use `caldaily=True`.

Because of the importance of the issue, `Haver.data()` will warn you of potential problems. The messages that are issued differ depending on whether you use local queries or DLX Direct queries. Local queries will warn you whenever a calendar daily (7-day) series is treated as a regular daily (5-day) series; and they will indicate the exact number of series that are concerned. DLX Direct queries will issue a generic warning message whenever daily or calendar daily series are queried under `caldaily=False`. These messages can be switched off using *Haver.verbose*.

In contrast to the above two rules, under `caldaily=True` the following rules apply:

1. If your query contains calendar daily series and series with lower frequencies and `frequency='caldaily'` is *not* specified, temporal aggregation is performed. Calendar daily series that have a valid aggregation type get aggregated using all seven days of the week, unless the target frequency is daily, in which case the calendar daily series get dropped.
2. If your query contains calendar daily series and series with lower frequencies and `frequency='caldaily'` *is* specified, only the calendar daily series are retrieved and all other series are dropped. This happens also if the query consists only of calendar daily and daily series: The daily series get dropped.

Points 1 and 2 above imply that it is not possible to directly generate a pandas DataFrame that shows all data points of calendar daily series along with daily series that have missings for weekend days. If you want that, you have to issue two separate queries (one for calendar daily series and another one for daily series) and then merge the resulting DataFrames manually.

Related to the above, remember that explicitly setting `frequency='caldaily'` will override the `caldaily` setting and set it to `True`. Similarly, explicitly setting `frequency='daily'` will override the `caldaily` setting and set it to `False`.

Lastly, when aggregating calendar daily series to a weekly frequency, the dates assigned to data points correspond to Sundays (the last day of the aggregated span).

Note: The Haver calendar daily frequency in Python follows a slightly different set of rules than calendar daily in Excel and DLXVG3. For example, in Excel it is possible to query calendar daily series and regular daily series in a single query. As noted previously, this is not possible in Python. Likewise, as pointed out in the above warning, correct aggregation of calendar daily series to lower frequencies require an explicit argument in Python (`caldaily=True`). Excel and DLXVG3 always use all seven values of a week for aggregation.

Another important thing to note concerns the application of *Math Functions*. The outcome for math function calculations for calendar daily series differs depending on whether `caldaily` is `True` or `False`. One reason for it is that this setting changes the number of periods per year, which enters some calculations.

Warning: If you apply a math function to a calendar daily series, be sure to explicitly use `caldaily=True`.

4.1.10 Query Limits

To provide a safeguard against large data queries that may either take very long or that consume an excessive amount of memory, `Haver.data()` by default checks settings on limits regarding data queries. There are two limits imposed. The first one specifies the maximum number of series in a query (default value: 5000) and the second one the maximum number of data points (default value: 15 million, which allows e.g. for 1000 daily series that span 50 years). You can query the current state and change these settings using *Haver.limits*, whose help entry discusses the issues related to query execution time and memory consumption in more detail.

Note that the above limits never apply to *DLX Direct* queries. See *Haver.direct*.

4.1.11 Interrupting Execution

If you have specified a large local query that is taking a long time to conclude, you can cancel it in the usual way. See *Query Execution Time*.

DLX Direct queries are more difficult to interrupt. See *Haver.direct*.

4.1.12 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

The following statements illustrate how you can specify series codes and database names. Argument usage of `codes` and `database` is identical to the statements of the 'Examples' section in `Haver.metadata()`. All queries below are equivalent. To limit the output, we supply a start and an end date.

```
ex_t1 = '2012-01-10'
ex_tN = '2012-01-13'
kwargs = {'startdate':ex_t1, 'enddate':ex_tN}
Haver.data([       'fxtwb',        'fxtwm',        'fxtwotp'], 'haverd'  , **kwargs)
Haver.data([       'fxtwb',        'fxtwm',        'fxtwotp'], 'haverd'  , **kwargs)
Haver.data([       'FXTWB',        'FXTWM',        'FXTWOTP'], 'HAVERD'  , **kwargs)
Haver.data(['haverd:fxtwb',        'fxtwm',        'fxtwotp'], 'haverd'  , **kwargs)
Haver.data(['haverd:fxtwb', 'haverd:fxtwm', 'haverd:fxtwotp'],     None  , **kwargs)
Haver.data(['haverd:fxtwb', 'haverd:fxtwm', 'haverd:fxtwotp'], 'havermqa', **kwargs)
```

Querying codes from multiple databases is possible:

```
ex_tN = '2012-01-31'
hd_1  = Haver.data(['haverD:fxtwb', 'haverW:lic', 'fcm1'] ,
        'haverMQA', startdate=ex_t1, enddate=ex_tN)
```

Querying all codes of a database is possible with `Haver.metadata()` but not with `Haver.data()`, so the following produces an error:

```
>>> Haver.data(database='haverd')     # <= produces an error!
```

You can assign the return value of `Haver.data()` to a variable, of course:

```
ex_tN = '2012-01-13'
hd_2  = Haver.data(['fxtwb', 'fxtwm', 'fxtwotp'] , 'haverd', startdate=ex_t1,
```

(continues on next page)

(continued from previous page)

```
        enddate=ex_tN)
print( hd_2 )
```

You can request a lower frequency than daily, e.g. quarterly (at this frequency, we do not have to use options 'startdate' and 'enddate' in order to limit output):

```
hd_3 = Haver.data(['fxtwb', 'fxtwm', 'fxtwotp'] , 'haverd' , frequency='q')
print( hd_3 )
```

The data points are NaNs because the default aggregation mode is 'strict'. When aggregating daily data, it is frequently appropriate to use aggregation modes that are less restrictive. For details, see section *Temporal Aggregation Modes* above.

```
hd_4 = Haver.data(['fxtwb', 'fxtwm', 'fxtwotp'] , 'haverd' , frequency='q',
       aggmode='relaxed')
print( hd_4 )
```

Note that the default aggregation mode remains 'strict'.

Let's see what the database 'HAVERMQA' contains:

```
hmd_mqa = Haver.metadata(database='havermqa')
print( hmd_mqa.iloc[0:10, 0:5] )
```

We pick and retrieve a monthly, a quarterly, and an annual series. The monthly and the quarterly series get aggregated to annual frequency.

```
hd_5 = Haver.data(['c', 'fcm1', 'ift'], 'havermqa', startdate='2008-01-13')
print( hd_5 )
```

When we explicitly request a quarterly frequency, monthly series get aggregated while annual ones get dropped:

```
hd_6 = Haver.data(['c', 'fcm1', 'ift'], 'havermqa', startdate='2008-01-13',
       frequency='q')
print( hd_6 )
```

We can investigate what series got dropped by using `Haver.codelists()`:

```
ex_cl = Haver.codelists(hd_6)
print( ex_cl.keys() )
print( ex_cl.values() )
print( ex_cl['nodisagg'] )
```

Every time series data retrieval also retrieves the series metadata and attaches it to the Haver DataFrame object. You can access it through the `haverinfo` member dict of Haver DataFrames:

```
print( hd_6.haverinfo['metadata'] )
```

The previous call returned a Haver Meta-DataFrame. When assigning it to a variable, you can customize its output:

```
hmd_6 = hd_6.haverinfo['metadata']
print( hmd_6.loc[:,['code', 'frequency']] )
```

You can retrieve an identical Haver DataFrame that displays dates instead of period values:

```
hd_7 = Haver.data(['c', 'fcm1', 'ift'], 'havermqa',
                  startdate='2008-01-13', frequency='q', eopdates=True)
```

You may find it more convenient or intuitive to receive the different informational parts of a data query - data, metadata, and query information - in a Python tuple. Use the `rtype` option for this:

```
(ex_data, ex_metadata, ex_info) = Haver.data(['c', 'fcm1', 'ift'], 'havermqa', rtype=
↪'3tuple')
```

A useful technique is to use a Haver Meta-DataFrame to specify codes for `Haver.data()` queries:

```
hmd_w = Haver.metadata(database='haverw')
hd_w  = Haver.data(hmd_w)
```

We can add codes from other Haver Meta-DataFrames:

```
ex_tbl_d    = Haver.metadata(database='haverd')
ex_tbl_dw   = ex_tbl_d.iloc[:,0:2].append(hmd_w.iloc[:,0:2])
print( ex_tbl_dw.iloc[0:6,:] )

import pandas as pd
ex_manual = pd.DataFrame({'database':['havermqa', 'havermqa'], 'code':['c', 'cd']})
ex_tbl_all  = ex_tbl_dw.append(ex_manual)
```

If you are not sure whether you can use an object other than a str or list or tuple as input for the `codes` argument of `Haver.data()`, you can check with:

```
print( Haver.hasfullcodes(ex_tbl_all) )
```

Since this is `True`, we can pull in the data:

```
hd_all = Haver.data(ex_tbl_all)
hmd_all = hd_all.haverinfo['metadata']
print( hmd_all.loc[:,['database', 'code', 'descriptor']] )
```

Math functions can be applied by individual code:

```
ex_cl2 = ['gdp@havermqa', 'diff%(gdp@havermqa)', 'c@havermqa', 'yryr%(xnet@havermqa)']
hd8 = Haver.data(ex_cl2)
print( hd8 )
```

One series got dropped because its data do not support the math function requested. You can state the previous query in simpler terms by using a default database. Here we addtionally specify a default function:

```
ex_cl3 = ['gdp', 'diff%(gdp)', 'c', 'yryr%(xnet)']
hd9 = Haver.data(ex_cl3, database='havermqa', function='yryr')
print( hd9 )
```

Some, but not all, of the advanced functions take an argument:

```
ex_cl4 = ['gdp', 'difalc(gdp,4)', 'index(c,20051=100)', 'zs(xnet)']
hd10 = Haver.data(ex_cl4, database='havermqa', function='diff%(4)')
print( hd10 )
```

The `vardescription` key of the haverinfo dictionary contains the function description:

```
hd10.haverinfo['vardescription']
```

```
>>> Haver.path('restore')
```

4.2 Haver.metadata

Haver.**metadata**(*codes=None*, *database=None*, *decode=True*, *histcodes='keep'*, *caldaily=False*, *wecodes='keep'*, *codeformat=[':', 'full']*, *rtype='havermetadataframe'*)

Description

`Haver.metadata()` queries metadata information on time series contained in Haver Analytics databases. The information is returned in a pandas DataFrame.

This help entry assumes that you are familiar with the Haver Python Reference Section *Introduction to the Haver Package for Python*.

Syntax

```
hmd = Haver.metadata(codes)
hmd = Haver.metadata(codes, database)
hmd = Haver.metadata(codes, database, [...])
hmd = Haver.metadata(None , database)
hmd = Haver.metadata(None , database, [...])
```

4.2.1 Arguments

codes

[str or tuple of str or list of str] codes is typically a list of str, or it is an object that fulfills the conditions of `Haver.hasfullcodes()`. The elements of the list (tuple) can be regular series codes or additionally supply the database where the code resides, in which case the elements must be specified either in *database:series* or *series@database* formats. Any element of `codes` can be a function code, but note that the specification of math functions has no influence on the metadata. The metadata retrieved always refer to the series in levels.

Examples of list of str elements are 'gdp', 'usecon:gdp', 'gdp@usecon', 'diff%(gdp)', 'diff%(usecon:gdp)', and 'diff%(gdp@usecon)'.

The only difference of usage of the codes argument in comparison to `Haver.data()` is that it IS optional here (see section 'Details' below). A single series code can also be supplied as str.

database

[str or list or tuple of str with one element] This argument is optional and specifies a default Haver database (excluding the path to the database, and excluding any file extension). If this argument is omitted, all elements of codes must be in *database:seriescode* format. Otherwise the function errors out.

decode

[bool] This argument is rarely used. By default `decode` is `True`, i.e. metadata information that is numerically encoded will be translated to meaningful character strings. For example, internally the aggregation

type 'AVG' (average) is encoded as '1', and the default `decode` value of `True` will re-translate this into 'AVG'.

histcodes

[str or list or tuple of str with one element] Only matters for queries that a) are run on entire databases and b) contain daily and/or weekly series. One of 'keep' (default), 'drop', 'sequential', or 'nonsequential'. 'keep' and 'drop' keep or drop historical component series in the database from the output DataFrame. 'sequential' keeps only proper historical component series. 'nonsequential' keeps only codes that could be confused with historical component series. For more details, see section *Dealing with Historical Component Series when Querying Entire Databases*.

`histcodes` cannot be used if there are any function codes in the input code list.

caldaily

[bool] If `True`, calendar daily series receive a frequency value of 'C' (or '61' under `decode=False`). The default `caldaily=False` shows calendar daily series as regular series with a frequency code of 'D'.

wecodes

[str or list or tuple of str with one element] This argument is similar to `histcodes`. It only matters for queries that a) are run on entire databases and b) contain calendar daily series. One of 'keep' (default) or 'drop' with the implied actions of keeping or dropping weekend component series in the database from the output DataFrame. For more details, see section *Calendar Daily Series and Their Weekend Component Series*.

`wecodes` cannot be used if there are any function codes in the input code list.

codeformat

[str or list of str] Specifies the desired output format of Haver series code lists. The most important values are ':', and '@', which yield output lists in *db:ser* and *ser:db* format, respectively. Values of '' or 'simple' will be automatically converted to ':'.

In the context of `Haver.metadata()`, this argument determines the default format of code lists of a Haver ErrorReport and the format of the code list when `Haver.codelists()` is called on the returned Haver Meta-DataFrame of this function.

For a detailed description of the `codeformat` specification, see the help entry for function *Haver.fmtcodes*, section *The codeformat Specification* and examples.

For more information about series code formats, see subsection *Series Code Formats* of the package introduction section.

rtype

[str] Specifies the return type. One of 'havermetadataframe' (default) or '2tuple', or any unambiguous abbreviation thereof. A Haver Meta-DataFrame is a pandas DataFrame that has a dictionary 'haverinfo' attached which contains additional query information. Return type '2tuple' returns a 2-element tuple whose first element is a pure DataFrame and whose second element is the dictionary with query information.

function

[str or int] Specifies a default math function to be applied to all codes that are not explicit function codes. Note that the specification of math functions has no influence on the metadata. For math function names and their numeric equivalents see *Math Functions* in the details section of `Haver.data()`.

4.2.2 Returns

pandas.core.frame.DataFrame
The return value of `Haver.metadata()` is a pandas DataFrame object. In the context of the Haver package, this is refered to as a *Haver Meta-DataFrame*.

Haver ErrorReport dictionary
In the case of invalid series code and/or database specifications, a *Haver ErrorReport dictionary* is returned.

4.2.3 Notes

(Syntax Detail)

Argument `codes` may be omitted if the argument `database` is supplied, in which case the function interprets the request as referring to all series codes contained in the database specified. Note that this is not possible with `Haver.data()`.

There are three possible combinations of using arguments `codes` and `database`:

1. only supply `codes` . In this case all elements of `codes` have to be in *database:seriescode* format.
2. only supply `database` . This will retrieve metadata for all series contained in the database specified.
3. supply both `codes` and `database` . Metadata for all elements of `codes` are queried. If an element of `codes` is not in *database:seriescode* format, the value of `database` will serve as the default database.

The order of series records within the returned object corresponds to the order of series in the input argument `codes`.

As is generally the case with Haver functions, (str) values of arguments supplied are not case-sensitive.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.2.4 Query Errors

As soon as one series code or one database specified in a query cannot be found, or as soon as a series code is syntactically misspecified (including math function misspecifications), `Haver.metadata()` returns a Haver ErrorReport dictionary. This structure contains details on what went wrong with the query. The function `Haver.codelists()` enables you to get at lists of series codes that could not be found.

4.2.5 Interrupting Execution

If you have specified a large local query that is taking a long time to conclude, you can cancel it in the usual way. See *Query Execution Time*.

DLX Direct queries are more difficult to interrupt. See *Haver.direct*.

4.2.6 Dealing with Historical Component Series when Querying Entire Databases

There is an upper limit on the length of a Haver time series: It can span at most 2100 time periods (2100 business days, 2100 weeks, etc.). This corresponds to about 8 years of business daily data and 39 years of weekly data. The limit is not relevant for lower frequencies.

For many daily series and for some weekly series, this constraint is binding. Haver databases deal with this limit in the following way: If a series covers more than 2100 time periods, additional history can be stored in auxiliary series that have the same base code but also have sequence numbers 2-9 appended to it. For example, suppose that a database contains a daily series FXCAN that has 5000 observations. Then the database contains the base code FXCAN along with the historical component series FXCAN2 and FXCAN3. If such a set of series exists in a database and you query the time series data of the base code, all data history is automatically spliced together. That is to say, when executing a data query on FXCAN you never notice that the data is actually stored in separate series.

However, there is one instance when you actually may come across the existence of historical component series, and this is when you query all metadata of a database that contains long daily and/or long weekly series. In the example above, a metadata query on the full database would result in three separate entries in the Haver Meta-DataFrame that are related to FXCAN: It would have entries for codes FXCAN, FXCAN2, and FXCAN3. Option `histcodes='drop'` now allows you to skip historical component series in your query, so the entries for FXCAN2 and FXCAN3, which are rarely of any use, do not show up in the query result.

This technique will matter if a database contains many long daily series. For example, if you would like to query all data of Haver's DAILY database, which contains daily economic statistics for the US economy, you can issue

```
hmd = Haver.metadata(database='daily', histcodes='drop')
hd  = Haver.data(hmd)
```

For databases like DAILY, this may reduce the number of series - and consequently the memory requirements and execution speed of the query - by 50-70%.

4.2.7 Calendar Daily Series and Their Weekend Component Series

The calendar daily frequency records seven data points per week, as opposed to the regular Haver daily frequency, which is based on five observations per week.

Important: The default is to not show calendar daily series separately, that is, in the output Haver Meta-DataFrame their frequency is, by default, indicated as 'D', as it is for regular daily series. If you want to identify calendar daily series, you must explicitly specify the argument `caldaily=True`. Then their frequency is shown as 'C'.

The calendar daily frequency is implemented using a technique similar to the one used for historical component series described in the previous section: A (5-day) daily series code is combined with two weekly series codes that hold data for Saturday and Sunday. The two weekly series' codes correspond to the code of the daily series with a 0 and 1 appended (Saturday and Sunday data, respectively). These series are referred to as 'weekend component series'. When querying an entire database, the argument `wecodes` works very similar as argument `histcodes` (see the previous section). Specifying `wecodes='drop'` will prevent weekend component series from appearing in the output Haver Meta-DataFrame.

4.2.8 Math Functions

Math function calculations, like growth rates, are more important with *Haver.data*, but `Haver.metadata()` too allows Haver code lists as inputs that contain function codes. They do nothing to the output, except that the 'code' field of the returned Haver Meta-DataFrame contains function codes in *func(ser)* format where appropriate.

It is important to note that the metadata shown always pertains to the original series. Similar to temporal aggregation, functions change metadata characteristics, like the number of observations, or the magnitude of the returned series. This, however, is not reflected in the return values of `Haver.metadata()`: The data always refer to the original series.

4.2.9 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

The following statements illustrate how you can specify series codes and database names. Argument usage of `codes` and `database` is identical to the statements of the 'Examples' section in `Haver.data()`. All queries below are equivalent.

```
Haver.metadata([       'fxtwb',        'fxtwm',        'fxtwotp'], 'haverd')
Haver.metadata([       'FXTWB',        'FXTWM',        'FXTWOTP'], 'HAVERD')
Haver.metadata(['haverd:fxtwb',        'fxtwm',        'fxtwotp'], 'haverd')
Haver.metadata(['haverd:fxtwb', 'haverd:fxtwm', 'haverd:fxtwotp']          )
Haver.metadata(['haverd:fxtwb', 'haverd:fxtwm', 'haverd:fxtwotp'], 'havermqa')
```

Querying codes from multiple databases is possible:

```
hmd_1 = Haver.metadata(['haverD:fxtwb', 'haverW:lic', 'fcm1'] , 'haverMQA')
```

Querying all codes of a database is not possible with `Haver.data()`, but it is with `Haver.metadata()`:

```
hmd_2 = Haver.metadata(database='haverd' )
```

Note that when querying an entire database, you must name the argument `database` explicitly.

```
Haver.metadata('haverd')    # <= produces an error!
```

The statement above produces an error since the first argument of `Haver.metadata()` is `codes`, but we meant to specify `database`, so to accomplish what we want to do we have to run:

```
print( Haver.metadata(database='haverd') )
```

or

```
print( Haver.metadata(None, 'haverd') )
```

When specifying `decode='False'`, some metadata remain encoded as numeric values. In rare cases it may be helpful to look at these numeric values. For example:

```
hmd_w = Haver.metadata(database='haverw', decode=False)
print( hmd_w.frequency.value_counts() )
```

shows a frequency count of the series in 'HAVERW'. 9 of those series have a release day-of-week of 53, 12 series of 56. These values correspond to Wednesday and Saturday, respectively (see *Haver Meta-DataFrame*). This can be confirmed by looking at one of the fields 'startdate' or 'enddate':

```
from datetime import datetime
ex_daynums = [datetime.isoweekday(x) for x in hmd_w.enddate.tolist()]
print( ex_daynums )
```

The help for `datetime.isoweekday()` confirms that a 3 in the output list corresponds to a Wednesday, and 6 corresponds to a Saturday.

```
>>> Haver.path('restore')
```

4.3 Haver DataFrame

4.3.1 Description

A Haver DataFrame is the return type of `Haver.data()`. It contains times series data queried from Haver Analytics databases.

To learn about the many possibilities that pandas DataFrames provide for indexing into the data and for data analysis in general, consult the pandas documentation.

If you have not done so yet, have a look at *Haver.data* before reading this help entry.

4.3.2 Details

Shape and Time Recording

If you query K variables, the resulting Haver DataFrame will have K columns. The time vector will be stored as the DataFrame's index. The time vector of a Haver DataFrame will never have gaps, which implies that missing values of time series are always explicitly shown as NaNs in the DataFrame.

`haverinfo` Member Dictionary

A Haver DataFrame has a special member dictionary called 'haverinfo' that stores additional information about the query. Available keys are:

Dict Key	Contents
description	query date and time (str)
vardescription	series labels (list of str)
metadata	series metadata (Haver Meta-DataFrame)
frequency	DataFrame frequency (str)
codelists	various lists of codes related to the query (dict)
codeformat	default display format of series code lists (list)
pkgversion	the Haver package version that created the Haver DataFrame (str)
hasfuncs	whether math functions were part of the query

Note the `metadata` key: It contains all the metadata for all time series of a Haver DataFrame as a Haver Meta-DataFrame. You can access them trough *Haver_DataFrame_name*.haverinfo['metadata'].

Warning: The `haverinfo` member is not included in copy operations.

For example, the Haver DataFrame `hd2` in

```
hd2 = hd.ix[0:10].copy()
```

will not have a member called `haverinfo`.

The `codelists` field contains information on lists of series codes that have been successfully retrieved and on codes that were dropped from the query. You can use *Haver.codelists* to extract these code lists.

Coercion to Other Types

See the pandas documentation on how to coerce pandas DataFrames into other types.

4.3.3 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

In the 'Examples' section of `Haver.data()` we saw instances where series were dropped because temporal aggregation or disaggregation could not be performed:

```
hmd_mqa = Haver.metadata(database='havermqa')
hd_mqa1 = Haver.data(hmd_mqa)
hd_mqa2 = Haver.data(hmd_mqa, frequency='q')
```

To determine which codes have been successfully queried and which ones got dropped, you can use the `Haver.codelists()` function:

```
ex_cl = Haver.codelists(hd_mqa1)
print( ex_cl.keys() )
print( ex_cl.values() )
print( ex_cl['noagg'] )
```

Both `hd_mqa1` and `hd_mqa2` contain a lot of data:

```
print( hd_mqa1.shape )
```

You do have the option of printing a subset of the data set to the screen by indexing into the DataFrame, of course. For example:

```
print( hd_mqa1.iloc[-10:,[0:3 4:7]] )
```

```
>>> Haver.path('restore')
```

4.4 Haver Meta-DataFrame

4.4.1 Description

A Haver Meta-DataFrame is the return type of `Haver.metadata()`. It contains metadata of time series stored in Haver Analytics databases.

To learn about the many possibilities that pandas DataFrames provide for indexing into the data and for data analysis in general, consult the pandas documentation.

A second use of Haver Meta-DataFrames is to use them as input for `Haver.data()`.

If you have not done so yet, have a look at *Haver.metadata* before reading this help entry.

4.4.2 Details

Metadata Fields

A Haver Meta-DataFrame contains 18 pieces of information for each series code. These pieces of information are referred to as 'fields' in the following. The table below lists the available fields.

Side note: In the Haver Analytics data browser 'DLXVG3', a subset of these fields is called 'series parameters' and can be accessed from the 'Tools' menu or through the 'Alt+F10' keyboard shortcut.

Field name	pandas dtype	Field description
database	object (str)	the database name (excluding the path to the database)
code	object (str)	the Haver series code
startdate	date-time.date	last day of the series starting period
enddate	date-time.date	last day of the series ending period
frequency	object (str)	the frequency code; one of C, D, W, M, Q, A (calendar daily (7-day), daily (5-day), weekly, monthly, quarterly, annual)
descriptor	object (str)	the series description (label)
numobs	numpy.int32	number of periods between (and including) startdate and enddate
datetimemod	pandas Timestamp	the time the series was last modified
magnitude	numpy.int8	the magnitude of the series (e.g. 3 for series expressed in thousands)
decprecision	numpy.int8	the decimal precision (number of digits after the decimal point)
diftype	numpy.int8	0/1 value: 1 if data can contain or contain non-positive values, 0 otherwise
aggtype	object (str)	the aggregation type; one of AVG, SUM, EOP, NDF, NST (average, sum, end-of-period, not defined, not set)
datatype	object (str)	one of US$, LocCur, US$/LC, LC/US$, INDEX, Units, %, RATIO or SCALAR (where LocCur and LC stand for local currency)
group	object (str)	the DLX update group
geography1	object (str)	primary Haver geography code
geography2	object (str)	secondary Haver geography code
shortsource	object (str)	the abbreviated name of the time series publisher
longsource	object (str)	the full name of the time series publisher

Since pandas does not have a dedicated string type, all fields whose values are strings have to be stored as pandas objects. The individual elements are of type `str`.

Field 'aggtype' controls the temporal aggregation arithmetics for a particular series. `Haver.data()` performs temporal aggregation according to the values of this field. In some DLX software components it is possible to apply a custom setting to the aggregation type for a particular series. Within the Haver package for Python, however, this functionality does not exist. Temporal aggregation is always performed according to the 'aggtype' field entry of a series.

Field 'diftype' controls the kinds of math functions that are allowed for a particular series. In particular, it controls whether percentage and log transformation are permitted (diftype=0) or not (diftype=1).

Internally, some fields are stored in an encoded format, i.e. as a numeric value. Changing the default behavior of `Haver.metadata()` by setting argument `decode` to `False` will store numeric values in the returned Haver Meta-DataFrame. In rare cases, it may be useful to access the numeric values. The list of encodings is:

Field name	Encoding
frequency	10=annual, 30=quarterly, 40=monthly, 51=weekly Monday … 57=weekly Sunday, 60=daily (5-day), 61=calendar daily (7-day)
aggtype	1=AVG, 2=SUM, 3=EOP, 9=NDF, 0=NST

`haverinfo` Member Dictionary

A Haver Meta-DataFrame has a special member dictionary called 'haverinfo' that stores additional information about the query. Available keys are:

Dict Key	Dict Contents
description	query date and time (str)
vardescription	DataFrame column labels (list of str)
codeformat	default display format of series code lists (list)
pkgversion	the Haver package version that created the Haver Meta-DataFrame (str)
hasfuncs	whether math functions were part of the query

Warning: The `haverinfo` member is not included in copy operations.

For example, the Haver DataFrame `hmd2` in

```
hmd2 = hmd.ix[0:10].copy()
```

will not have a member called `haverinfo`.

4.4.3 Filtering Operations

A frequent operation is to filter a Haver Meta-DataFrame by matching the values of a column to a list of values or to a pattern. There several ways to accomplish this, two of which are presented here:[1]

1. Filter on the values of a column.
2. Copy the values of the column into the index and then use the powerful filter methods that the index provides.

Strategy 1 is typically appropriate when you want to filter on any column but on column 'code'. For example, you may want to reduce a Haver Meta-DataFrame by matching column 'geography1' to a list of geographic codes you are interested in. One method that suggests itself for this type of operation is `df.FIELDNAME.isin()`. See the pandas documentation for details.

Strategy 2 is extremely helpful when you want to filter codes. The most helpful DataFrame methods are `df.drop()`, `df.reindex()`, `df.loc[]`, and `df.filter()`. Note that the axis default may differ for these methods. To be safe, you can always specify the argument `axis=0` to make sure you operate on the index and not the columns.

The examples section below includes, among other things, illustrations of both of the above strategies.

[1] A useful and efficient approach that is omitted here is to use `pandas.DataFrame.merge()`.

4.4.4 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

We start out by pulling in all metadata for 'HAVERMQA'.

```
hmd_mqa = Haver.metadata(database='havermqa')
```

When printing the data

```
print( hmd_mqa.loc[:,['code', 'frequency']] )
```

we see that there are monthly series between rows 75 and 101 (not all rows may be displayed on your screen due to pandas' `display.max_rows` option). We choose to assign the rows in question to another Haver Meta-DataFrame using row-indexing:

```
hmd_2 = hmd_mqa.iloc[74:101,:]
```

Let's do a frequency count:

```
print( hmd_mqa.loc[:,'frequency'].value_counts() )
```

We are interested in the monthly exchange rate series starting at row 79 of 'hmd_mqa'. Let's build a subset of codes that start with 'fx…'. We utilize regular expressions (see e.g. `re`)).

```
hmd_3 = hmd_mqa.loc[hmd_mqa.loc[:,'code'].str.contains('^fx'),:]
```

Before we can use these series for analysis we must determine whether they are quoted in an unambiguous way (units of domestic currency per units of foreign currency, or vice versa). The 'datatype' metadata field can help in this.

```
print( hmd_3.loc[:,'datatype'].value_counts() )
```

We see that the rates are not quoted uniformly, so we must do adjustments after pulling in the data and before doing analysis.

Let's look at another example. Imagine that we need exchange rate data going back to 1970. We can immediately determine which series fail this criterion:

```
hmd_4 = hmd_3.loc[hmd_3.startdate >
                  datetime.strptime('1970-12-31', '%Y-%m-%d').date(),:]
print( hmd_4.loc[:,['code','startdate','descriptor']] )
```

We invert the above filtering criterion to get the series that have the desired coverage. Then we feed the returned object as `codes` argument into `Haver.data()`:

```
hmd_5 = hmd_3.loc[hmd_3.startdate <=
                  datetime.strptime('1970-12-31', '%Y-%m-%d').date(),:]
hd_5  = Haver.data(hmd_5)
print( hd_5.iloc[:, 0:3] )
```

If we want to investigate several full databases, the best way is to stack DataFrames. Below we create a cross tabulation of the frequency and database fields of a Haver Meta-DataFrame that contains data from two Haver databases.

```
hmd_d = Haver.metadata(database='haverD')
hmd_full = hmd_mqa.append(hmd_d)

import pandas as pd
pd.crosstab(hmd_full.database, hmd_full.frequency)
```

A previous section mentioned different strategies regarding filter operations on a column or on the DataFrame index. We already gave a few examples on how to filter on the values of a column using boolean subsetting. Here is another one: We check for membership in a list of values:

```
ex_geolist = ['156', '158', '128', '142', '144', '146']
hmd_6 = hmd_mqa[hmd_mqa.geography1.isin(ex_geolist)]
```

Let's now look at code filtering operations using index methods:

```
hmd_7 = hmd_mqa.copy()
hmd_7.index = hmd_7.code.tolist()
print( hmd_7.head() )
```

Keeping a list of codes:

```
ex_codes = ['c', 'cdh', 'fxaus', 'fxcan', 'm', 'x']
print( hmd_7.reindex(labels=ex_codes) )
```

Alternatively:

```
print( hmd_7.loc[ex_codes,] )
```

Dropping a list of codes:

```
print( hmd_7.shape )
hmd_8 = hmd_7.drop(labels=ex_codes)
print( hmd_8.shape )
```

Filtering according to a regular expression:

```
print( hmd_7.filter(regex='^fx', axis=0) )
```

pandas DataFrames provide many more useful operations on indexes. See the pandas documentation for details.

```
>>> Haver.path('restore')
```

4.5 Haver ErrorReport dictionary

4.5.1 Description

A Haver ErrorReport dictionary is returned by `Haver.data()` and `Haver.metadata()` if the query is incorrectly specified, i.e. as soon as one series code or database cannot be found.

4.5.2 Details

The dictionary key-value pairs provide information on the database path used, the time of the query, and, most importantly, lists of codes that caused the query to fail. You can access these lists of series codes either directly or using the function *Haver.codelists*. The code lists are helpful for fixing up queries that have failed.

4.5.3 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

The database 'HAVERD' is found but it does not contain series 'wrong1' and 'wrong2', so a Haver ErrorReport dictionary is returned.

```
her = Haver.data(['wrong1', 'wrong2', 'fxtwb'], 'haverd', startdate='2012-02-20')
```

The list of codes that caused the query to fail as well as the list of good codes can be accessed with the `Haver.codelists()` function. The lists are stored as entries of a Python dict:

```
ex_cl = Haver.codelists(her)
print( ex_cl.keys() )
```

If we can do without the codes that could not be found, we can query the data of the good codes immediately:

```
hd = Haver.data(ex_cl['codesfound'], start='2012-02-20')
```

```
>>> Haver.path('restore')
```

4.6 Haver.path

`Haver.`**`path`**(*pset=None*, *save=False*)

Description

Set, query, and save the path to Haver databases. A correct path setting is a prerequisite for data and metadata queries run by *Haver.data* and *Haver.metadata*.

Note that `Haver.path()` is in no way related to the Python module search path.

Syntax

```
P = Haver.path()
Haver.path(pset)
Haver.path(save=True)
```

4.6.1 Arguments

pset
: [str or dict] Specifies the directory where Haver databases reside.

save
: [bool] Save current path setting to file.

4.6.2 Returns

str or dict
: When no argument is supplied, a str or dict that holds the current setting of the database path(s). If no path is set, an empty str is returned.

4.6.3 Notes

(Syntax Detail)

`Haver.path()` returns the current path setting.

`Haver.path(pset)` sets the database path to *pset*. *pset* can be one of several things:

- *pset* is typically a str specifying a directory. It must be supplied as an absolute path, not as a path relative to some parent directory. If the supplied database path does not exist, an error occurs. If the supplied database path does not contain any Haver Analytics database, a warning is issued.
- It can also be a special 'keyword': a str that is one of 'auto', 'ini', 'examples', 'restore', or 'saved', or any abbreviation thereof.
  - 'auto' tries to automatically detect the correct database path if it is unknown to the user. If successful, it will set the Haver path to the directory where the default database of the Haver data browser DLXVG3 resides. In some instances this may not work, depending on the Python configuration and the DLX configuration of your machine.
  - 'ini' reads the main DLX INI file, which contains database paths for individual databases. These are generally the databases that you see when you open the Haver data browser DLXVG3. Databases accessed in this way do not have to reside in the same directory. See also *Path Setting Details* below.
  - 'examples' sets the database path to the example DLX databases that come with the Haver package. Before doing so, `Haver.path()` will store the existing setting. You can re-enable this setting by issuing `Haver.path('restore')`.
  - 'restore' restores the database path setting that was in place when `Haver.path('examples')` was invoked.
  - 'saved' sets the path according to a previously saved state.
- *pset* can also be a dictionary, whose keys specify the database names and whose values are the paths to databases. This permits to access databases in different locations in a single query. Codes in databases not contained in the dictionary will not be found by queries.
- Finally, *pset* can be a str specifying a path to a file that has extension '.ini'. This file is then taken to be an 'INI' file. For details on INI files, see *Path Setting Details* below.

`Haver.path(save=True)` saves the current path setting to the Haver package settings file. This setting gets automatically restored upon import of the Haver package, or when reloading the Haver package. Note: Saving the Haver path affects all users that access a particular package installation.

(Package Import)

When the Haver package is imported, it first checks for and tries to restore a saved database path. If this is not successful, it tries to detect the correct setting from the DLX setup of your machine. If neither of these attempts are successful you have to set the database path manually.

(Usage with DLX Direct)

If *DLX Direct* mode is switched on, you can still set and query the database path. Be aware, however, that the local database path setting has no effect as long as you have DLX Direct mode turned on. See *Haver.direct* for more details.

(Directory Write Permission)

Since database operations are logged, you must have write permission to the database directory in order to access the databases.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.6.4 Path Setting Details

Alternatives to a Fixed Database Path

When setting the Haver path to a fixed directory, you can only access databases from that directory. Since all Haver databases typically reside in the same directory, this should be sufficient. In some situations, however, you may want to access databases from different directories. In such cases, it may be the simplest solution that you explicitly switch the Haver path from one directory to another. This may be turn out to be cumbersome, however. Moreover, you would not be able to access data from databases in different directories in a single query.

If you want to specify database paths on a per-database basis, there are two possibilities:

1. You can supply a dictionary to `Haver.path()` with database names as dictionary keys and database paths as dictionary values.
2. You can specify database paths in a so-called INI file. This functionality is most often used to read the main INI file of your DLX installation, which gives you access to all of your institution's databases. You may also create your own, customized INI file.

In both cases `Haver.path()` will store the information provided in a Python dictionary. Querying the current Haver path via `Haver.path()` will return this dictionary, and its information will be used in queries. As far as path management is concerned, a dictionary path setting behaves like a regular, fixed path setting: It can be saved in between Python sessions (see the next section), it can be restored after switching to examples databases via the keyword 'restore', etc.

The rest of this section elaborates on the usage and creation of INI files. INI files are DLX configuration files that contain lines that specify database locations on a per-database basis. The keyword 'ini' supplied as argument *vset* to `Haver.path()` will use the main INI file of the DLX installation on your computer. This file should always be present, so executing

```
>>> Haver.path('ini')
```

should always work and give you access to all databases of your DLX installation. It is therefore a good alternative to

```
>>> Haver.path('auto')
```

which will set the Haver path to the directory where the default database of the Haver data browser DLXVG3 resides.

In some rare instances, it may even be useful to create an INI file yourself. Relevant entries are composed as:

```
DBID [_someword_=] _dbpath_\_dbname_
```

These mimic entries in the main INI file. Lines that do not comply with the above pattern are ignored. More specifically, the first word of an INI entry must start with the string literal 'DBID', followed by some whitespace. The optional *_someword_=* may be any non-empty string that ends in '=' and does not contain '=' anywhere else. It is present in real DLX configuration files but can be omitted for Python purposes. *_dbpath_* is the path to the directory containing the database. It must be an absolute path, not a path relative to some parent directory. *_dbname_* must be the name of the database, without any file extension. Casing does not matter. Finally, you can comment out lines using '!'. A file containing the four lines

```
! my first database location file
DBID   c:\dlx\data\USECON
! DBID   c:\dlx\data\WEEKLY
DBID   d:\some\other\path\EMERGE
```

would give you access to databases USECON and EMERGE. WEEKLY will not be found since its line, like the first line, is taken to be a comment.

Note that such customized files only work with Python. You cannot use these files with any other component of DLX. Therefore, if you create such customized INI files, we recommend naming them accordingly, e.g., 'Haver_python_databases.ini'. It is always required that the file extension be '.ini' (casing does not matter).

Preservation of the Path Setting between Python Sessions

Normally you never have to worry about setting the Haver path, as the Haver package will typically apply the correct path setting automatically. Should this mechanism fail, however, you have to set the database path manually. After doing this, you can use `Haver.path(save=True)` to save this setting. That way it will get automatically restored upon package import and you do not have to manually set that Haver path repeatedly, e.g. in new Python sessions.

Warning: Be aware that, in an environment where multiple users share the same Haver package installation, saving a path setting affects all users. This is not true for any other path operations: They only affect a single user.

Running 'Examples' Sections of Help Entries of the Haver Package

At the top of the examples section in each help entry the database path is set to point to the example databases:

```
>>> Haver.path('examples').
```

At the end of each examples section, the previous setting is restored:

```
>>> Haver.path('restore').
```

If you re-set the Haver database path manually after issuing `Haver.path('examples')`, a call to `Haver.path('restore')` will not change this setting anymore.

4.6.5 Examples

Note: executing the following example statements will change the Haver database path setting.

Query the current setting (the returned path will most likely differ on your computer):

```
ex_origpath = Haver.path()
```

Let `Haver.path()` try to determine the correct setting automatically:

```
Haver.path('auto')
print( Haver.path() )
```

Set path to examples databases:

```
Haver.path('examples')
print( Haver.path() )
```

Restore the previous setting:

```
Haver.path('restore')
print( Haver.path() )
```

Restore to setting that was in place before the examples section was run:

```
Haver.path(ex_origpath)
```

Manually set the database path to an explicitly given directory:

```
Haver.path('c:/dlx/data')
```

4.7 Haver.databases

`Haver.`**`databases`**(*disp=False*)

Description

Print or return list of Haver Analytics databases.

Syntax

```
Haver.databases(disp=True)
dblist = Haver.databases()
```

4.7.1 Arguments

disp
: [bool] *disp=False* (the default) returns a list of Haver Analytics database files ('.DAT' extension files) that are accessible under the current setting of `Haver.path()`.

  disp=True prints the same list to the screen and returns *None*.

4.7.2 Returns

None
: returned by `Haver.databases(disp=True)`.

list of str
: returned by `Haver.databases()`. List of Haver Analytics database files found. The entries are in upper case and the '.DAT' has been removed. The list is build with respect to the current setting of `Haver.path()`.

4.7.3 Notes

(Usage with DLX Direct)

If *DLX Direct* mode is switched on, `Haver.databases()` returns an error. See *Haver.direct* for more details.

4.7.4 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

Let's look at the example databases that are shipped with the Haver package:

```
print( Haver.path() )
Haver.path('examples')
print( Haver.databases(disp=True) )
```

We can save the database names in a list of str:

```
ex_dbs = Haver.databases()
print( ex_dbs )
```

```
>>> Haver.path('restore')
```

4.8 Haver.dbcodes

`Haver.dbcodes(`*database*, *codeformat=['simple']*, *format='simple'*`)`

Description

Obtain a list of codes from a Haver database.

Syntax

```
C = Haver.dbcodes(database, codeformat=['simple'], format='simple')
```

4.8.1 Arguments

database
: [str or list or tuple of str with one element] The database whose codes you want to retrieve.

codeformat
: [str or list of str] Specifies the desired output format of Haver series code lists. The most important values are '' / 'simple', ':', and '@', which yield output lists in *ser*, *db:ser*, or *ser:db* format, respectively.

  In the context of `Haver.dbcodes()`, this argument determines the format of the code list that is returned by the function. Note that its default value here is 'simple'.

  For a detailed description of the `codeformat` specification, see the help entry for function *Haver.fmtcodes*, section *The codeformat Specification* and examples.

  For more information about series code formats, see subsection *Series Code Formats* of the package introduction section.

format
: [str] This is a legacy argument that is still included for backwards compatibility. If you are writing new code, use argument `codeformat` instead. Old code that was written before the `codeformat` argument was introduced in package version 3.3.0 continues to work, except if you had specified the `format` argument as a positional argument with value 'a'. This used to be interpreted as 'at' (*ser@db* format), but is now interpreted as 'auto', which translates into *db:ser* format here.

  Possible specifications of `format` were 'full', 'colon', ':', 'at', '@', or 'simple', or any unambiguous abbreviation thereof, with obvious mappings into simple, *db:ser*, and *ser@db* formats.

4.8.2 Returns

list
: A list of series codes contained in `database`.

4.8.3 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

`Haver.dbcodes()` returns all codes in a database. The example database HAVERD only has three codes:

```
print( ex_codes = Haver.dbcodes('haverd') )
```

The fastest way to determine the size of a database is to check the `len()` of this list. This is a lot faster than using `Haver.metadata()` on an entire database and then checking the number of rows of the returned Haver Meta-DataFrame. Compare:

```
print( len(Haver.dbcodes('havermqa')) )
print( Haver.metadata(database='havermqa').shape[0] )
```

With large databases of 100,000+ codes, the speed differences are substantial, and particularly so when you work in *DLX Direct* mode.

Relatedly, you may be familiar with the technique of using `Haver.metadata()` to query all data of a database:

```
hmd = Haver.metadata(database='haverd')
hd  = Haver.data(hmd)
print( tail(hd, 10) )
```

You can achieve the exact same result using `Haver.dbcodes()`:

```
ex_codes2 = Haver.dbcodes(database='haverd')
hd2 = Haver.data(ex_codes2, 'haverd')
print( hd2.equals(hd) )
```

or:

```
ex_codes3 = Haver.dbcodes(database='haverd', codeformat='full')
hd3 = Haver.data(ex_codes3)
print( hd3.equals(hd) )
```

Note, however, that `Haver.dbcodes()` does not have the `histcodes` parameter that `Haver.metadata()` has. For databases that contain series with frequencies of weekly and higher, if you want to get a proper series count (i.e., not counting *historical component series*) , you have to resort to `Haver.metadata()`.

```
>>> Haver.path('restore')
```

4.9 Haver.limits

`Haver.`**`limits`**(*parameter=None*, *value=None*)

Description

Set and get limits for Haver Analytics data queries.

`Haver.limits()` handles the settings with regards to the limits of data queries of `Haver.data()`. Note that these limits have no impact on the metadata queries of `Haver.metadata()`.

Limits on the size of queries are imposed to protect you from unintentionally conducting very large queries that are problematic in terms of execution time or memory consumption.

Syntax

```
S = Haver.limits()
Haver.limits('default')
Haver.limits(parameter, value)
```

4.9.1 Arguments

parameter
[str] one of 'numcodes', 'numdatapoints', or 'default', or any unambiguous abbreviation thereof.

`parameter='numcodes'` sets the maximum allowed number of codes in a query to `value`.

`parameter='numdatapoints'` sets the maximum allowed number of data points in a query to `value`.

`parameter='default'` ignores `value`, if supplied, and restores default values for 'numcodes' and 'numdatapoints', which are 1,000 and 15,000,000, respectively.

value
[numeric non-negative scalar] specifies the limit for the 'numcodes' or 'numdatapoints' setting.

4.9.2 Returns

list
The numbers in the first and second position indicate the limits on the number of codes and the number of data points, respectively.

4.9.3 Notes

(Syntax Detail)

The values set by `Haver.limits()` are taken into account by `Haver.data()`. It will issue an error if a query is beyond what is allowed by query limits, unless you specify its option `uselimits='False'`.

`Haver.limits()`, with an empty argument list, returns the current settings.

`Haver.limits('default')` restores Haver default values.

Any values set by `Haver.limits()` will remain in effect until Python is restarted. A restart will result in the restoration of default values.

The default values for query limits allow you to conduct queries which are fairly sizeable. The default for the maximum number of codes is 5000. The default for the maximum number of data points is 15 million, which is

a little bit more than a daily data set with 1000 time series, covering 50 years of data. Such a data set requires more than 100 megabyte of memory, but the memory necessary for performing the retrieval is several times as high. Before you increase values for limits, make sure these are tenable for your system.

(Usage with DLX Direct)

Query limits apply only to local queries. In *DLX Direct* mode limits are ignored. See *Haver.direct* for more details.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.9.4 Query Speed and Memory consumption

Large data queries can become problematic with respect to execution time and memory consumption. Whether this is the case depends on the limits of your machine, and it may also depend the speed of your institution's network. The following paragraphs provide recommendations for managing speed and memory issues, in general terms.

Query Execution Time

Should a query take too long, you can interrupt execution as with other Python commands (CTRL+C). Still, after an interrupted query the memory claimed by Python is likely to be larger than before the query. You can try to reclaim memory by

```
import gc
gc.collect()
```

but this may not help in all cases. If you have interrupted a large query, it may be advisable to save your work and restart the Python console.

Memory Consumption

When specifying large queries, assessing a tenable level of memory consumption by `Haver.data()` should not be geared towards the total free memory on your computer, for the following reason:

1. The memory consumed by data query operations is likely to be a multiple of the memory consumed by the Haver DataFrame returned.
2. Commands issued after the query may also consume a large amount of memory. For example, if you create a copy of the returned DataFrame, you are requesting the amount of memory consumed by the Haver data set for a second object.

4.9.5 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

`Haver.limits()` returns a list with the current settings for limits. We use it here to record these settings in order to be able to restore them later.

```
ex_storedlimits = Haver.limits()
```

We reset the limit on the maximum number of codes allowed in a query:

```
Haver.limits('numcodes', 10)
print( Haver.limits() )
```

HAVERW has 21 codes. The limit we just set does not affect metadata retrievals:

```
hmd = Haver.metadata(database='haverw')
print( hmd.iloc[0:3,:] )
```

Querying the data for 21 codes is not possible with the example settings for limits. The following generates an error:

```
>>> hd = Haver.data(hmd);   # <- generates error!
```

A query for 10 codes still works:

```
hd = Haver.data(hmd.iloc[0:10,:])
```

We can also explicitly state that `Haver.data()` should ignore limits:

```
hd = Haver.data(hmd, uselimits=False)
```

With default values, the full 21 code query goes through of course:

```
Haver.limits('default')
print( Haver.limits() )
hd = Haver.data(hmd)
```

We restore the limits that were in place before the above examples were executed:

```
Haver.limits('numcodes'      , ex_storedlimits[0])
Haver.limits('numdatapoints' , ex_storedlimits[1])
print( Haver.limits() )

>>> Haver.path('restore')
```

4.10 Haver.codelists

Haver.**codelists**(*objin*, *codeformat=None*, *stripfunc=False*)

Description

Extract lists of Haver series codes.

`Haver.codelists()` extracts lists of Haver series codes from Haver Meta-DataFrames, Haver DataFrames, and Haver ErrorReport dictionaries.

These objects contain information on various Haver series code lists. *Haver.codelists* accesses this information. For the most part, this information is related to the section 'When Data Cannot Be Retrieved' of *Haver.data*.

Syntax

```
L = Haver.codelists(objin)
```

4.10.1 Arguments

objin : Haver ErrorReport dictionary or Haver DataFrame or Haver Meta-DataFrame

codeformat
[str or list of str] Specifies the desired output format of the returned Haver series code list. The most important values are '' / 'simple', ':', or '@', which yield return lists in *ser*, *db:ser*, and *ser@db* formats, respectively.

The default value if `codeformat` is omitted varies. It depends a) on the object supplied to `Haver.codelists()` and b) on the `codeformat` value under which that object had been created.

For a detailed description of the `codeformat` specification, see the help entry for function *Haver.fmtcodes*, section *The codeformat Specification* and examples.

For more information about series code formats, see subsection *Series Code Formats* of the package introduction section.

stripfunc
[bool] If `True`, strips all math functions from codes in all output code lists. Default: `False`.

4.10.2 Returns

dict of length 3 or 4, or list of str
See the Haver Python Reference for details.

4.10.3 Notes

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.10.4 Details

If the input argument to `Haver.codelists()` is a Haver DataFrame, the return value is a `dict`. Associated with each key is a list of str, whose elements are Haver codes in *database:seriescode* format. The following table lists the keys and describes the associated code lists:

argument type	element name	vector contents
HaverData	noobs	series that do not have any valid data points
	noagg	series that cannot be aggregated
	nodisagg	series that cannot be disaggregated
	nopct	series that cannot have a particular math function applied
	codesout	series whose data has been retrieved
HaverError dict	codesfound	series whose code and database were found
	databaseaccess	series whose database could not be accessed. Potential causes are: Misspelling of the database name; an incorrect Haver database path setting; lack of write permission to the database directory.
	codesnotfound	series whose series code was not found
	metadataaccess	series whose metadata could not be accessed

In addition to the above, if the input argument is a Haver Meta-DataFrame, `Haver.codeslists()` returns a list of str instead of a dict. Its elements hold the full series codes of series with records in the table.

4.10.5 Examples

Examples for `Haver.codeslists()` can additionally be found in the examples section of *Haver.fmtcodes*.

```
>>> Haver.path()
>>> Haver.path('examples')
```

When applying the function to a Haver Meta-DataFrame, a list of codes that have records in the DataFrame is returned.

```
hmd_w = Haver.metadata(database='haverw')
print( Haver.codelists(hmd_w) )
```

Applying the function to Haver DataFrames yields information on the successfully retrieved series and on the ones that got dropped:

```
hmd_mqa = Haver.metadata(database='havermqa')
hd_q    = Haver.data(hmd_mqa, frequency='quarterly')
print( Haver.codelists(hd_q) )
```

Lastly, one can apply `Haver.codelists()` to Haver ErrorReport dictionaries. These are returned by `Haver.data()` and `Haver.metadata()` when there is a fundamental flaw in a query. The code lists are returned as a Python `dict`.

```
ex_codes = ['wrongdb:code1', 'wrongdb:code2', 'haverd:wrongser']
her_hd  = Haver.data(ex_codes)

ex_cl = Haver.codelists(her_hd)
print( ex_cl.keys() )
print( ex_cl.values() )
print( ex_cl['databaseaccess'] )
```

The same object is returned from a correspondingly flawed metadata query:

```
her_hmd = Haver.metadata(ex_codes)
print( Haver.codelists(her_hmd) )
```

```
>>> Haver.path('restore')
```

4.11 Haver.verbose

`Haver.`**`verbose`**(*vset=None*)

Description

Set or query the verbose message mode of Haver functions

`Haver.verbose()` sets the message display mode of Haver functions to be on or off. When you set the message display mode to non-verbose, the Haver functions will not generate any printed output, except for warnings and error messages.

Syntax

```
M = Haver.verbose()
Haver.verbose(vset)
```

4.11.1 Arguments

vset
: [bool or numeric 0/1] A numeric 0 (1) or boolean `False` (`True`) turn the verbose mode off (on).

4.11.2 Returns

bool
: A logical `False` (`True`) indicates that the verbose mode is off (on).

4.11.3 Notes

(Syntax Detail)

`M = Haver.verbose()` returns a bool indicating whether the mode of displaying messages is on (`True`) or off (`False`).

`Haver.verbose(vset)` sets the display mode of messages on or off. `vset` must be a bool or a numeric 0/1 value.

The verbose setting concerns regular messages issued via the `print()` function only. Warnings and error messages are not affected. You can handle the display of warnings and errors as usual in Python.

Upon package import, the Haver verbose mode is always turned on. Unless there is a specific reason to set the verbose mode to `False`, we recommend not doing so.

4.11.4 Examples

```
>>> Haver.path()
>>> Haver.path('examples')
```

We first save the current setting for the verbose mode:

```
ex_verb = Haver.verbose()
```

Verbose mode settings affect data queries, among other things:

```
hmd = Haver.metadata(database='havermqa')  # preparatory statement
Haver.verbose(1)
hd  = Haver.data(hmd)

Haver.verbose(0)
hd  = Haver.data(hmd)
```

After verbose mode was turned off, we were not notified that two series got dropped from the query.

```
Haver.verbose(ex_verb)
```

```
>>> Haver.path('restore')
```

4.12 Haver.fmtcodes

`Haver.fmtcodes`(*codes*, *database=None*, *codeformat=[':', 'full']*, *check=True*, *function=None*, *stripfunc=False*, *separate=False*)

Description

Format lists of Haver series codes.

Syntax

```
L = Haver.fmtcodes(codes, database=None, codeformat=[':', 'full'], [...])
```

4.12.1 Arguments

codes
[str or list of str] Specified as in *Haver.metadata*.

database
[str or list or tuple of str with one element] Specified as in *Haver.metadata*.

codeformat
[str or list of str] Specifies the desired output format of Haver series code lists. The most important values are '' / 'simple', ':', and '@', which yield output lists in simple, *db:ser*, or *ser:db* format, respectively.

In the context of `Haver.fmtcodes()`, this argument determines the format of the code list that is returned by the function.

For a detailed description of the `codeformat` specification, see the subsection *The codeformat Specification* of this help entry.

check
[bool] Determines whether the input list of codes should be checked for validity. One example for a check that is applied is the search for non-alphanumeric characters. The `check` argument does not comprise checking whether the series codes exist in the database(s).

function
[str or int] Specifies a default math function to be applied to all codes that are not explicit function codes. For math function names and their numeric equivalents see *Math Functions* in the details section of `Haver.data()`.

stripfunc
[bool] If `True`, strips all math functions from codes in all output code lists. Default: `False`.

separate
[bool] If `True`, returns a 4-element list of lists, each of which containing a single code component: The simple codes, the database specifications, the opening parts of math function specifications, and the closing parts of math function specifications. This order is independent of argument `codeformat`. If a code is not a function code, the last two lists contain empty strings in the appropriate position. If no code in the input list is a functions code, elements 3 and 4 of the returned list are empty lists. Under `codeformat='simple'`, the second list (databases) will be an empty list. Default: `False`.

4.12.2 Returns

list of str
A list containing the formatted Haver series codes.

4.12.3 Notes

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.12.4 The codeformat Specification

The `codeformat` argument applies to several functions. Its specification is laid out in more detail in this section.

Please familiarize yourself with *Series Code Formats* first before reading on.

Important Basics

While a complete description of the `codeformat` argument as it applies to different functions is lengthy, most use cases can be summarized with just a few rules. This section does that. In addition, applying `codeformat` in practice is easier than the complete description suggests. See the examples section below if you would like to learn about the argument in a more hands-on way.

As another preliminary remark, remember that `codeformat` is only about output lists. Input lists to functions (e.g., code input lists to `Haver.metadata()`) can always be in *db:ser* format, *ser@db* format, or a mixture thereof. Output lists of functions, as determined by `codeformat` will be homogeneous, i.e., they adhere to either simple, *db:ser* or *ser@db*, but never to a mixture of them.

The *canonical form* of the argument `codeformat` is, in informal syntax notation,

codeformat = [*codesep* , *codefmt*]

where

codesep = { '' | ':' | '@' }
codefmt = { 'simple' | 'auto' | 'full' }

codesep is one of '', ':', or '@', which determines the series code separator and thereby the series code formats 'simple', *db:ser*, and *ser@db*, respectively. The second element, *codefmt*, is one of 'simple', 'full', and 'auto', or any abbreviation thereof. The order of appearance of the two elements does not matter. The Haver package documentation as well as function default values typically use the canonical form above.

As shortcuts to the canonical form, you can also supply just one element, in which case it can be either passed as a str or a one-element list. This is useful since most use cases can be covered by simply specifying one of '' / 'simple', ':', or '@'. They map as follows:

Value	Maps to	Meaning
'simple'	['' , 'simple']	simple code format
''	['' , 'simple']	simple code format
':'	[':' , 'full']	*db:ser* format
'@'	['@' , 'full']	*ser@db* format

There are additional single-element mappings which are given further below.

Regardless of the value of *codefmt*, lists contained in a *Haver ErrorReport dictionary* and a *haverinfo Member Dictionary* always receive a format of *codefmt*='full'. You can only choose between *db:ser* (default) and *ser@db* formats via *codesep*=':' or *codesep*='@'. If you want to obtain those lists in a different format, you can apply the `codeformat` argument of the extraction function `Haver.codelists` without any restrictions. Additionally, you can always run `Haver.fmtcodes()` on a list directly without any restrictions.

Detailed Exposition

You can see some redundancy contained in the canonical form. For example, specifying either *codesep*='' or *codesep*='simple' imply a 'simple' code format, so the canonical form `['', 'simple']` is redundant. The reason why `codeformat` has a two-element canonical form that sometimes contains redundancy is a) because it must allow for backwards compatibility with argument defaults of previous package versions and b) complications introduced by `Haver.data`.

Complications for `Haver.data` are due to the following: Normally `codeformat` is about output code lists. `Haver.data()` generates those output code lists when `Haver.codelists()` is run on its return value (a Haver DataFrame). However, in addition to that, the question arises which code format should be used for the labeling of the columns of the Haver DataFrame. For `Haver.data()`, a combination of *codesep* and *codefmt* determines that. *codefmt*='auto' applies simple series codes when there are no duplicate simple series codes, and full series codes otherwise. For the full series code format, *codesep* pins down whether *db:ser* or *ser@db* is used in the column labels. *codesep* in addition determines the format of the output code lists. *codefmt*='auto' is only relevant for `Haver.data()`. Other functions automatically convert 'auto' to 'full'.

There are 3 x 3 = 9 possible combinations of *codesep* and *codefmt*, but not all of them are allowed. Allowed are only those where *codesep* and *codefmt* are not in contradiction with one another:

```
['' , 'simple']
[':'  ,  'full']
['@' , 'full']
[':'  ,  'auto']
['@' , 'auto']
```

Not allowed are therefore:

```
['' , 'full']
['' , 'auto']
[':'  ,  'simple']
['@' , 'simple']
```

Again, `Haver.data()` has some exceptions to the above. It does allow

```
[':'  ,  'simple']
['@' , 'simple']
```

which apply a simple code format to DataFrame column labels and a *db:ser* (or *ser@db*) format to output code lists.

Partly because of the redundancy of the canonical form, and partly to conform to default values of previous package versions, the following mappings of single-element specifications apply (extending the table from above):

Value	Maps to
`''`	`['' , 'simple']`
`'simple'`	`['' , 'simple']`
`'auto'`	`[':'  ,  'auto']`
`'full'`	`[':'  ,  'full']`
`':'`	`[':'  ,  'full']` (but to `[':'  ,  'auto']` for `Haver.data()`)
`'@'`	`['@' , 'full']` (but to `['@' , 'auto']` for `Haver.data()`)

4.12.5 Examples

This example section contains examples not just for `Haver.fmtcodes`, but also for other functions that have the `codeformat` argument.

```
>>> Haver.path()
>>> Haver.path('examples')
```

All input formats are always accepted.

```
ex_cl = ['havermqa:c', 'fxtwb', 'xnet@havermqa']
hmd = Haver.metadata(ex_cl, database='haverd')
Haver.codelists(hmd)
```

The basic output formats are:

```
print( Haver.fmtcodes(ex_cl, database='haverd', codeformat='simple') )
print( Haver.fmtcodes(ex_cl, database='haverd', codeformat='') )
print( Haver.fmtcodes(ex_cl, database='haverd', codeformat=':') )
print( Haver.fmtcodes(ex_cl, database='haverd', codeformat='@') )
```

Haver ErrorReport dictionaries and the haverinfo member dictionary always contain full Haver codes.

```
her = Haver.data(['gdp', 'junk'], 'havermqa', codeformat='simple')
print( her )
Haver.codelists(her)
```

You can re-format them using `Haver.codelists()`.

```
Haver.codelists(her, codeformat='simple')
```

Setting `codesep='@'` at query time also works.

```
her_2 = Haver.data(['gdp', 'junk'], 'havermqa', codeformat='@')
Haver.codelists(her_2)
```

The same rules apply to the haverinfo member dict.

```
hd_2 = Haver.data(['c', 'gdp', 'inet'], 'havermqa', frequency='q', codeformat='')
print( Haver.codelists(hd_2) )

hd_3 = Haver.data(['c', 'gdp', 'inet'], 'havermqa', frequency='q', codeformat='@')
print( Haver.codelists(hd_3) )

print( Haver.codelists(hd_3, codeformat='simple') )
```

`Haver.dbcodes()` defaults to simple Haver codes.

```
ex_cl_2 = Haver.dbcodes(database = 'haverw')
print(ex_cl_2)

ex_cl_3 = Haver.dbcodes(database = 'haverw', codeformat='@')
print(ex_cl_3)
```

You can always re-format using `Haver.fmtcodes()`.

```
Haver.fmtcodes(ex_cl_3, codeformat=':')
```

All of the above also works with function codes. For example:

```
ex_cl2 = ['diffl(havermqa:c)', 'yryrl(fxtwb)', 'difal(xnet@havermqa)']
hmd_2 = Haver.metadata(ex_cl2, database='haverd')
print( Haver.codelists(hmd_2, codeformat='@'))

hd_4 = Haver.data(ex_cl2, database='haverd')
print( hd_4 )
print( Haver.codelists(hd_4) )
```

```
>>> Haver.path('restore')
```

4.13 Haver.direct

`Haver.`**`direct`**(*mode=None*)

Description

Set and query the DLX Direct mode. If *DLX Direct* mode is switched on, databases access is directed to a Haver server. Otherwise local databases are accessed.

Syntax

```
M = Haver.direct()
Haver.direct(mode)
```

4.13.1 Arguments

mode
: [str or int or bool] Specifies whether to switch DLX Direct on or off. `mode` must be one of 'off'/'on', 0/1, or False/True. In addition, 'force' is allowed (see below).

4.13.2 Returns

bool
: When no argument is supplied, a bool that holds the current DLX Direct setting; and None otherwise.

4.13.3 Notes

(Syntax Detail)

`Haver.direct()` returns the current DLX Direct setting as a bool.

`Haver.direct(mode)` switches DLX Direct on or off.

(Authentication)

Upon package import, DLX Direct is always turned off. That is, the DLX Direct mode is *not* preserved between Python sessions. Consequently, if you want to use DLX Direct, you have to issue `Haver.direct('on')` after each restart of the Python kernel. At this point, two-factor authentication may be invoked, in which case a DLX window will open up, asking you for your email address and password. Then a security code will be sent to your email address. After entering the security code along with your password again, you are logged in.

Two-factor authentication will only be invoked again after a certain time interval. If you quit and restart Python before two-factor authentication has expired, a call to `Haver.direct()` requires no re-entry of credentials. An important implication of this is that there are barely any restrictions for scripting and automating tasks. If you have questions concerning details of the authentication process, please contact Haver support.

Once you are logged on to DLX Direct in a Python session, you can turn DLX Direct mode on and off, instantly, and as often as you please.

Important: If your query wholly fails due to database access errors and you have verified that your query specification should work, try issuing `Haver.direct('force')`.

You can determine whether your query wholly failed due to database access by either looking at the text of an error message (if a _HaverPkgError had been thrown) or by verifying that a returned Haver ErrorReportDictionary has all entries in the 'databaseaccess' code list. While this is frequently due to query misspecification, for example, because of a misspelled database name, it may happen in rare cases that the cause is a lost connection to the Haver server. `Haver.direct('force')` will then try to re-establish it. If the attempt fails, it will tell you so. In this case, you must restart your Python console and re-issue `Haver.direct(1)` before you can successfully execute your query.

(Equivalence of Local and DLX Direct Queries)

There are barely any restrictions on the metadata and data queries that you can execute with DLX Direct mode switched on. Almost any query that executes using local databases will also execute as a DLX Direct query.

(DLX Direct Query Speed and Interrupting Execution)

The equivalence of local and DLX Direct queries is, naturally, subject to the caveat that queries over the internet take much longer than queries that use the local network. For example, querying the metadata of the local USECON database may take five seconds. That very same query, when run under `Haver.direct('on')`, may take two minutes (the exact timings in your setting may deviate from these numbers, of course). Therefore, if you have large-scale tasks in place that run on local databases, you may encounter difficulties executing these same tasks under DLX Direct mode, simply because the execution time is much longer.

Another notable difference of DLX Direct to local queries is that DLX Direct queries often will not stop if you try to interrupt them (CTRL+C). In those cases, if you are adamant about aborting the query, you have to restart the Python interpreter. To avoid such situations, we recommend the following:

- When you first start out doing DLX Direct queries, start small, see how long the queries take, and gradually work your way to larger queries. You can look at this command's examples section, which lists queries that grow in size.
- Whenever you do larger DLX Direct queries, think for a second about the time contstraints imposed by the remote processing, and whether the expected execution time is acceptable to you.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

4.13.4 Differences Between Local and DLX Direct Queries

Query speed and query interruption aside, there are very few and only minor exceptions to the equivalence of local and DLX Direct functionality. They are listed below.

- `Haver.databases()` errors out if it is called under DLX Direct mode.
- Differences in query results arise if local databases are not up to date. In rare cases, differences may additionally occur in terms of N/A paddings at the start and/or end of returned DataFrames, even if local databases have been fully updated.
- Query limits, whether set by *Haver.limits* or default ones, only apply to local queries, for the following reason: With remote queries, establishing whether a limit has been broken can take a considerable amount of time. Oftentimes it would be counterproductive to stop execution at that point.

4.13.5 Examples

While example sections of other Haver commands are based on queries to example databases that ship with the Haver Python package, the examples for `Haver.direct()` below are cast in terms of live Haver databases. Your institution's subscription must include these databases for the examples to work. If one or several example statements use databases that you cannot access, you can try to rewrite these statements in terms of databases that are available to you.

Before using DLX Direct, you must switch it on:

```
Haver.direct(1)
```

Our first DLX Direct query concerns the metadata from a small database, WBPRICES.

```
hmd = Haver.metadata(database='wbprices')
print( hmd )
```

You can switch DLX Direct off, do local queries, and instantly turn DLX Direct mode back on:

```
Haver.direct(0)
Haver.path('examples')
print( Haver.data('havermqa:gdp') )
Haver.path('restore')

Haver.direct(1)
print( Haver.data('usecon:gdp') )
```

You can use query functions without any restrictions on the function parameters; for example:

```
ex_codes = ['usecon:gdp', 'usecon:lanagra', 'eudata:s132gdpn', 'eudata:s134gdpn']
(hd, hmd, info) = Haver.data(ex_codes, startdate='1980-01-01',
                             frequency='A', rtype='3tuple')
print( hd )
```

Since DLX queries oftentimes do not stop when you try to cancel them (CTRL+C), it is important that you can assess their expected processing time. Below, we provide example queries that that will successively take longer. By executing them, you can quickly get a feel for the speed of DLX Direct queries in your particular setting. In the examples below, note that the rough number of codes queried and the rough expected execution time are as of September 2021. The expected execution time is dependent on your particular network setting and may therefore deviate considerably from the numbers given.

Task: Getting lists of codes of an entire databases:

```
import timeit
args = {'globals': globals(), 'number': 1}
tt = timeit.timeit
print( tt("Haver.dbcodes(database='wbprices')", **args) ) #     190 codes,     1 sec
print( tt("Haver.dbcodes(database='usecon')"  , **args) ) #  64,000 codes,   3-5 sec
print( tt("Haver.dbcodes(database='cmdty')"   , **args) ) # 300,000 codes, 15-20 sec
```

Task: Getting metadata of an entire database:

```
print( tt("Haver.metadata(database='wbprices')", **args) ) #    200 codes,   2-4 sec
print( tt("Haver.metadata(database='fdic')"    , **args) ) #  3,500 codes,  5-10 sec
print( tt("Haver.metadata(database='g10')"     , **args) ) # 19,000 codes, 20-30 sec
print( tt("Haver.metadata(database='usecon')"  , **args) ) # 64,000 codes,   1-3 min
```

Task: Querying data from a single database:

```
ex_codes_g10  = Haver.dbcodes('g10'   , format='full')
print( tt("Haver.data(ex_codes_g10[:10]  , frequency='A')", **args) ) #  1-3 sec
print( tt("Haver.data(ex_codes_g10[:100] , frequency='A')", **args) ) #  1-3 sec
print( tt("Haver.data(ex_codes_g10[:1000], frequency='A')", **args) ) #  4-6 sec
print( tt("Haver.data(ex_codes_g10[:5000], frequency='A')", **args) ) # 8-12 sec
```

As an aside to the above, note that `Haver.dbcodes()` is the quickest way of determining the size of a database.

```
print( tt("Haver.dbcodes('g10')", **args) )
print( len(ex_codes_g10) )
```

Task: Querying data from multiple databases. This is somewhat slower than single-database queries. We first shuffle codes from three different databases in order to get an input code list.

```
ex_codes_eme  = Haver.dbcodes('emerge', format='full')
ex_codes_pop  = Haver.dbcodes('unpop' , format='full')
ex_codes_mix  = ex_codes_g10[0:2000] + ex_codes_eme[0:2000] + ex_codes_pop[0:2000]
import random
random.seed(123456)
random.shuffle(ex_codes_mix)
print( ex_codes_mix[:10] )

print( tt("Haver.data(ex_codes_mix[:10]  , frequency='A')", **args) ) #   5-7 sec
print( tt("Haver.data(ex_codes_mix[:100] , frequency='A')", **args) ) #   5-7 sec
print( tt("Haver.data(ex_codes_mix[:1000], frequency='A')", **args) ) #   6-8 sec
print( tt("Haver.data(ex_codes_mix[:5000], frequency='A')", **args) ) # 20-30 sec
```

Task: Querying high-frequency (daily) data. Querying data of a higher frequency also may take a bit of extra time. Moreover, for series of daily frequency we must use `Haver.metadata()` to get a proper database series list that excludes *historical component series*. The next few statements will construct our input code list:

```
hmd_dly = Haver.metadata(database='daily', histcodes='drop')  # 30-60 sec
ex_codes_dly = (hmd_dly.database + ':' + hmd_dly.code).tolist()
print( len(ex_codes_dly) )
```

We are now ready to time queries of daily series:

```
print( tt("Haver.data(ex_codes_dly[:10]  , frequency='D')", **args) ) #  1-3 sec
print( tt("Haver.data(ex_codes_dly[:100] , frequency='D')", **args) ) #  1-3 sec
print( tt("Haver.data(ex_codes_dly[:1000], frequency='D')", **args) ) # 8-12 sec
```

Warning: We are now going to query 5000 series of daily data. This is very often demanding in terms of memory requirements. It is likely to require several gigabyte of RAM. Only proceed if your system can handle this. Note that for local queries, you are better protected against running into queries that overburden your system via *Haver.limits*. You do not have this protection for DLX Direct queries.

```
print( tt("Haver.data(ex_codes_dly[:5000], frequency='D')", **args) ) #   1-2 min
```

If you would like to go back to local queries, simply switch DLX Direct off again:

```
Haver.direct(0)
```

4.14 Haver.usermode

`Haver.usermode(mode=None)`

Description

Set and query the user mode. The default is single-user. You should not change this setting unless technical support asks you to do so.

Syntax

```
M = Haver.usermode()
Haver.usermode(mode)
```

4.14.1 Arguments

mode
: [str] Specifies whether to use single-user mode or multi-user mode. `mode` must be one of 'singleuser' or 'multiuser', or any unambiguous thereof.

4.14.2 Returns

str
: When no argument is supplied, a str that holds the current user mode setting; and None otherwise.

4.14.3 Notes

(Syntax Detail)

`Haver.usermode()` returns the current user mode setting.

`Haver.usermode(mode)` sets the user mode to either single-user or multi-user

(User Modes)

The default user mode, 'singleuser', gives you fast access to local databases. In rare cases, however, this database access method can interfere with other processes accessing the database, which can lead to problems. Therefore, an additional access method, multi-user mode, is available. It provides safe database access, but at the cost of a substantial slowdown in query speed.

The default setting is 'singleuser', i.e., the fast access version.

Do not change the user mode unless a technical support person asks you to do so.

Note: Changing the user mode may fail during a running Python session. In that case, restart Python and change the user mode right after the `import Haver` statement.

The user mode is only relevant for local queries, not for *DLX Direct* queries.

The user mode setting is preserved in between Python sessions.

CHAPTER

FIVE

EXTENSION FUNCTIONS FOR MODIFYING HDM DATABASES

5.1 Introduction

This part of the documentation assumes that you are already familiar with Haver queries.

You can use the HDM functionality only if your institution has a current HDM subscription with Haver Analytics.

HDM is not supported in 32-bit Python.

5.1.1 The Haver Data Manager (HDM) Tool

The Haver Data Manager (HDM) provides clients with the capability to maintain DLX databases of their own proprietary time series information. These databases can be used in precisely the same way as DLX databases managed by Haver Analytics. In particular, you can query data from your HDM databases using Python, Excel, or any other interface that Haver Analytics offers. The only difference to regular databases is that you are responsible for creating and maintaining the contents of the database.

Note: The HDM functions that modify data only work on databases that were created by HDM. They do not operate on regular Haver databases.

5.1.2 Installation

For installation instructions, please follow the Excel HDM User Manual.

If you cannot copy the HDM API DLL file into c:\windows, see *Haver.hdm.apipath*.

5.1.3 Overview of HDM Data Modification in Python

As you have to set the path to regular Haver databases, you have to specify the path to your HDM databases. While regular databases typically reside in a single directory, you can choose to have your HDM databases in different directories. This requires you to call `Haver.hdm.path()` more often for switching directories, but if you are willing to do so, it is perfectly possible. In general, *Haver.hdm.path* works much in analogy to *Haver.path*.

In order to create one or more series in an HDM database, you have to supply the data as well as the metadata. Both pieces of information are supplied in the form of pandas DataFrames. The two Input-Frames are almost identical to the frames you receive when you execute a `Haver.data()` query with the `rtype='2tuple'` argument; that is, they are almost identical to a *Haver DataFrame* plus a *Haver Meta-DataFrame*.[2]

[2] They are not fully identical because they do not carry the `haverinfo` member dictionary.

It is easy to query data from a regular or an HDM database and feed them into another HDM database: You just query the data using `Haver.data()` in conjunction with its option `rtype='2tuple'`. This gives you two separate frames holding the data and metadata. You will likely have to modify the group field in the metadata frame - more about this later - but after that you can supply both frames as input to `Haver.hdm.create()`, which will create the series in the target database.

If you want to create series that contain data from your own (non-DLX/HDM) sources, you generate two empty DataFrames using `Haver.hdm.inputframes()` that you have to fill with time series data and their metadata. You use `Haver.hdm.create()` to write the series to the HDM database.

Before you can start creating series, however, you will have to learn about one additional concept: series groups. Series in all Haver databases are organized in groups. You can create, delete, and rename groups, thereby structuring your HDM databases to your liking. The functions that operate on groups are the same ones that you use for operating on series: `Haver.hdm.create()`, `Haver.hdm.delete()`, and `Haver.hdm.inputframes()`. Groups are treated in detail in section *Series Categories and Groups*.

To sum up, you can create new groups and new series based on your own data; and you can transfer groups and series from any database into your HDM databases. The following table shows the sequence of functions that applies to each one of these purposes.

	New group	New series	Transfer group	Transfer series
Haver.data()				X
Haver.hdm.groups()			X	
Haver.hdm.inputframes()	X	X		
Haver.hdm.create()	X	X	X	X

Naturally, you can also delete content, and you will always use `Haver.hdm.delete()` for this. Changing the time series data of existing series is possible via `Haver.hdm.update()`. You can also change the series code and the series metadata, e.g. the series labels. However, this requires series re-creation. This can be done quickly by querying the series, modifying the returned data and metadata frames in the desired way, and feeding them back in using `Haver.hdm.create()`, possibly in conjunction with its `replace=True` switch.

Note: The three content-modifying functions are `Haver.hdm.create()`, `Haver.hdm.delete()`, and `Haver.hdm.update()`.

When something goes wrong, there are multiple mechanisms in place to help fixing up incorrect content modification requests. You can read about these mechanisms in section *HDM Error Handling*.

5.1.4 Interrupting Content Modification

We strongly recommend to refrain from interrupting code execution (CTRL+C) when using content-modifying functions. Doing so will, at best, leave you with partially modified content. At the worst, it can corrupt your HDM database.

Warning: Interrupting (CTRL+C) content modification may corrupt your HDM database.

Rather than running the risk of having to interrupt content modification functions, start out with smaller database modification tasks in order to get a feel for the time they take on your computer/network. Then move on to larger tasks that you expect to finish quickly enough. Since databases can get corrupted, it is prudent to create backup copies frequently.

5.1.5 Code Examples

Example Database

The Haver package for Python contains one HDM example database. It is called 'HDMEX'. It will be used in many code examples. In addition to that, you can use it for playing around with HDM content modification functions.

Some code examples will create new content and some will modify or delete existing content. Consequently, the database content of the example database will fluctuate. In order to make sure that all examples can build on the database content they need, function *Haver.hdm.exampledb* prepares the database for the execution of a particular example section. For instance, the example section of `Haver.hdm.create()` has a statement:

```
Haver.hdm.exampledb('prep_create')
```

at the beginning of the code.

You can always reset the HDM example database to its original state using:

```
Haver.hdm.exampledb('reset')
```

See the function's help entry for more details.

Path Settings

As laid out in section *Setting the Haver Database Path to Actual and Example Databases*, example code needs to run on correct path settings. Therefore, code examples are typically enclosed by the statements:

```
Haver.hdm.path('examples')
```

and:

```
Haver.hdm.path('restore')
```

Whenever needed, the same is done for the path to regular Haver databases, i.e. using `Haver.path()`.

Adding the Example Database to DLXVG3

While learning the features of HDM for Python, it will be of great value if you are able to look at example database content in DLXVG3. That way you can verify the database changes that you make using Python statements. Please search the Excel HDM User Manual for "DLX View & Graph". This will point you to instructions on how to add a database to the DLXVG3 menus. A necessary piece of information for this is the full file path to the example database. Executing the following statements will display it

```
import Haver, os
os.path.split(Haver.__file__)[0] + '\\dat\\HDMEX.DAT'
```

When verifying changes to the database note that you may have to back out from the menu page you are looking at and then pull it up again. Suppose that you have added series to group 'E01' and that the menu page of 'E01' was already open when you created the series. At this point the new content is not yet displayed. You have to move up one level in the menu hierarchy (e.g. by hitting escape) and then click on the 'E01' link again. Then you can see the newly created series.

Data Range

Many code examples combine the creation of new (fictitious) data in the HDM example database with data from queries on the regular Haver example databases. The latter have never changed since the release of Haver's various interfaces to statistics packages. They will continue to hold the very same data in the future. That way all examples will always be exactly replicable. The downside is that all examples use time periods that are somewhat outdated, ending around 2011.

Example Code Variable Names

Example code snippets in all section of this guide use the name 'hmd' for a variable that holds a Haver Meta-DataFrame ('hmd' is short for '**H**aver**M**eta**D**ata'). This can easily be confused with the abbreviation 'hdm' for **H**aver **D**ata **M**anager.

> **Warning:** Be careful to not mix up the abbreviations 'hmd' and 'hdm' in your code.

The most important objects of HDM package functions are the following: *Group-DataFrame*, *Input-DataFrame*, and *Haver Meta-DataFrame*. They will be described below and in other help entries. Whenever an example Python code statement in a help entry assigns such an object to a workspace variable, the names 'gd', 'ind', and 'inmd' are used (read: 'group data', 'input (series) data', 'input metadata'). All of the above names are sometimes appended with suffixes that are easily understood within the context ('gd_1', 'gd_2', 'inmd_d', etc.) of the example. Any other Python variables that are defined in example code start with 'ex_'.

5.1.6 Other Preliminaries

Casing

The HDM family of functions follows the same rules as laid out in section *Casing*. There is one important distinction: Group codes are - in contrast to series codes - always returned in upper case. That is, you will get return values like 'gdp', 'usecon', 'usecon:gdp', but 'K20' and 'S97'. In particular, this applies to code lists returned in a *Haver ErrorReport dictionary*.

> **Tip:** We recommend that you always specify series codes in lower case and group codes in upper case.

This is consistent with the casing used in Haver-Meta-DataFrames. Related to that, in practical work you frequently have to apply filtering operations to different Haver frame types which are easy to accomplish using pandas. Lists of codes that differ in casing would make these operations unnecessarily complicated.

Finally, `Haver.hdm.inputframes()`, which can receive a series code list or a group code list as input and uses it for its return value, leaves the casing from the input list untouched.

Terminology

Databases that are maintained by Haver Analytics are referred to (interchangeably) as '*regular* databases', '*regular* DLX databases', or '*regular* Haver databases'. They are exclusively maintained by Haver Analytics. You cannot modify regular databases in any way. Databases that you can modify using the HDM family of functions of the Haver package for Python, or through the Excel HDM add-in, are referred to as 'HDM databases'. All databases are based on Haver's DLX technology, so the term 'DLX databases' refers to any Haver database, be it regular or an HDM one.

DLX Direct

If your institution subscribes to the *DLX Direct* service, please note that several HDM functions require you to set the DLX Direct mode to 'off'. This is because the said HDM functions query data either from HDM databases or from requlars local databases. These operations would fail if the actual query were performed remotely on DLX Direct databases.

5.1.7 Where to Go from Here

During the course of learning about HDM data creation, you will have to refer to three locations of databases: the regular database directory, one or more directories that contain your HDM databases, and the directory of the example HDM database. Read *Haver.hdm.path* for an exposition on how to access and switch between these locations.

A fundamental concept that you need to know about are series groups. They will be explained in the next section. Next, skim through section *HDM Input-Frames* and go back to it for details as needed. Then give *Haver.hdm.create* a thorough read and try out its examples. At this point you will have come across a number of other function names. You can then choose to either advance further by reading their help entries, or you can opt for learning from more examples. Section *Additional Examples* provides them. You can also find examples at the end of each function help entry.

For the purpose of trying out series creation and updating, function *Haver.hdm.exampledata* greatly speeds up the generation of valid input data, so we recommend using it for learning purposes. Just keep in mind that later on when you do serious data management work, you must take charge of some additional settings that `Haver.hdm.exampledata()` handled automatically for you.

5.1.8 Differences to HDM in Excel

In Python, you can (as in Excel) update the time series data, but you cannot change the metadata of a series. Whenever you want to do this, you have to replace the entire series.

The Python HDM functions do not make use of the HDM.ini file. Therefore, you must supply the location of the HDM database when creating or deleting series, either via function argument `path` or through *Haver.hdm.path*.

There is no function for HDM database creation in Python. You must do this in Excel.

Note that this does not mean that you cannot add your HDM databases to the DLX.ini file in order to have them listed automatically in the data directory of the Haver data browser DLXVG3. This is possible, of course. Please see the the Excel HDM User Manual for further instructions.

5.2 Series Categories and Groups

5.2.1 Series Groups in Regular and HDM Databases

Series in all DLX databases are organized in groups. Group names are always 3 characters wide and start with an upper case letter that is followed by two digits. Examples are "A07" and "N22".

Note: Group names are also referred to (interchangeably) as 'group codes'.

In your HDM databases, groups serve two purposes:

1. **DLXVG3 menu grouping:** For regular databases, the data diretory in DLXVG3 (left-hand side panel) shows specially tailored menus. For HDM databases, the panel displays group descriptions. Therefore, a sensible group structure will make for a better browsing experience when you look at your data in DLXVG3.
2. **Source description:** the metadata fields 'shortsource' and 'longsource' really pertain to groups, so if maintaining proper source information within HDM databases is important to you, you have to organize your series in groups according to data source.

Let's expand on each point in turn.

5.2.2 Series Groups and and the DLXVG Menu

The display of groups in DLXVG3 works as follows. Groups that end in "00" are top-level groups. Their links open a menu page to same-letter groups with numbers higher than "00". For example, groups "N01", "N02", and "N53" will be accessible via the top-level link for group "N00".

Note: Top-level groups are also (and preferably) called *categories*. For ease of reference, both categories and groups are sometimes referred to as just 'groups'.

Categories do not contain any series. Categories contain series groups. Groups, in turn, contain series. They do not contain further subgroups. This yields a nesting structure of all databases that goes: categories - groups - series. Both categories and groups have a description property that you can set, the main purpose of which is to improve browsing in DLXVG3.

Regular Haver databases have a feature that HDM databases do not have: specially tailored menus for browsing. Nevertheless, regular Haver databases also have an *underlying* group structure, which is displayed in DLXVG3 when hitting CTRL+G (hit CTRL+G again to switch back to the regular data directory).

As mentioned above, when looking at the group structure, navigation is done using the (top-level) category and (second-level) group descriptions as well as the (third-level) series labels. Looking at the group structure of a regular database may give you some ideas of how you can structure your own groups. Be aware, however, that there are group criteria that apply to regular databases but not to HDM databases. For example, all series in a group are typically updated at the same time in regular databases. Update time may not be a criterion to structure your own data.

Category and group descriptions are set via the *group data field* 'description'. Series labels are set via the *description field* of an Input-Meta-DataFrame. For details, see the next section and *Haver.hdm.create*.

5.2.3 Series Groups and Source Information

An important location where source information pops up is in DLXVG3 graphs. The Haver default graph template places the source information in the bottom left corner of the graph, displayed as 'Source: *full source information* / Haver', where *full source information* contains the same information as the *metadata field* 'longsource'.

You can customize any part of the source display in DLXVG3. For example, you can omit the ' / Haver' part from being shown. Moreover, you can define templates that apply your desired source display automatically so you do not have to repeat the same graph modifications for each series. You simply assign a pre-defined graph template to the graph.

For help on how to perform any of the changes suggested above, see the online help in DLXVG3. You can change the source information display under the 'Misc' category in the section for formatting graphs. The section on managing templates provides all details for adding, changing, and applying graph templates.

Hint: It may be useful for you to define one or more graph templates in DLXVG3 for your HDM series. For example, such a template could replace the default source word 'Source' by 'Source (HDM)' and set the source display to 'Full' (which then omits the ' / Haver' part). If the source information is not important to you, you can define a template that omits it altogether.

What does all of this mean for HDM group creation? As indicated above, the source information that you can set via HDM corresponds to the *metadata fields* 'shortsource' and 'longsource' (referred to in DLXVG3 graph templates as 'short' and 'full'). The important point to grasp now is that these are not set by individual series, but by group (via *group data fields* 'shortsource' and 'longsource'; see the next section). So, if it is important to you to display proper source information, this will impact your organization of groups. Suppose you have data on stock prices from source A and source B for companies in Brazil and Argentina, respectively. Then you could set up groups as follows:

X00 - Stock prices
X01 — Brazil (Source: A)
X02 — Argentina (Source: A)
X03 — Brazil (Source: B)
X04 — Argentina (Source: B)

5.2.4 Further Remarks

Groups in Code Examples

Unless there is a specific need to demonstrate how different categories work, code examples use categories 'E00' ('E' as in 'Example') and 'X00 ('X' as in 'eXample'). 'E00' contains pre-existing content (groups and series), while 'X00' is, in the original state of the example database, an empty category to be filled with content in the example sections.

Whenever a code example wants to demonstrate something for which the group structure is secondary, it uses the `autogroup=True` option of `Haver.hdm.create()`. Then groups are automatically created when adding series to a database, and group creation statements do not distract from the main feature to be demonstrated.

5.3 HDM Input-Frames

5.3.1 Overview

All content to be created has to be supplied via pandas DataFrames that fullfil specific criteria. For example, only specific sets of column names are allowed, there are restrictions on the data types that each column can have, and so forth.

There are three different Input-Frames: One for group creation (called **Group-DataFrame**) and two for series creation (called **Input-DataFrame** and **Input-Meta-DataFrame**). All types of Input-Frames can be generated using *Haver.hdm.inputframes*. To some extent this function also helps to add data content to the frames, but in many cases you will have to issue additional statements in order to populate the Input-Frames with data values.

5.3.2 Group-DataFrame

Group-DataFrames have four columns corresponding to the four pieces of information that exist for each group: the group code (name), the group description, the short source, and the long source. The following table lists the DataFrame columns, their data types, and value restrictions:

Field name	pandas dtype	Value restrictions
name	object (str)	length must be 3 chars; first char is letter, chars 2-3 are numbers; examples: 'N00', 'Z07'; duplicates are not allowed; automatically converted to upper case
description	object (str)	max. 80 chars
shortsource	object (str)	max. 8 chars
longsource	object (str)	max. 48 chars

Warning: Codes are automatically converted to upper case. Haver databases only hold upper case group codes.

Note: The group description field is called 'descript**ion**'. The series description field in Haver-Meta-DataFrames and in Input-Meta-DataFrames is called 'descript**or**'.

Group-DataFrames are used both as input and output. You supply them to `Haver.hdm.create()` as input, and query them via `Haver.hdm.groups()`.

When used for group creation, the only required field (i.e. the only field that needs to be filled with values) is 'name'. Field 'description' is not strictly required but must be filled with sensible values if one wants to create an informative menu structure in DLXVG3.

For rows that pertain to categories (group names with digits '00'), information entered in fields 'shortsource' and 'longsource' is ignored. Categories do not carry source information.

5.3.3 (Series) Input-Frame-Pair

In order to create one or more series in an HDM database, you have to supply the data as well as the metadata. Both pieces of information are supplied in the form of pandas DataFrames. The two Input-Frames are almost identical to the frames you receive when you execute a `Haver.data()` query with the `rtype='2tuple'` argument; that is, they are almost identical to a *Haver DataFrame* plus a *Haver Meta-DataFrame*.

Since they are slightly different and in some respects have stricter requirements, we give them different names. The time series data for series creation is held in an Input-DataFrame and the metadata in an Input-Meta-DataFrame. Together, they are simply referred to as an **Input-Frame-Pair**. For series creation, the Input-Frame-Pair must be specified as a function argument in the form of a 2-element Python tuple. Note that the order in which the frames appear in the tuple matters: It must always be an Input-DataFrame followed by an Input-Meta-DataFrame. For updating time series data of existing series, the required input argument is (solely) an Input-DataFrame.

Input-DataFrame

Input-DataFrames hold the time series data. They must fullfil the following requirements:

- The vertical axis index must be a pandas `PeriodIndex` or a pandas `DatetimeIndex`, sorted in ascending order. We recommend using a `PeriodIndex`.
- Only certain pandas frequencies map into Haver frequencies.

  Use 'B', 'M', 'Q', 'A' for Haver's (business) daily, monthly, quarterly and annual frequencies. Use one of 'W-MON', 'W-TUE', ..., 'W-SUN' for weekly series.

  'Q' and 'A' are aliases for the more precise strings 'Q-DEC' and 'A-DEC', (quarter/year ending in December) which is how they are recorded in pandas PeriodIndexes. For weekly series you have to specify the controlling-day-of-week, which is typically the day on which the data are released or refer to in another way. Frequency 'W' is an alias for 'W-SUN'.

  Other frequencies are not allowed.

  Since a DataFrame holds data of one particular frequency, you can only create series of the same frequency with a single call to `Haver.hdm.create()`. If you want to create series that have different frequencies, you have to call the function several times.
- No periods may be omitted. For example, if the first and last period of the index are 2011Q1 and 2011Q4, the DataFrame must contain all 4 implied periods. It may not contain duplicate periods.
- Different minimum starting years apply to different frequencies. Daily, weekly, and monthly DataFrames may not start before 1921, quaterly ones not before 1920, and annual ones not before 1901.
- The column labels must consist of simple Haver codes. Duplicate codes are not allowed.
- All columns must be of a numeric dtype, i.e. either int or float.
- Each column must have at least one valid data point (i.e. not `numpy.nan`).
- A pandas `Series` is also allowed as long as a conversion into a DataFrame via `pandas.DataFrame()` satisfies the above criteria.

If you want to create data series that do not stem from a DLX / HDM database, the safest and most convenient way to construct an Input-DataFrame is to generate a skeleton frame using `Haver.hdm.inputframes()` and then add data values to it. You can also opt to generate the Input-DataFrame yourself, but it is then your responsibility to create a DataFrame that is in line with the above requirements. If you violate any requirement, you will receive error messages.

If you transfer series from a DLX / HDM database, the DataFrame returned by *Haver.data* can immediately be used for series creation.

Caution: Haver databases can only store values reliably if they have at most 8 significant digits.

For example, if your Input-Frame contains the data point 134,030,455, it cannot be reliably stored as such in a Haver database. In such a situation, we recommend to cut precision. You can store the data point as 13,403,045 in combination with a magnitude=1 setting. This means that the numbers of the series are to be interpreted "in tens". Effectively, you record the value as 134,030,450 - you drop the information in the last digit. Often you may be willing to sacrifice even more precision in favor of a more sensible magnitude setting. For example, suppose that the series in question is measured in dollars. Then it may seem sensible to trade off a bigger loss in precision for a more customary magnitude of 3: You record the data as 134,030, measured in thousands of dollars.

The limit of 8 significant digits also means that you have to set the level of decimal precision accordingly. For example, if you want to store a series containing a number that has an integer part of 40,325 (5 digits), the maximum conforming setting for the decimal precision is 3.

Warning: You are responsible for complying with the precision rules stated above. If you violate them you will not receive warnings or error messages from HDM functions. Your stored data will suffer from inaccuracies after the 8th significant digit.

Important: Haver's encoding for missing values is the number **1e9**.

Frequently you do not have to know about Haver's missing value encoding. You can just use the familiar `numpy.nan`. There are some cases, however, where you have to use 1e9 if you want to set existing database values to missing; see section *Setting Database Values to Missing*.

Note: Data input larger than 1e9 in absolute value will result in an error.

Another storage limit concerns the maximum possible length of a Haver series. There is an upper limit of 2100 periods that an individual series can cover. For daily and weekly series, this maximum is extended to 18900 periods. This limit does not imply that your Input-DataFrame cannot have more time periods. It means that every single column in the Input-DataFrame, when trimmed by `numpy.nan`s at the beginning and the end, must not exceed a time span longer than the maximum allowed time periods. Otherwise an error occurs. These limits are very unlikely to be binding for your work. However, some more words need to be said about series limits and daily and weekly series, which is done in section *Long Daily and Weekly Series*.

Input-Meta-DataFrame

An Input-Meta-DataFrame is very similar to a *Haver Meta-DataFrame*. It consists of a subset of the *Metadata Fields* of a Meta-DataFrame. They are:

Field name	pandas dtype	Value restrictions
code	object (str)	must start with letter; only letters and numbers; max. 8 chars (7 chars for daily/weekly); duplicates are not allowed; automatically converted to upper case
descriptor	object (str)	max. 80 chars; content of last parentheses is displayed in DLXVG3 in the subheading (typically the units of the series)
magnitude	numpy.int8	0-16
decprecision	numpy.int8	0-6
diftype	numpy.int8	0 (all DLXG3 functions allowed) or 1 (restricted set of functions) or 2 (automatically set by DLX)
aggtype	object (str)	one of 'AVG', 'SUM', 'EOP', 'NDF', 'NST'
datatype	object (str)	one of 'US$', 'LocCur', 'US$/LC', 'LC/US$', 'INDEX', 'Units', '%', 'RATIO' or 'SCALAR'
group	object (str)	length must be 3 chars; first char is letter, chars 2-3 are any numbers except 00; examples: 'N07', 'Z93'
geography1	object (str)	max. 7 chars
geography2	object (str)	max. 7 chars

Warning: Codes are automatically converted to upper case. Haver databases only hold upper case series codes.

Note: The series description is called 'decript**or**'. The group description field in Group-DataFrames is called 'descript**ion**'.

Similarly, you can supply the values for the fields 'aggtype', 'datatype', and 'group' in any casing you want. They will automatically get converted to the casing shown in the above table.

Note the specification for the series descriptor: The content of the subheading, which typically specifies the units of the series, must be placed in parentheses.

The 'diftype' determines which functions you can apply to series in DLXVG3. A value of 0 indicates no restrictions. A value of 1 indicates that the series contains non-positive values and hence disables functions like percentage changes. A value of 2 automatically sets the diftype to 0 or 1 as appropriate, based on the series data.

Why are some of the fields of a Haver-Meta-DataFrame excluded from an Input-Meta-DataFrame? This is because the time series data of the Input-DataFrame already implies some pieces of information that would only be repeated in metadata fields: The Input-DataFrame determines the frequency and the start and end date of each series (and, by implication, the number of observations). To avoid ambiguity and confusion about which information source takes precedence, fields 'frequency', 'startdate', 'enddate', and 'numobs' are not included in the Input-Meta-DataFrame. Think in terms of an Input-DataFrame that holds time series information and that is complemented by an Input-Meta-DataFrame that holds only fields that truly add new information.

Fields that are required for series creation are 'code', 'magnitude', 'decprecision', 'diftype', 'aggtype', 'datatype', and 'group'. Optional fields are 'descriptor', 'geography1' and 'geography2'. Even though 'descriptor' is not required, we recommend that you always supply this information. Section *Series Descriptors and Group Descriptions in DLXVG3* tells you why this is important and shows a minimalist but quick way of doing it.

5.3.4 Additional Remarks

Input-Frames Cannot Be Series

pandas sometimes converts DataFrames to series if an indexing operation results in a single row or column. You cannot use such a result as an Input-Frame. If you try to do so, you will receive an error in most cases.

To illustrate the problem and a solution, we quickly generate an example Input-Frame-Pair using `Haver.hdm.exampledata()`:

```
(ind, inmd) = Haver.hdm.exampledata(codes=3)
print( ind )
print( inmd )
```

Indexing down to a single row or column returns a pandas series:

```
print( ind.iloc[:, 0 ] )
print( type(ind.iloc[:, 0 ]) )
print( inmd.iloc[ 0 ,:] )
```

which you cannot use as Input-Frames. The solution is to use list operators when indexing:

```
print( ind.iloc[:, [0] ] )
print( type(ind.iloc[:, [0] ]) )
print( inmd.iloc[ [0] ,:] )
```

in which case pandas preserves the DataFrame data type.

Filtering Operations

Section *Filtering Operations* and the examples section of *Haver Meta-DataFrame* already discussed filtering operations for Haver Meta-DataFrames. These filtering operations become even more relevant for HDM Input-Frames.

Section *Additional Examples* provides examples of the usefulness of these operations.

5.4 HDM Error Handling

5.4.1 General Principles

All content modification functions performs comprehensive checks of the validity of the input data. If any inconsistency is found an error occurs, accompanied by an informative error message. The error message is informative in the sense that it should give you a good idea of where to look for the offending specification. For example, if you want to create series and specify an invalid 'aggtype' in the metadata, the error message will tell you that there is an invalid aggtype somewhere. You will, however, have to find the code(s) with the invalid aggtype by yourself. This is easily done by comparing the list of valid aggtypes from the package documentation to the ones occuring in the Input-Meta-DataFrame.

Multiple inconsistencies will not be listed simultaneously. Rather, content modification functions error out on encountering the first problem. Once that is fixed up, they will stop at the second misspecification, and so forth. This means that fixing up Input-Frames with multiple errors is an iterative process.

There are, however, exceptions to the above behavior. They aim to make fixing up content modification requests easier. First, if series that should not exist do exist - and vice versa - you will receive a *Haver ErrorReport dictionary* with which you are already familiar from the Haver data query functions. Secondly, for errors that would be difficult or tedious to fix up on your own, a HDM InputErrorReport dictionary, to be discussed below, is returned. It gives you access to lists of problematic codes.

Important: The detection of invalid input data and the ensuing error or InputErrorReport generation takes place without any content modification. You will not be left with partially modified content. This is a great advantage, since partially created content can quickly lead to confusion about the current state of the database.[3]

Lastly, in the unlikely case that all of the above checks fail to detect data input problems, an HDM InputAPIErrorReport dictionary is returned.

Since HDM content modification functions only operate on a single database at a time, codes returned in the dictionary lists of any ErrorReport are *simple Haver codes*. This is different from data and metadata queries, which can operate on multiple databases at once. Here the code lists returned consist of full Haver codes.

The two new ErrorReport dictionaries are discussed in more detail now.

5.4.2 InputErrorReport Dictionary

HDM InputErrorReport dictionaries contain general information about the content modification request (keys 'database', 'databasepath', and 'querydateandtime') as well as a nested dictionary (key 'codelists') that in turn has keys for different error types (keys 'frequencymismatch', 'serieslength', and 'seriesconcatenation'). With these keys you can access lists of codes that are offending. 'frequencymismatch' code entries occur when you try to update a series with a DataFrame of a different frequency. 'serieslength' code entries indicate that you tried to update a series beyond the maximum number of observations that a series can hold. Lastly, 'seriesconcatenation' can occur for all content modification functions. The error code list is related to daily and weekly series that are longer than 2100 observations. For more information on this point, see section *Long Daily and Weekly Series*.

5.4.3 InputAPIErrorReport Dictionary

The InputAPIErrorReport dictionary is meant as a last safeguard when other higher-level checks fail. It is unlikely that you will encounter this type of dictionary.

Its structure is identical to the one of a regular InputErrorReport (see above), except for the codelist keys, since the errors it reports are different and comprehensive. For example, the code list under key 'HDME_DATA_LEN' will tell you that you tried to update the listed series beyond the maximum number of observations that a series can hold. Since it is unlikely that you will see this type of dictionary, the numerous other keys shall not be listed here. However, one error type, 'HDME_TIME_EXPIRED', should not go unmentioned: It means that the license for the HDM tool has expired.

The important thing to understand about InputAPIErrorReports is that the principle of no-partial-content-modification no longer holds. That is, once you receive an InputAPIErrorReport, it may well be the case that your content modification request succeeded for some codes but not for others. Which part went through and which part failed can be gathered from the dictionary code lists. This behavior contrasts with the regular InputErrorReports where no-partial-content-modification is guaranteed.

[3] 'Partial content creation' refers to the situation were some series (or groups) get modified but others do not. This is possible with InputAPIErrorReports (which you should never receive) but not with InputErrorReports. 'Partial content modification' does not refer to the partial application of the input data for a single series. This never occurs. Either a series is modified according to the full input specifications, or it is not.

5.5 Haver.hdm.create

`Haver.hdm.create`(*inputframes*, *database*, *path=None*, *replace=False*, *queryframes=False*, *autogroups=False*)

Description

`Haver.hdm.create()` creates new groups and series in an HDM database, or overwrites existing ones.

Syntax

```
Haver.hdm.create(inputframes, database, [...])
er = Haver.hdm.create(inputframes, database, [...])
```

5.5.1 Arguments

inputframes
[tuple] A 2-element tuple containing an Input-Frame-Pair, or a Group-DataFrame. See *HDM Input-Frames* for the relevant definitions.

database
[str] The name of the database.

path
[str] The path to the HDM database. It must be specified as a full path, i.e. not as a path relative to the working directory. If `path` is not supplied, the HDM path setting from `Haver.hdm.path()` is checked next. If the path information cannot be retrieved from one of these two sources, an error occurs.

replace
[bool] Specifies how to handle content that already exists. Default behavior is to check for any existing groups/series before creating new ones. If some of the groups/series in the input data already exist, an error occurs before any new content is created. Under `replace=True`, any existing groups/series are overwritten with the input data, without any further warning.

queryframes
[bool] Set this to `True` whenever you transfer a series from an existing database. If set to `True`, the usual requirements of Input-Frames are relaxed to allow for features that DataFrames from Haver queries have. For example, an Input-Meta-DataFrame normally does not allow for a field 'datetimemod'. Under `queryframes=True`, this field is simply ignored.

Applies to series creation only.

autogroups
[bool] If set to `True`, automatically creates any non-existing groups (and corresponding categories, if necessary) that are specified in the 'group' field of the Input-Meta-DataFrame. Neither descriptions nor source information will be set for the automatically created groups and categories. Use this feature only if the group structure is of no concern for your purposes and you are sure that you are not damaging any existing groupings in the database.

Applies to series creation only.

5.5.2 Returns

None
if no content creation conflict occurs.

Haver ErrorReport dictionary
If groups or series to be created already exist and `replace=False`, an *Haver ErrorReport dictionary* is returned. The value of its key 'codesfound' contains a list of the relevant codes.

InputErrorReport dictionary
If there are violations to requirements for *long daily and weekly* series creation, a *InputErrorReport Dictionary* is returned. The value of its key 'seriesconcatenation' contains a list of the relevant codes.

5.5.3 Notes

(Cascading Creation of Higher-Level Content)

Creation of a group will cause creation of the top-level category that contains the group. The category created will not have a description set.

The attempt to create a series whose group (as specified in the metadata field 'group') does not exist will, by default, *not* cause creation of a containing group and category. Instead, an error will occur. For automatic group creation to happen, you must use option `autogroups=True`. If used, the groups (and possibly categories) so created will not have descriptions, and the groups will not have source information.

(Preservation of Lower-Level Content)

Modifying a category leaves groups and series within the category untouched.

Modifying a group leaves series within the group untouched.

(Creating Long Daily and Weekly Series)

Create *long daily and weekly series* through their base codes only. Inluding historical component series along with the base code in the Input-Frame-Pair will cause an InputErrorReport dictionary to be issued.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

5.5.4 Handling of Invalid Input Data

For a general discussion of error handling common to all content modification functions, see section *HDM Error Handling*. The following remarks discuss behavior specific to `Haver.hdm.create()`.

If `replace=False` (the default), the routine checks whether any of the input codes already exist in the database. If this is the case, no new content will be created. Rather, a *Haver ErrorReport dictionary* will be returned with the list of problematic codes (i.e. codes that already exist) saved as a list under the key 'codesfound'. Likewise, the key 'codesnotfound' will contain input codes that do not yet exist and did not prevent content creation. These are the only two keys of the Haver ErrorReport dictionary that `Haver.hdm.create()` ever populates. You can use these lists to fix up the Input-Frames.

Under `replace==True`, no existence check is performed. If any content already exists, it will get replaced by the input data.

5.5.5 Series Descriptors and Group Descriptions in DLXVG3

DLXVG3 behaves differently for series descriptors and group descriptions that are not set. If a group does not have a description, DLXVG3 menus display the group code in place of the group description. By contrast, series that do not have a descriptor set are not listed in the DLXVG3 menus via their series code. Instead, a blank spot is displayed. If the series that you create should be accessible nicely via DLXVG3 but if you do not want to invest the time for generating proper series descriptors, we recommend that at the very least you explicitly set series codes as series descriptors. This can be done easily. For example:

```
(ind, inmd) = Haver.hdm.inputframes(content='pair', startdate='2010-01-01',
              periods=3, frequency='m', codes=3)
inmd.descriptor = inmd.code
print( inmd )
Haver.hdm.create((ind, inmd), [...])
```

5.5.6 Transferring Weekly Series

When you want to transfer multiple weekly series at once (or otherwise want to use queried weekly data for series creation) you must be careful not to change the reference-day-of-week. This is because a query of multiple series that differ in their reference-day-of-week will result in a single DataFrame. This DataFrame can have only a single frequency, of course. Consequently, information on the different reference-days-of-week is lost. The rules according to which a reference-day-of-week is picked for the DataFrame are detailed in section *Daily and Weekly Queries*.

In this situation, a full information-preserving procedure is the following:

1. Query the metadata of the weekly series using the `decode=False` option. This will leave the frequencies in field 'frequency' in the original Haver encoding, which allows you to distinguish between different weekly frequencies. They run from 51 (weekly-Monday) to 57 (weekly-Sunday). Numeric values for all Haver frequencies are listed in section *Metadata Fields*.
2. Divide the full list of weekly codes according to the different numeric frequency levels.
3. Run a query on each sublist *separately* using the `rtype=2tuple` option and use the returned DataFrames for the creation of series.

5.5.7 Examples

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_create')
```

As a first example, we will transfer series from Haver example (regular) databases to the example (HDM) database. We start out transferring a daily series to an existing group in the example database. Then we generate three new groups for holding data in three different frequencies. Finally, we will populate the new groups with series, again by transferring data from example databases.

The first task is the transfer of a daily series to an existing group. We query 'haverd:fxtwotp' (an effective exchange rate), and put it into group 'E01'. In general, whenever we want to transfer data, we use the return type '2tuple' in order to have time series data and metadata in a convenient format for further processing. We get the data by:

```
(ind, inmd) = Haver.data('haverd:fxtwotp', rtype='2tuple')
```

inmd is a Haver-Meta-DataFrame. It consists of just one row since we only queried one series. A necessary change that we need to make to the metadata is the group field: We need to specify the destination group. In many cases the destination group is different from the source group. The source group in the HAVERD database is 'R03' but we would like to add the series to group 'E01' in the HDM example database, since 'E01' already contains similar exchange rate series. In addition, we opt to change the series code to 'EFFEX':

```
inmd.group = 'E01'
inmd.name  = 'effex'
Haver.hdm.create((ind, inmd), 'hdmex', queryframes=True)
```

This is all that we need to do for a series transfer. Note the `queryframes=True` option. Whenever we create series based on a `Haver.data()` query, we have to use this option.

Next, we would like to transfer series of three different frequencies. Each frequency receives its own destination group. We first need to create the groups. In order to do so we use the helper function *Haver.hdm.inputframes* which gives us a skeleton Group-DataFrame, which we will then populate:

```
gd = Haver.hdm.inputframes('groups', 3, {'shortsource':'TRANSFER',
     'longsource':'Transferred from example databases'})
gd.name        = ['x01', 'x02', 'x03']
gd.description = ['Monthly', 'Quarterly', 'Annual']
Haver.hdm.create(gd, 'hdmex')
```

Note that `Haver.hdm.create()` is used for both series and group creation. We now query the series that we would like to transfer, assign the proper group code, and add them to the databases.

```
ex_codes = ['FTBS3', 'FTBS4W', 'C', 'CD', 'IFT', 'IU']
(ind, inmd) = Haver.data(ex_codes, database='havermqa', rtype='2t')
```

We queried two codes per frequency. The correct group assignment is:

```
inmd.group = ['X01', 'X01', 'X02', 'X02', 'X03', 'X03']
Haver.hdm.create((ind, inmd), 'hdmex', queryframes=True)
```

This completes the first exercise.

Next we generate content that does not stem from regular Haver databases. To this end, we simulate daily stock price data and then generate annual series of highs and lows. We first create dedicated groups again.

```
gd = Haver.hdm.inputframes('groups', 2, {'short':'SIM',
     'long':'Simulated / made-up data'})
gd.name        = ['x04', 'x05']
gd.description = ['Made-up stock price data, daily',
                  'Made-up stock price data, annual']
Haver.hdm.create(gd, 'hdmex')
```

Next we use `Haver.hdm.inputframes()` again to generate an *Input-DataFrame* that contains the time series data. The statements below create a frame that contains three simulated stock price series in business daily frequency.

```
np.random.seed(123456)
ex_values = np.random.randn(2100, 3).cumsum(0) + 100
ind = Haver.hdm.inputframes('data', startdate='2001-01-01', periods=2100,
```

(continues on next page)

(continued from previous page)

```
      frequency='B', codes=['stkbrz', 'stkeur','stkus'], datavalues=ex_values)
ind.plot()
```

We need to add metadata. During yet another call to `Haver.hdm.inputframes()` we supply information for the required metadata fields

```
inmd = Haver.hdm.inputframes('meta', numrows=ind.shape[1],
       fieldvalues={'code':ind.columns.tolist(), 'mag':0, 'dec':4, 'dif':0,
       'agg':'avg', 'datatype':'index', 'group':'x04'})
print( inmd )
```

We also fill in some optional information. Series descriptors are not mandatory, but we highly recommend to always add them.

```
inmd.descriptor = ['Stock prices [Made-up!]: Brazil (2001-12=100)', \
                   'Stock prices [Made-up!]: Europe (2001-12=100)', \
                   'Stock prices [Made-up!]: USA (2001-12=100)']
```

Note that we specified the subheading (the units) in parentheses and used brackets to keep a label part in the main heading. See section *Input-Meta-DataFrame* for more details. We also choose to set geographical information.

```
inmd.geography1 = ['223', '023', '111']
```

The input data are now complete for series creation

```
Haver.hdm.create((ind, inmd), 'hdmex')
```

We no longer used the `queryframes=True` option since we used input data that did not come from a Haver query.

We now generate series that contain annual highs and lows. The data are:

```
ind_hl = pd.concat((ind.resample('A').max() , ind.resample('A').min()), axis=1)
ind_hl.columns = ['stkbrzh', 'stkeurh','stkush', \
                  'stkbrzl', 'stkeurl','stkusl'  ]
```

The metadata are:

```
inmd_hl = Haver.hdm.inputframes('meta', ind_hl.shape[1],
          {'code':ind_hl.columns.tolist(), 'agg':'nst', 'datat':'index',
          'gr':'x01', 'mag':0, 'dec':4, 'dif':0})
ex_desc = ['Stock Prices [Made-up!]: ' + country + ': Annual ' + hl \
           for hl in ['High', 'Low'] \
           for country in ['Brazil', 'Europe', 'US']]
inmd_hl.descriptor   = ex_desc
inmd_hl.geography1   = ['223', '223', '023', '023', '111', '111']
```

Series creation:

```
Haver.hdm.create((ind_hl, inmd_hl), 'hdmex')
```

This concludes the introductory examples of database content creation.

```
>>> Haver.path('restore')
>>> Haver.hdm.path('restore')
```

5.6 Haver.hdm.delete

Haver.hdm.**delete**(*content*, *codes*, *database*, *path=None*, *skip=False*)

Description

Haver.hdm.delete() deletes groups and series from HDM databases.

Syntax

```
Haver.hdm.delete(content, codes, database, path=None, skip=False)
er = Haver.hdm.delete(content, codes, database, path=None, skip=False)
```

5.6.1 Arguments

content
[str] One of 'groups' or 'series', or any abbreviation thereof. *content='groups'* deletes categories and groups. *content='series'* deletes series.

codes
[str or tuple of str or list of str] One or more simple Haver codes (i.e. pure Haver series names without 'database:' part) or one or more group codes. Note that HDM codes are not case-sensitive: all supplied group or series codes will be converted to upper-case ones.

database
[str] The name of the database.

path
[str] The path to the HDM database. It must be specified as a full path, i.e. not as a path relative to the working directory. If path is not supplied, the HDM path setting from Haver.hdm.path() is checked next. If the path information cannot be retrieved from one of these two sources, an error occurs.

skip
[bool] Specifies how to handle groups or series that do not exist. Default behavior is to check for any non-existing codes before attempting to delete them. If one or more of the codes in the input data do not exist, an error occurs, before any content is deleted. Under skip=True, any non-existing codes are ignored, without any further warning.

5.6.2 Returns

None
if no content deletion conflict occurs.

Haver ErrorReport dictionary
If groups or series to be deleted do not exist and skip=False, an *Haver ErrorReport dictionary* is returned. The value of its key 'codesnotfound' contains a list of the relevant codes.

InputErrorReport dictionary
An *InputErrorReport Dictionary* provides information on input data prolems with regards to *long daily and weekly series*.

5.6.3 Notes

(Cascading Deletion of Lower-Level Content)

Deletion of categories will cause deletion of all groups contained within it.

Deletion of groups will cause deletion of all series contained within it.

> **Warning:** There is no safety check for the existence of lower-level content. If you accidentially specify a deletion statement for a category or a group, all of its content will be deleted, without any further warning.

The following deletes all content from a database (here called 'MYHDMDB')

```
>>> gd = Haver.hdm.groups('MYHDMDB')
>>> Haver.hdm.delete('groups', gd, 'MYHDMDB')
```

(Preservation of Higher-Level Content)

Deletion of series leaves the containing group untouched. A group will continue to exist even if all of its series get deleted.

Deletion of groups leaves the containing category untouched. A category will continue to exist even if all of its groups get deleted.

(Deleting Long Daily and Weekly Series)

Delete *long daily and weekly series* through their base codes only. Inluding historical component series along with the base code in the input series list will cause an InputErrorReport dictionary to be issued.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

5.6.4 Handling of Invalid Input Data

For a general discussion of error handling common to all content modification functions, see section *HDM Error Handling*. The following remarks discuss behavior specific to `Haver.hdm.delete()`.

If `skip=False` (the default), the routine checks whether any of the input codes do not exist in the database. If this is the case, no content will be deleted. Rather, an *Haver ErrorReport dictionary* will be returned with the list of problematic codes (i.e. codes that do not exist) saved as a list under the key 'codesnotfound'. Likewise, the key 'codesfound' will contain input codes that do exist and did not prevent content deletion. These are the only two keys of the Haver ErrorReport dictionary that `Haver.hdm.delete()` ever populates. You can use these lists to fix up the input code list.

Under `skip=True`, no existence check is performed. If any content does not exist, it is simply ignored as an input.

5.6.5 Examples

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('reset')
>>> Haver.hdm.exampledb('prep_delete')
```

Content deletion is very easy. Just remember that the first argument specifies whether you want to delete groups or series.

We first confirm the existence of some series by querying their metadata. Then we delete two of them.

```
Haver.metadata(['stkeurh', 'stkeurl', 'stkeur'], database='hdmex')
Haver.hdm.delete('series', ['stkeurh', 'stkeurl'], 'hdmex')
```

Since some of the series no longer exist, the data query now returns a Haver ErrorReport dictionary:

```
Haver.metadata(['stkeurh', 'stkeurl', 'stkeur'], database='hdmex')
```

We repeat the same steps for groups.

```
Haver.hdm.groups('hdmex')['name'].tolist()
Haver.hdm.delete('groups', ['X04', 'X05'], 'hdmex')
```

The groups and all series in it no longer exist:

```
Haver.hdm.groups('hdmex')['name'].tolist()
```

Deletion of categories:

```
Haver.hdm.delete('groups', 'X00', 'hdmex')
```

The category and all groups and series within it no longer exist:

```
Haver.hdm.groups('hdmex').loc[:,'name'].tolist()
```

You can erase an entire database by querying the groups contained in it and then using the group list in a `Haver.hdm.delete()` statement.

```
Haver.hdm.exampledb('prep_delete')
gd = Haver.hdm.groups('hdmex')
Haver.hdm.delete('groups', gd.name.tolist(), 'hdmex')
```

You may encounter situations in which you do not know whether database content exists, but you would like to make sure that it does not exist. In these cases, it is convenient to use a `Haver.hdm.delete()` statement in conjunction with its `skip=True` option. If the content exists, it gets deleted. If it does not exist, the deletion instruction is ignored and no error is generated.

```
Haver.hdm.delete('groups', ['X01', 'X02'], 'hdmex', skip=True)
```

```
>>> Haver.path('restore')
>>> Haver.hdm.path('restore')
```

5.7 Haver.hdm.update

Haver.hdm.**update**(*inputdataframe*, *database*, *path=None*, *nocheck=False*)

Description

Haver.hdm.update() updates data of time series in an HDM database. Existing data values are overwritten, if necessary.

Syntax

```
Haver.hdm.update(inputdataframe, database, path=None, nocheck=False)
er = Haver.hdm.update(inputdataframe, database, path=None, nocheck=False)
```

5.7.1 Arguments

inputdataframe
[Input-DataFrame] A Haver Input-DataFrame. See *Input-DataFrame* for the relevant definition.

database
[str] The name of the database.

path
[str] The path to the HDM database. It must be specified as a full path, i.e. not as a path relative to the working directory. If path is not supplied, the HDM path setting from Haver.hdm.path() is checked next. If the path information cannot be retrieved from one of these two sources, an error occurs.

nocheck
[bool] Omits many checks on the Input-DataFrame and series to be updated in order to save execution time. Use this option only if you are an experienced HDM user. Default: False.

5.7.2 Returns

None
if no content modification conflict occurs.

Haver ErrorReport dictionary
If groups or series to be updated do not exist, a *Haver ErrorReport dictionary* is returned. The value of its key 'codesnotfound' contains a list of the relevant codes.

InputErrorReport dictionary
An *InputErrorReport Dictionary* provides information on input data prolems with regards to *long daily and weekly series*, frequency mismatches, and time series length limits.

5.7.3 Notes

(Scope)

`Haver.hdm.update()` modifies the time series data of a Haver series. Note that it only modifies the data itself. It cannot modify any metadata. If that is your goal, you have to re-create the series.

You can update data values of any time period that is in line with Haver's series length limits. The modifications can be applied to periods before a series' starting period or after a series' ending period, or any time in between.

(Setting Database Values to Missing and the Meaning of numpy.nan in Input-DataFrames)

Missing values (`numpy.nan`) in Input-DataFrames for a particular series are treated differently depending on where they occur. If they occur at the beginning or at the end of the series, they are ignored: Corresponding values in the database are left unchanged. However, missing values in the middle of a series are taken to indicate that database values, if they exist, should be set to missing. To give an example, suppose your annual Input-DataFrame spans 2010-2018 and contains several series. One of these series only has (non-`numpy.nan`) data for years 2011, 2014, and 2016. For this series, values for years 2010, 2017 and 2018 in the database are left as they are (if they exist), and likewise for any existing values before 2010 or after 2018. However, any existing values for 2012, 2013, and 2015 are set to missing in the database.

If you would like to treat missing values at the beginning and end of a series differently and have them set corresponding database values to missing, you can replace all `numpy.nan`s in the Input-DataFrame by the number 1e9, which is Haver's encoding for a missing value. Substituting any `numpy.nan`s in the middle of time series by 1e9 has no effect, since these observations set to missing in the databases in any case.

Important: `np.nan`s set database values to missing in most cases. They do not do so if they occur at the beginning and/or end of series in your Input-DataFrame. The number **1e9**, Haver's encoding for missing values, *always* sets corresponding database values to missing.

(Updating Long Daily and Weekly Series)

Update *long daily and weekly series* through their base codes only. Inluding historical component series along with the base code in the Input-DataFrame will cause an InputErrorReport dictionary to be issued.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

5.7.4 The `nocheck` Option

If you are an experienced user and are reasonably certain that your update request will execute without error and/or you are certain that you can deal with partially created content, you can employ `nocheck==True` in order to skip consistency tests of the input data versus the database. This may speed up the data update. If any inconsistencies exist, you will receive an *InputAPIErrorReport Dictionary*.

5.7.5 Setting All Values of a Series to Missing

It is possible to set all data values of a series to missing and create an empty series. However, keep in mind that you will not be able to query the data of empty series, since `Haver.data()` drops such series from the query. You will be able to query their metadata, however. In Haver-Meta-DataFrames, you recognize empty series either by a value of 0 for 'numobs'; or by the start date being one period after the end date.

In some instances, it may make sense to set all data to missing temporarily. For example, you may want to make sure that you have removed all old data before updating with new data. In order to set a data series to missing, you can query its data, set all data in the Haver-DataFrame to the value 1e9 (Haver's encoding for a missing value), and use the DataFrame as an Input-DataFrame for `Haver.hdm.update()`. In generic form, this looks like:

```
import numpy as np
Haver.path('hdm')
ind = Haver.data('mydb:myseries')
ind.iloc[:,:] = 1e9
Haver.hdm.update(ind, 'mydb')
Haver.path('restore')
```

5.7.6 Examples

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_update')
```

We would like to update the values of four weekly series:

```
codes = ['lic', 'licfe', 'farbn', 'farmn']
print( Haver.metadata(codes, 'hdmex').loc[:,
            ['code', 'startdate', 'enddate', 'frequency']] )
```

In order to generate some data for the update, we simple take the last four observations of each series and append them to the series:

```
ind = Haver.data(codes=codes, database='hdmex', startdate='2012-02-03')
print( ind )
ind.index = ind.index + 4
print( ind )
```

However, our update attempt fails:

```
Haver.hdm.update(ind, 'hdmex')
```

Even though all codes are of weekly frequency, the returned InputErrorReport dictionary lists database codes FARBN and FARMN as not being of the same frequency as the Input-DataFrame. The frequency of the Input-DataFrame is:

```
print( ind.index.freqstr )
```

The reference day-of-week is Saturday. As illustrated in the examples section of *Haver.metadata*, we can check of reference day-of-week of the series in the database using the `decode=False` option.

```
print( Haver.metadata(codes, 'hdmex', decode=False).loc[:,
           ['code', 'startdate', 'enddate', 'frequency']] )
```

The values of the numeric frequency encodings 56 and 53 can be looked up in the description of the *Metadata Fields* of a Haver-Meta-DataFrame. There we gather that they stand for weekly-Saturday and weekly-Wednesday, respectively. Since an Input-DataFrame cannot have multiple frequencies, we have to split the update statement in two. The update for weekly-Saturday is easy:

```
ind_sat = ind.loc[:,['lic', 'licfe']]
Haver.hdm.update(ind_sat, 'hdmex')
```

The update for weekly-Wednesday requires a bit of care. First, we have to set the frequency correctly:

```
ind_wed = ind.loc[:,['farbn', 'farmn']]
print( ind_wed )
ind_wed = ind_wed.asfreq('W-WED')
print( ind_wed )
```

Comparing the dates of the Input-DataFrame to the database, it turns out that we have to push all periods back by 1:

```
ind_wed.index = ind_wed.index - 1
print( ind_wed )
Haver.hdm.update(ind_wed, 'hdmex')
```

Executing the original data query shows that all values have been appended to the four series:

```
ind_new = Haver.data(codes=codes, database='hdmex', startdate='2012-02-03')
print( ind_new )
```

```
>>> Haver.path('restore')
>>> Haver.hdm.path('restore')
```

5.8 Haver.hdm.inputframes

Haver.hdm.**inputframes**(*content*, *numrows=None*, *fieldvalues={}*, ***, *frequency=None*, *datavalues=None*, *startdate=None*, *enddate=None*, *periods=None*, *codes=None*)

Description

`Haver.hdm.inputframes()` creates different kinds of empty Input-Frames: empty *Group-DataFrame*s, empty *Input-Meta-DataFrame*s, empty *Input-DataFrame*s, or empty *(Series) Input-Frame-Pair*s. Once values have been placed into the fields of these Input-Frames, they can serve as input to `Haver.hdm.create()` and `Haver.hdm.update()`. Depending on which Input-Frame should be created, `Haver.hdm.inputframes()` accepts different arguments.

Syntax

```
gd   = Haver.hdm.inputframes(content='groups',
              numrows=None, fieldvalues=dict())
inmd = Haver.hdm.inputframes(content='metadata',
              numrows=None, fieldvalues=dict())
ind  = Haver.hdm.inputframes(content='data', freq=None, datavalues=None,
              startdate=None, enddate=None, periods=None, codes=None)
inf  = Haver.hdm.inputframes(content='pair'     , [...])
```

5.8.1 Arguments

content
[str] One of 'groups', 'metadata', 'data', 'pair', or any abbreviation thereof. Determines which Input-Frame is created.

numrows
[int] [used for: Group-DataFrame, Input-Meta-DataFrame] Number of empty rows that the empty frame should have.

fieldvalues
[dict] [used for: Group-DataFrame, Input-Meta-DataFrame] Assigns values to the Input-Frame. The dictionary's keys can be any of the field names that the input frame has, or any unambiguous abbrevations thereof. Values can be either strings or integers or sequence types. The latter must have a number of elements that is consistent with the number of rows of the frame to be created.

The `fieldvalues()` dict is sometimes referred to as the 'initialization dictionary'.

See section *Populating the Input-Meta-DataFrame* for examples of usage.

frequency
[str] [used for: Input-DataFrame] One of the frequency strings listed under *Input-DataFrame*.

You can also use some of the frequency strings from `Haver.data()` ('monthly', 'quarterly', 'annual'/'yearly'), or any abbreviations thereof. 'daily' and 'weekly' are not allowed. 'daily' is not allowed to emphasize that Haver's 'daily' is 'business daily' in a pandas context. You must use the pandas frequency string 'B'. For weekly data you must use the pandas specification that contains the controlling-day-of-week.

datavalues
[various] [used for: Input-DataFrame] Specifies the values that the Input-DataFrame should be filled with. The frame will contain `numpy.nan`s if you do not use this parameter.

`datavalues` accepts anything that the pandas DataFrame constructor can use for its `data` parameter. The dimensions of the argument must comply with the number of time periods (rows) and number of series codes (columns).

Two convenience features that are geared mostly towards your learning phase of HDM functionality are the following: Supplying a numeric scalar will fill the Input-Frame with a constant value. Supplying the string 'random' will fill the Input-Frame with (normal) random values using `numpy.random.randn()`.

startdate
[str or datetime.date] [used for: Input-DataFrame] Specifies the starting date of the data. If is is specified as str, it is converted to a `datetime.date` object using the input format yyyy-MM-dd, e.g. ('2005-12-31).

enddate
[str or datetime.date] [used for: Input-DataFrame] Specifies the ending date of the data. Analogous comments as for `startdate` apply.

periods
[int] [used for: Input-DataFrame] Determines the number of periods that the data cover.

codes
[str or tuple of str or list of str or int] [used for: Input-DataFrame] Code names to be used as column names in the Input-DataFrame.

You can also supply an integer value in the range 1-999. It determines the number of series of the Input-Frame. Column names will be generic: 'ser001, 'ser002', ..., up to the value specified.

5.8.2 Returns

pandas.core.frame.DataFrame

The returned frame is either a Group-DataFrame, a Input-Meta-DataFrame, a Input-DataFrame, or a Input-Frame-Pair.

5.8.3 Notes

(Syntax Detail)

Parameters `numrows` and `fieldvalues` are for the creation of Group-DataFrames and Input-Meta-DataFrames. All other parameters are for the creation of Input-DataFrames.

The creation of Input-Frame-Pairs (`content='pair'`) accepts any of the parameters. However, the `numrows` parameter is ignored if supplied. Moreover, series names supplied via `codes` take precedence over series names that may be supplied via the `code` key of the `fieldvalues` dict.

For Input-DataFrame creation (`content='data'`), exactly two of the parameters `startdate`, `enddate`, and `periods` must be supplied. The third parameter is implied.

`content='data'` and `content='pair'` require that you specify all arguments as keyword (not positional) arguments. That is, you may not use

```
Haver.hdm.inputframes(content='data', None, {'mag':0}, 'annual', [...])
```

Use instead

```
Haver.hdm.inputframes(content='data', fieldvalues={'mag':0},
    frequency='annual', [...])
```

(A Convenient Wrapper Function)

Haver.hdm.exampledata is a convenient wrapper function for `Haver.hdm.inputframes()`. It allows you to quickly generate Input-Frame-Pairs without going through the trouble of specifying all required information, using instead a set of default values that can be modified. The function's purpose is to make it easier for new HDM users to 'play' with content modification statements without having to spend a lot of time on manually putting together Input-Frame-Pairs. `Haver.hdm.exampledata()` is not suitable for actual data management work. For this, always use `Haver.hdm.inputframes()`.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

5.8.4 Populating the Input-Meta-DataFrame

You can populate the Input-Frames with data at two stages. The first stage is by using parameters `codes` and, in particular, `fieldvalues` (the 'initialization dictionary') during the `Haver.hdm.inputframes()` call. The second stage is after the call to `Haver.hdm.inputframes()`.

For example, when creating a Group-DataFrame with three rows, you can use the initialization dictionary as in

```
gd = Haver.hdm.inputframes(content='groups', numrows=3, fieldvalues=
     {'name':['K00', 'K01', 'K03'], 'short':'FRB'})
```

An example statement for generating a Input-Meta-DataFrame with five rows and partially filled-in data is

```
inmd = Haver.hdm.inputframes(content='meta', numrows=5, fieldvalues=
       {'mag':6, 'dif':0, 'dec':2, 'agg':'sum', 'gr':'K01'})
```

The initialization dictionary is an optional argument and each of its permitted keys is optional. That is, you can choose which fields you want to fill with data. You can set a column to a fixed value (`{'dif':0}`) or you can use lists of values. As an example for the latter, suppose that you already have an Input-DataFrame named `ind` and that you want to create a companion Input-Meta-DataFrame. Then you can code

```
inmd = Haver.hdm.inputframes('met', numrows=ind.shape[1],
       fieldvalues={'code':ind.columns.tolist(), [...]})
```

Any string fields that are not specified in the `Haver.hdm.inputframes()` call are set to zero-length strings. The default for 'diftype' is 2, which indicates that the appropriate diftype setting should be determined automatically. The default for the other two numeric fields ('magnitude' and 'decprecision') is -1, a value that is not permitted by `Haver.hdm.create()`. You will have to change these fields to different values before creating series. The default value of -1 was chosen because pandas/numpy integer data types cannot hold `numpy.nan`s.

Any field that you specify in the initialization dict will undergo a check for admissible values.

The second stage at which you can fill values into the Input-Frames is after the call to `Haver.hdm.inputframes()`. You can do so in any way you like. The only condition is that all required fields of the input frames are populated with admissible values before the Input-Frame (Pair) is used in `Haver.hdm.create()` or `Haver.hdm.update()`. Among other things, this means that any numeric fields that have been automatically set to -1 at stage one ('magnitude', 'decprecision') must be changed at stage two.

Section *HDM Input-Frames* lists admissible values for required and optional group and metadata fields.

5.9 Haver.hdm.path

`Haver.hdm.`**`path`**(*vset=None*, *save=False*)

Description

Set, query, and save the path to HDM databases. This path setting serves as the default for modifications of HDM databases by *Haver.hdm.create* and *Haver.hdm.delete*, and for database access by *Haver.hdm.groups*.

`Haver.hdm.path()` is different from *Haver.path*. The latter sets and queries the path to regular Haver databases. Its setting is relevant for Haver queries.

Syntax

```
P = Haver.hdm.path()
Haver.hdm.path(vset)
Haver.hdm.path(*keyword*)
Haver.hdm.path(save=True)
```

5.9.1 Arguments

vset
: [str] Specifies the default directory to look for HDM databases.

save
: [bool] Save current path setting to file.

5.9.2 Returns

str
: When no argument is supplied, a str that holds the current setting of the HDM database path. If no path is set, an empty str is returned.

5.9.3 Notes

(Syntax Detail)

`Haver.hdm.path()` returns the current HDM path setting.

`Haver.hdm.path(vset)` sets the database path to *vset*. *vset* must be supplied as an absolute file path. If the supplied database path does not exist, an error occurs.

`Haver.hdm.path(*keyword*)` takes different actions. *keyword* can be one of 'examples', 'restore', and 'saved', or any abbreviation thereof.

- 'examples' sets the database path to the example HDM database directory that comes with the Haver package. Before doing so, `Haver.hdm.path()` will store the existing setting. You can re-enable this setting by issuing `Haver.hdm.path('restore')`.
- 'restore' restores the database path setting that was in place when `Haver.hdm.path('examples')` was invoked.
- 'saved' sets the path according to a previously saved state.

`Haver.hdm.path(save=True)` saves the current path setting to the Haver package settings file. This setting gets automatically restored upon import of the Haver package, or when reloading the Haver package. Note: Saving the HDM path affects all users that access a particular package installation.

(Package Import)

When the Haver package is imported, it checks whether a HDM path had been saved and, if that is the case, restores it.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

5.9.4 HDM Path Setting Details

Preservation of the Path Setting between Python Sessions

If you want the current HDM path setting to be automatically restored after package import, simply issue `Haver.hdm.path(save=True)`.

Warning: Be aware that, in an environment where multiple users share the same Haver package installation, saving a path setting affects all users. This is not true for any other path operations: They only affect a single user.

Running 'Examples' Sections of HDM Help Entries

At the top of the examples section in each HDM help entry the HDM databasepath is set to point to the example databases:

```
>>> Haver.hdm.path('examples').
```

At the end of each examples section, the previous setting is restored:

```
>>> Haver.hdm.path('restore').
```

If you re-set the Haver database path manually after issuing `Haver.hdm.path('examples')`, a call to `Haver.hdm.path('restore')` will not change this setting anymore.

5.9.5 Location of Databases

While regular databases are typically all located in the same directory, you can have your own HDM databases in any number of directories. For quick access to the different directories, all HDM functions that need to access HDM databases have an argument `path` that takes precedence before the HDM path setting that is in place.

If you think about storing your HDM databases in different directories, keep in mind that individual query-related function calls can only address a set of databases that reside in the same directory. If you want to query HDM databases that reside in different directories and you want to have the data in a single DataFrame, you have to manually join Haver-DataFrames from multiple queries.

The fact that it is not possible to query data from databases located in different directories also implies the following: If you store your HDM databases in a directory that is different from the directory that contains the regular Haver databases, you cannot query data from both types of databases at the same time. Again, in that situation you have to join Haver-DataFrames manually if you want to have data from both types of databases in a single DataFrame.

5.9.6 Path Lookup

All HDM functions that access HDM databases have an optional argument `path`. If it is not set, all HDM functions check whether a default HDM path has been set via `Haver.hdm.path()`. The HDM functions will look for this default, but nowhere else. That is, they will error out if no path argument is supplied and no default HDM path has been set. In particular, they will neither check the regular Haver path nor the current working directory.

If your HDM databases are in the same directory as the regular databases, just set the HDM path to the same directory as the Haver path:

```
>>> Haver.hdm.path(Haver.path())
```

5.9.7 Relation to `Haver.path()`

In general, operations for setting the regular Haver database path and operations for setting the HDM path are completely separate. `Haver.hdm.path()` only operates on the HDM path. `Haver.path()` only operates on the Haver path. In particular, this is true for setting the path to example databases and for saving a path for automatic restoration in a new Python session. With respect to these two features, `Haver.hdm.path()` works in full analogy to what has been said for Haver.path() in section *Path Setting Details*.

`Haver.path()` also takes a special argument value `vset='hdm'` which sets the Haver path to the HDM path for easy access to data stored in HDM databases. The special argument value `vset='restore'` then restores the Haver path setting. Note that

```
hp = Haver.hdm.path()
Haver.path(hp)
```

is not completely the same as

```
Haver.path('hdm')
```

since only the latter allows you to restore the original path using

```
Haver.path('restore')
```

5.9.8 Examples

```
>>> Haver.hdm.path('examples')
>>> expath = Haver.hdm.path()
>>> Haver.hdm.path('restore')
>>> print( expath )
```

One difference between the path setting for regular databases and the path setting for HDM databases is that HDM functions accept an argument `path`, so the statement

```
print( Haver.hdm.groups('hdmex', path=expath)  )
```

generates the same output as

```
Haver.hdm.path('examples')
print( Haver.hdm.groups('hdmex') )
```

The path to regular databases can quickly be switched back and forth between regular and HDM database folders. Your path to regular databases is:

```
Haver.path('restore')
print( Haver.path() )
```

Setting the regular database path to the HDM path is done by:

```
Haver.path('hdm')
print( Haver.path() )
```

Now queries can access the databases that are available in the current HDM path. In the current situation the HDM path is identical to the path to the example databases. During your normal work this will of course not be the case, which is why the example code maintains the following subtle distinction: If an example needs to query data from a regular example database, it will be preceded by:

```
Haver.hdm.path('examples')
Haver.path('examples')
```

If an example needs to query data from the HDM example database, the introductory statements read:

```
Haver.hdm.path('examples')
Haver.path('hdm')
```

This will result in the same path settings. As mentioned, in your applied work the directories of regular and HDM databases may differ, so you do need different path settings for querying from regular databases and for querying from HDM databases.

If you have HDM databases in many different directories and you need to query data from them, you can always set the regular path explicitly, as in:

```
Haver.path('YOUR_HDM_PATH_1')
Haver.data([...])

Haver.path('YOUR_HDM_PATH_2')
Haver.data([...])
```

You can then revert to the path setting for regular Haver databases using

```
Haver.path('saved')
```

or

```
Haver.path('auto')
```

```
>>> Haver.path('restore')
>>> Haver.hdm.path('restore')
```

5.10 Haver.hdm.groups

Haver.hdm.**groups**(*database*, *path=None*)

Description

Haver.hdm.groups() returns all group information that is contained in a Haver database.

Syntax

```
gd = Haver.hdm.groups(database, path=None)
```

5.10.1 Arguments

database
[str] Name of the database.

path
[The directory in which the database resides.] The path to the HDM database. It must be specified as a full path, i.e. not as a path relative to the working directory. If path is not supplied, the HDM path setting from Haver.hdm.path() is checked next. If the path information cannot be retrieved from one of these two sources, an error occurs.

5.10.2 Returns

pandas.core.frame.DataFrame
The returned DataFrame is a *Group-DataFrame*. Each entry (row) is a category or group that exists in the database.

5.10.3 Notes

(Usage as Input-Frame)

You can use the Group-DataFrame returned by Haver.hdm.groups() directly as an Input-Frame for Haver.hdm.create(). This makes it easy to transfer individual groups or an entire group structure from one database to another.

5.10.4 Examples

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('reset')
```

The example data base has groups E00, E01, and X00 in its initial state:

```
print( Haver.hdm.groups('hdmex') )
```

When preparing the example database for different example sections, `Haver.hdm.exampledb()` creates additional groups:

```
Haver.hdm.exampledb('prep_delete')
print( Haver.hdm.groups('hdmex') )
```

As a precursor for more complex exercises in other sections of this guide, we now duplicate existing groups X01-X05 as groups X10-X14. To this end, we use a modified version of the returned Group-DataFrame as input to `Haver.hdm.create()`.

```
gd = Haver.hdm.groups('hdmex')
gd2 = gd.iloc[4:9,:].copy()
gd2.loc[:,'name'] = ['X10', 'X11', 'X12', 'X13', 'X14']
gd2.description = gd2.description + ' (DUP)'
print( gd2 )
Haver.hdm.create(gd2, 'hdmex')
```

The groups have been duplicated:

```
print( Haver.hdm.groups('hdmex') )
```

Section *Group Restructuring without and with Series* provides more complex examples of group modifications.

```
>>> Haver.hdm.path('restore')
>>> Haver.path('restore')
```

5.11 Haver.hdm.exampledb

`Haver.hdm.`**`exampledb`**(*action=None*)

Description

`Haver.hdm.exampledb()` assists in preparing the HDM example database according to the needs of a particular example section.

Syntax

```
kl = Haver.hdm.exampledb()
Haver.hdm.exampledb(*keyword*)
```

5.11.1 Returns

None
: if a function keyword is supplied.

list
: List of allowed keywords if no keyword is supplied.

5.11.2 Notes

(Setting the Haver Path)

You must set the Haver path to the examples databases before calling `Haver.hdm.exampledb()` using

```
>>> Haver.path('examples')
```

or using

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
```

This is because the function queries data from example databases. Do not forget to restore the Haver path afterwards:

```
>>> Haver.path('restore')
```

(Syntax Detail)

`Haver.hdm.exampledb(*keyword*)`

`Haver.hdm.exampledb(*keyword*)` takes different actions. Examples of *keyword* are: 'reset', 'preexist', 'prep_create', etc. This usage of the function requires that the regular path be set to the example databases.

- 'reset' resets the HDM example database to its original state.
- 'preexist' makes sure that the pre-existing content of the example database in its original state is available. It does not delete any additional content.
- 'prep_*' keywords prepare code examples. For example, after execution of `Haver.hdm.exampledb('prep_create')`, the statements of the examples section of `Haver.hdm.create()` will always execute without errors.

Important: If you are looking at a series of HDMEX in DLXVG3, `Haver.hdm.exampledb('reset')` may not work but instead result in a file permission error. In that case, point DLXVG3 to a series in another database and retry.

5.12 Haver.hdm.exampledata

`Haver.hdm.`**`exampledata`**(*codes=1*, *frequency='Q'*, *startdate='2010-01-01'*, *enddate=None*, *periods=12*, *datavalues='random'*, *fieldvalues=None*)

Description

`Haver.hdm.exampledata()` quickly creates Input-Frame-Pairs that can immediately be used for series creation. The Input-DataFrame of the Input-Frame-Pair can immediately be used for data updates. The purpose of this function is to make it easier for new HDM users to 'play' with content modification statements without having to spend a lot of time on manually putting together Input-Frame-Pairs.

By default, the Input-DataFrame contains random data. You should never use this function for serious data management work.

Syntax

```
inf = Haver.hdm.exampledata(codes=1, frequency='Q', startdate='2010-01-01',
            enddate=None, periods=12, datavalues='random', fieldvalues=None)
```

5.12.1 Arguments

all :
All parameters are identical to the same-named ones in *Haver.hdm.inputframes*.

5.12.2 Returns

pandas.core.frame.DataFrame

Input-Frame-Pair

5.12.3 Notes

(Default Values)

`Haver.hdm.exampledata()` is a short wrapper for `Haver.hdm.inputframes()`, specifically geared towards the fast creation of example Input-Frame-Pairs. As such, it sets many parameters that must be specified for valid input data to more or less arbitrary default values. You can selectively override these default values to quickly create input data that have the essential features you are looking for.

Metadata default values are not visible from the function signature. They are: 'magnitude'=0, 'decprecision'=0, 'diftype'=0, 'aggtype'='sum', 'datatype'='index', and 'group'='X99'. You can override them via the `fieldvalues` parameter.

The default behavior for the 'descriptor' field is to set it equal to 'code'. Section *Series Descriptors and Group Descriptions in DLXVG3* explains the rationale behind this.

Specifying `endingdate` will set `startdate=None`, unless you set `periods=None`.

(Haver Python PDF Reference)

If you are reading this help entry in the Python online help, you can find additional detail in the Haver Python PDF Reference, including examples.

5.12.4 Examples

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_exampledata')
```

Even without any argument, the function returns an Input-Frame-Pair that can immediately be fed into the database:

```
inf = Haver.hdm.exampledata()
print(inf[0])
print(inf[1])
Haver.hdm.create(inf, 'hdmex', autogroups=True)
```

We had to use `autogroups=True` in the `create()` statement since the default group of `Haver.hdm.exampledata()`, 'X99', did not exist in the database. We do not have to use the `autogroups` option in subsequent statements since the group has now been created.

When code names are not explicitly supplied, the default is to use 'SER###', with '###' being a 3-digit sequence number starting at '001'. When we use `Haver.hdm.exampledata()` repeatedly, we have to either specify new code names explicitly or use `replace=True` in `Haver.hdm.create()`.

```
inf2 = Haver.hdm.exampledata(codes=3, frequency='M', periods=8)
print(inf2[0])
print(inf2[1])
Haver.hdm.create(inf2, 'hdmex')
```

The previous `create()` request did not go through because we tried to create series 'SER001', which had already been created in our first `create()` request.

```
Haver.hdm.create(inf2, 'hdmex', replace=True)
```

You can also change metadata via the initialization dictionary (the `fieldvalues` parameter):

```
inf3 = Haver.hdm.exampledata(fieldvalues={'group':'X98',
       'datatype':'US$'}, frequency='B', codes='mydaily', periods=1000)
Haver.hdm.create(inf3, 'hdmex', autogroups=True)
```

Here we used `autogroups=True` again in the `create()` request since we specified a non-existing group.

```
>>> Haver.path('restore')
>>> Haver.hdm.path('restore')
```

5.13 Haver.hdm.apipath

Haver.hdm.**apipath**(*path=None*)

Description

Haver.hdm.apipath() sets a custom path to the HDM API DLL. It is only relevant in Python >=3.8 and if you cannot place your HDM API DLL into 'c:/windows'.

Syntax

```
P = Haver.hdm.apipath()
Haver.hdm.apipath(path)
```

5.13.1 Returns

str
When no argument is supplied, a str that holds the current setting of the HDM API DLL path. If no custom path is set, an empty str is returned.

5.13.2 Notes

(Setting the HDM API DLL Path)

This function is only relevant if you are working in Python version 3.8 and higher. If you cannot copy the HDM API DLL to its default location ('c:/windows'), you must set a custom path. Otherwise no HDM functionality will be available. Before you can set the path, you must copy the HDM API DLL to the target location.

(Preservation of the Path Setting between Python Sessions)

The path setting is preserved between Python settings, so you should only have to apply the setting once.

5.14 Long Daily and Weekly Series

5.14.1 Background: How Long Series Are Stored in Haver Databases

Important: There is an upper limit on the length of a Haver time series: It can span at most 2100 time periods (2100 business days, 2100 weeks, etc.). This corresponds to about (a little more than) 8 years of business daily data and 39 years of weekly data.

Important: The limit is not relevant for lower frequencies. What you read in this section only pertains to **daily** and **weekly** series.

For many daily series and for some weekly series, the above constraint ist binding. Haver databases deal with this limit in the following way: If a series covers more than 2100 time periods, additional history can be stored in auxiliary series that have the same base code but also have sequence numbers 2-9 appended to it.[4]

Important: Even though long daily and weekly series exists as separate series in the database, you never have to specify historical component series in queries or in content modification statements.

To give an example, as of this writing the DAILY database contains the following daily codes:

Series code	Years covered
FXCAN	2012-2019
FXCAN2	2004-2011
FXCAN3	1996-2003
FXCAN4	1988-1995
FXCAN5	1980-1987
FXCAN6	1972-1979
FXCAN7	1971-1971

The data query `Haver.data('daily:fxcan')` retrieves the entire history 1971-2019 in a single column of a HaverDataFrame.

Important: Whenever you perform a content modification operation (create/delete/update) on a long daily or weekly series, do so via the base code only. The historical segements will be modified as appropriate automatically.

Warning: Modifying historical component series directly will likely lead to unintended outcomes. Base code data queries will likely return incorrect data after the modification has been applied.

This is the reason why it is forbidden to include historical component series along with their base codes in content modification statements. For example, if you try to create daily or weekly series BASE and BASE2 in a single statement, you will receive an HDM *InputErrorReport Dictionary*. Its code list 'seriesconcatenation' will list the offending historical codes, in this case BASE2.

However, even though you do not need to manage historical component series by yourself, you do need to know about them for two other reasons:

1. **Series naming:** When you devise the codes of your own data series, you should be cautious when creating series that end in a number. For example, when creating two unrelated daily series BASE and BASE2, querying BASE will concatenate the data of both series, which is unintended. Moreover, if BASE2 is of a different frequency, say, monthly, querying BASE will result in an error. Note that it is not a problem if BASE is monthly and BASE2 is daily. Monthly and lower frequencies do not have historical components, so BASE2 is interpreted as an independent series.

   Since Haver codes can have at most 8 characters, base codes can have at most 7 characters in order to leave space for the digit of the history components.

2. **Series transfer across databases of all codes in a source database:** Series creation based on the code list returned by `Haver.metadata(database='MYDB')` will fail if database MYDB contains long series. This is because the historical component series are included in the metadata query, so your creation statement will receive

[4] This implies that the maximum number of observations for a long series (BASE, BASE2, …, BASE9) is 9 * 2100 = 18900 time periods, or about 72 years of daily data.

an input list that contains both base codes and corresponding historical codes, which is forbidden and results in an error. The solution is simply to use option `histcodes='drop'` in the metadata query.

History components must exist sequentially in the database. Suppose series BASE and BASE3 exist, but not BASE2. Then BASE and BASE3 are treated as independent series. If you would like to examine regular databases or your own HDM databases with respect to the existence of historical series, you can use options `histcodes='sequential'` and `histcodes='nonsequential'` of `Haver.metadata()`. They work as follows. Suppose that your HDM database contains codes BASE, BASE2, BASE3, BASE6 and BASE7. `histcodes='sequential'` will include codes BASE2 and BASE3 in the metadata query results. Since their numbering is sequential starting at 2, they are interpreted as proper historical codes. Option `histcodes='nonsequential'` will include BASE6 and BASE7. The sequential numbering has been interrupted at 4, so any code after that will not be interpreted as a historical code. Existence of such a series in an HDM database may go back to a conscious naming decision, but it may also point to a problem. For example, you may have inadvertently deleted the historical codes BASE4 and BASE5. In this case a query of code BASE would suffer from incomplete history.

Point 1. above mentioned the problem of invalid history chains, an example for which is the following: A database contains a monthly series called BASE3. Then a long daily series BASE, which has a historical component BASE2, is added to the database. In such a case, both data and metadata queries of code BASE will issue a Haver ErrorReport dictionary and list code BASE under 'metadataaccess'. Note that metadata queries of the entire database no longer work. Whenever you get a 'metadataaccess' error, think of invalid history chains as a potential culprit. You can troubleshoot such errors by comparing the codes with 'metadataaccess' to codes under 'codesfound'. If the 'codesfound' list contains codes that look like historical component series for codes under 'metadataaccess', query their metadata and check them for consistency.

Lastly, it needs to be mentioned how content modification functions compare series to be modified with what actually exists in the database. The basic rule is that they only check for the base code. For example, before trying to create a series `Haver.hdm.create()` checks whether the base code exists in a database. If it does, an error report is issued, unless option `replace=True` is used. Note that it is only the base code whose (non-)existence is checked: Any historical components are always created as necessary, and replace any series that may exist under the same name. Similarly, `Haver.hdm.update()` creates historical components as needed, without checking whether that will overwrite other series. Finally, `Haver.hdm.delete()` checks whether a series has historical components and automatically deletes them along with the base code.

5.14.2 Examples

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_longseries')
```

We draw up a long daily series using the convenience tool `Haver.hdm.exampledata()`

```
inf = Haver.hdm.exampledata(codes='long', frequency='B', enddate='2010-01-01',
      periods=5000, fieldvalues={'group':'X41'})
Haver.hdm.create(inf, 'hdmex', autogroups=True)
```

Note that the base code in the database captures the full series span:

```
hmd = Haver.metadata(database='hdmex')
print( hmd.loc[hmd.group=='X41',['code', 'startdate', 'enddate']] )
```

Historical series have automatically been created and placed in group 'Z99':

```
print( hmd.loc[hmd.group=='Z99',['code', 'startdate', 'enddate']] )
```

All operations are done through the base code. When updating data, all historical codes are reorganized automatically:

```
inf2 = Haver.hdm.exampledata(codes='long', frequency='B',
       startdate='2008-12-31', enddate='2020-01-01', periods=None)
ind = inf2[0]
Haver.hdm.update(ind, 'hdmex')
hmd = Haver.metadata(database='hdmex')
print( hmd.loc[hmd.group.isin(['X41', 'Z99']),
              ['code', 'startdate', 'enddate']] )
```

Deleting a series is invoked through the base code only, yet comprises any historical series parts that may exist automatically:

```
Haver.hdm.delete('series', 'long', 'hdmex')
hmd = Haver.metadata(database='hdmex')
print( hmd.loc[hmd.group.isin(['X41', 'Z99']),
              ['code', 'startdate', 'enddate']] )

>>> Haver.path('restore')
>>> Haver.hdm.path('restore')
```

5.15 Additional Examples

5.15.1 Filter Operations on Input-Frames

Please have a look at section *Filtering Operations* before reading on.

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_addex_filter_1')
```

Section *Haver.hdm.groups* already provided an example of how one can easily drop all category records from a GroupDataFrame that contains both category and group records. We will expand on this here. Suppose you are restructuring a database and want to delete all groups and series while preserving the categories. There are two easy ways to achieve this. The first one is to delete all database content and recreate all categories. *Haver.hdm.delete* showed how to delete an entire database:

```
gd = Haver.hdm.groups('hdmex')
Haver.hdm.delete('groups', gd.name.tolist(), 'hdmex')
```

We recreate the categories by keeping only the categories in gd and feeding it back into the database. The filter operation is:

```
gd2 = gd.copy()
gd2.index = gd.name.tolist()
gd2 = gd2.filter(regex='.00', axis=0)
print( gd2 )
Haver.hdm.create(gd2, 'hdmex')
```

Alternatively, you could have executed

```
gd3 = gd.loc[gd.name.str.contains('.00'),:]
print( gd3 )
Haver.hdm.create(gd3, 'hdmex')
```

The second way to delete everything but the categories from a database is to query all groups, drop all categories from the Group-DataFrame, and use it to delete all groups.

```
Haver.hdm.exampledb('prep_addex_filter_1')
gd4 = gd.loc[~gd.name.str.contains('.00'),:]
print( gd4 )
Haver.hdm.delete('groups', gd4.name.tolist(), 'hdmex')
```

Let's now look at a filter operation on an Input-Frame-Pair for series creation. Suppose you have put together an Input-Frame-Pair. We mimick this situation by simulating data:

```
Haver.hdm.exampledb('prep_addex_filter_2')

np.random.seed(123456)
ex_values = np.random.randn(120, 3).cumsum(0) + 100
ind = Haver.hdm.inputframes('data', startdate='2001-01-01', periods=120,
      frequency='M', codes=['simfx1', 'simfx2','simfx3'], datavalues=ex_values)
print( ind.head() )

inmd = Haver.hdm.inputframes('meta', numrows=ind.shape[1], fieldvalues=
       {'code':ind.columns.tolist(), 'mag':0, 'dif':0, 'dec':0,
       'agg':'avg', 'datatype':'index', 'group':'X20'})
inmd.descriptor = inmd.code
print( inmd )
```

In this example, we use two shortcuts. First, we use codes as series descriptors. Moreover, we will not bother with proper group creation. Instead, we use the `autogroups=True` option when creating the series in order to generate a group with a generic description.

Suppose now that, when trying to create the series, you notice that some of the codes you would like to create already exist. A Haver ErrorReport dictionary is returned:

```
Haver.hdm.create((ind,inmd), 'hdmex', autogroups=True)
```

Usually, one would examine whether the existing codes refer to the same concept and contain the same data as the 'new' ones, and if they do not, change the series codes of the new series. Here we have the case that the series do refer to the same concept.

The simplest solution for fixing the `Haver.hdm.create()` call would be to use the `replace=True` option. Then the existing series would then get deleted and recreated. Assume that we do not want to do this. This could be because we are not sure whether the new data are fully in line with the existing data, or because we do not want to change the time stamp of the series. Instead, we would like to separate new codes from existing codes in the Input-Frames.

The first step is to repeat the `Haver.hdm.create()` statement in order to capture the error dictionary's code lists:

```
her = Haver.hdm.create((ind,inmd), 'hdmex', autogroups=True)
```

We can use the code lists for filter operations:

```
inmd.index = inmd.code.tolist()

(ind_f, inmd_f) = (ind.loc[:,ex_f], inmd.loc[ex_f,:])
print( ind_f.head() )
print( inmd_f )

(ind_nf, inmd_nf) = (ind.loc[:,ex_nf], inmd.loc[ex_nf])
```

(continues on next page)

(continued from previous page)

```
print( ind_nf.head() )
print( inmd_nf )
```

Now we feed in the series that do not yet exist:

```
Haver.hdm.create((ind_nf,inmd_nf), 'hdmex', autogroups=True)
```

Note that the statements above are general. While we have operated on very small frames and code lists, the same statements would equally apply to frames and code lists of any size.

```
>>> Haver.hdm.path('restore')
>>> Haver.path('restore')
```

5.15.2 Group Restructuring without and with Series

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_addex_restruct_1')
```

Suppose you have the following groups in your HDM database and that all of these groups are empty (do not contain any series):

X31 : annual series
X32 : monthly series

Suppose that you would like to add a group for quarterly series. In addition, you would like to have the groups ordered in increasing order by frequency, and also assign higher group numbers because you would like to leave some space for other additions you have planned. You would like to have:

X35 : monthly series
X36 : quarterly series
X37 : annual series

First we have to re-assign group codes of the existing two groups. Then we have to add the additional group. The first task can be accomplished by querying the two groups, making the necessary changes to the Group-DataFrame, feeding it back into the database, and deleting the old groups.

```
ex_groups = ['X31', 'X32']
gd = Haver.hdm.groups('hdmex')
gd.index = gd.name.tolist()
gd = gd.loc[ex_groups,]
print( gd )
gd.name = ['X37', 'X35']
Haver.hdm.create(gd, 'hdmex')
```

We add the quarterly group:

```
gd_2 = Haver.hdm.inputframes('gr', 1)
gd_2.loc[0,:] = ['X36', 'quarterly series', 'qtrsrc', 'quarterly series source']
Haver.hdm.create(gd_2, 'hdmex')
```

To conclude, we delete the old groups:

```
Haver.hdm.delete('groups', ex_groups, 'hdmex')
```

Suppose now that we have the identical task as above, but let's assume in addition that the existing groups already contain series. The following statement sets up the database for this.

```
Haver.hdm.exampledb('prep_addex_restruct_2')
```

Creation of groups X35-X37 is as before:

```
Haver.hdm.create(gd  , 'hdmex')
Haver.hdm.create(gd_2, 'hdmex')
```

We now have to re-assign the series group parameter and re-create the series in the new groups before deleting the old groups. We first get a list of the relevant series:

```
hmd = Haver.metadata(database='hdmex')
hmd.index = hmd.group.tolist()
hmd = hmd.loc[['X31', 'X32'],:]
print( hmd )
```

Then we query the data for the relevant series, re-assing groups and create the series in the destination groups:

```
(ind, inmd) = Haver.data(hmd, rtype='2t')
print( inmd.group )
inmd.group = ['X37', 'X37', 'X35', 'X35']
Haver.hdm.create((ind,inmd), 'hdmex', queryframes=True, replace=True)
```

Note that we had to use the `replace=True` options since the series already exist in the database. Finally, we can delete the old groups and the series contained in them:

```
Haver.hdm.delete('groups', ex_groups, 'hdmex')
```

```
>>> Haver.hdm.path('restore')
>>> Haver.path('restore')
```

5.15.3 Weekly Series

```
>>> Haver.hdm.path('examples')
>>> Haver.path('hdm')
>>> Haver.hdm.exampledb('prep_addex_aggweekly')
```

In this example we look at creating weekly series. Weekly series are special in that they need a reference day-of-week. Here we convert simulated daily stock price series to weekly series by taking only data from Wednesdays and by taking only data from Fridays.

```
ex_codelist = ['stkbrz', 'stkeur','stkus']
ind = Haver.data(ex_codelist, 'hdmex')
```

First we select all Mondays and, separately, all Fridays (`dayofweek==0` below corresponds to Mondays).

```
ind_wed = ind.loc[ind.index.to_timestamp().dayofweek==2,:]
ind_fri = ind.loc[ind.index.to_timestamp().dayofweek==4,:]
```

We convert to a weekly frequency and rename the series codes.

```
ind_wed = ind_wed.asfreq('W-WED')
ind_fri = ind_fri.asfreq('W-FRI')

ind_wed = ind_wed.rename(columns = lambda x : x + 'w')
ind_fri = ind_fri.rename(columns = lambda x : x + 'f')
```

We need to add the metadata. There are different ways to set up and populate a Input-Meta-DataFrame. In the following, we generate a skeleton frame, query the metadata of the original series, copy the relevant fields into the Input-Frame, and then modify where necessary.

```
inmd_wed = Haver.hdm.inputframes('meta', numrows=ind_wed.shape[1],
          fieldvalues={'code':ind_wed.columns.tolist()})
hmd = Haver.metadata(ex_codelist, database='hdmex')

ex_copyfields = ['descriptor', 'magnitude', 'decprecision', 'diftype',
                 'aggtype', 'datatype', 'geography1']
inmd_wed.loc[:,ex_copyfields] = hmd.loc[:,ex_copyfields]
inmd_wed.descriptor = inmd_wed.descriptor + ' [WED]'
inmd_wed.group = 'X40'

inmd_fri = inmd_wed.copy()
inmd_fri.descriptor = inmd_fri.descriptor.str.replace('WED', 'FRI')
inmd_fri.code = ind_fri.columns.tolist()
```

We quickly generate a properly labeled group and then create the series:

```
gd = Haver.hdm.inputframes('gr', 1)
gd.loc[0,:] = ['X40', 'Simulated Data, Weekly Aggregation', 'SIM',
               'Simulated / made-up data']
Haver.hdm.create(gd, 'hdmex')

Haver.hdm.create((ind_wed, inmd_wed), 'hdmex')
Haver.hdm.create((ind_fri, inmd_fri), 'hdmex')
```

```
>>> Haver.hdm.path('restore')
>>> Haver.path('restore')
```

5.15.4 Creating Quick Archives

You can use HDM in Python in various ways to archive data. For example, you can use a certain category within a database to hold groups for archive data. For example, you could have the following group structure:

K00 - Labor market
K01 - Cross-country data
...
L00 - Business indicators
L01 - Surveys
...
Z00 - Backup / archive data
Z01 - Archive 2019-09-15
Z02 - Archive 2019-10-15
Z03 - Archive 2019-11-15
...

One caveat of the above approach is that you cannot archive the same series at multiple points in time, since series codes in a database must be unique. A solution to this would be to use one HDM database per archive operation.

In the context of these suggestions, please keep in mind that Haver Analytics provides separate tools for archiving full regular databases. Use these tools for this task. If your archiving demands are more fine-grained, the Haver Python package is an option.

5.15.5 Input-DataFrames with Missing Periods

```
>>> Haver.path('examples')
```

`Haver.hdm.create()` does not accept Input-DataFrames that have rows missing for one or more periods. Here we hint at how you can address this situation by fixing up the Input-DataFrame. Our example DataFrame is:

```
(ind_d, inmd_d) = Haver.data('haverd:fxtwb', rtype='2tuple',
                             startdate='2012-02-13')
print( ind_d )
```

```
ind_d_gap = ind_d.iloc[list(range(0,4)) + list(range(8, ind_d.shape[0])),:]
print( ind_d_gap )
```

`ind_d_gap` misses rows for dates between '2012-02-16' and '2012-02-23'. We use the `resample()` method to fill them back in. The filled-in rows contain `numpy.nans`

```
ind_d_filled = ind_d_gap.resample('B').first()
print( ind_d_filled )
```

Note that we used the frequency string `'B'` since we have a business daily frequency. `'D'` would fill in dates for weekends as well, which is not what we want. Instead of specifying the frequency string explicitly, you can plug in `ind_d_gap.index.freqstr`, which should always work.

`resample()` also works for all other relevant frequencies. To give an example for a monthly series:

```
(ind_m, inmd_m) = Haver.data('havermqa:fxtha', rtype='2tuple',
                             startdate='2010-07-01')
print( ind_m )
```

```
ind_m_gap = ind_m.iloc[list(range(0,4)) + list(range(8, ind_m.shape[0])),:]
print( ind_m_gap )
```

```
ind_m_filled = ind_m_gap.resample('M').first()
print( ind_m_filled )
```

An alternative strategy to using `resample()` is to generate an index that contains all periods and use it in a call to `index.reindex()`. Below we use `Haver.hdm.inputframes()` to get a hold of a contiguous index. The subsequent `reindex()` statement is deliberately cast in general terms so that it works for many situations.

```
import numpy as np
ind_jnk = Haver.hdm.inputframes('data',
             startdate=ind_m_gap.index.min().to_timestamp(),
             enddate  =ind_m_gap.index.max().to_timestamp(),
             frequency=ind_m_gap.index.freqstr,
             codes=1)
print( ind_jnk )
```

```
ind_m_filled2 = ind_m_gap.reindex(ind_jnk.index, fill_value=np.nan)
ind_m_filled2.equals(ind_m_filled)
```

For more details on these methods, see the pandas documentation.

```
>>> Haver.path('restore')
```

INDEX